UNIVERSIDAD NACIONAL DE INGENIERÍA – UNI

(DACTIC)

Dirección del Área de Conocimiento de Tecnologías de la Información y Comunicación

Carrera:

Ingeniería de Computación

Asignatura:

Algoritmización y estructura de datos (AED)

Tema: "MercadeoNica Sistema web para la gestión de ventas e inventario de comerciantes y emprendedores"

Elaborado por:

Br. Sofía Elienai Guevara Jarquín

Br. Franklin Osman Salina Rivas

Br. Harvin Ulises González Herrera

Grupo: 2V1-COM-S

Docente: Ing. Yahaira Andrea Altamirano Briceño

Managua, 02 de noviembre 2026

Índice

1. Introducción 5
2. Antecedentes 6
   2.1. Uso de los sistemas de información 6
   2.2. Origen de la propuesta MercadeoNica 7
   2.3. Antecedentes 7
3. Justificación 9
4. Objetivos 10
   4.1. Objetivo general 10
   4.2. Objetivos específicos 10
5. Requerimientos del sistema. 11
   5.1. Sistemas de Información 11
   5.2. Arquitectura Java Enterprise (Jakarta EE) 12
   5.3. Patrón MVC (Modelo-Vista-Controlador) 12
   5.4. Microsoft SQL Server 2022 12
   5.5. Metodología 13
      5.5.1. Enfoque Metodológico 13
      5.5.2. Fases del Proyecto 13
      5.5.3. Roles del Equipo SCRUM 14
      5.5.4. Herramientas Utilizadas 14
      5.5.5. Backlog del Producto (resumen) 14
   5.6. Desarrollo del sistema 15
      5.6.1. Arquitectura General 15
   5.7. Estructura de Clases Java 17
      5.7.1. Descripción de Tablas 17
      5.7.2. Restricciones de Integridad 18
      5.7.3. Procedimientos Almacenados Principales 19
      5.7.4. Triggers Implementados 20
      5.7.5. Estructura de los paquetes 21
   5.8. Módulos Desarrollados 23
      5.8.1. Módulo de Autenticación Unificada 23
      5.8.2. Módulo de Gestión de Clientes 23
      5.8.3. Módulo de Gestión de Vendedores 24
      5.8.4. Módulo de Catálogo de Productos 24
      5.8.5. Módulo de Carrito (Vendedor) 25

5.8.6. Módulo de Carrito (Cliente) 25
5.8.7. Módulo de Pedidos y Facturación 25
5.8.8. Módulo de Tarjetas de Crédito 26
5.8.9. Módulo de Reseñas 26
5.8.10. Módulo de Reportes (ADMIN) 27
5.8.11. Módulo de Auditoría 27
5.9. Pruebas 28
5.9.1. Tipos de Pruebas Realizadas 28
5.9.2. Casos de Prueba 28
5.9.3. Pruebas de Seguridad 29
5.9.4. Pruebas de Rendimiento 30
5.9.5. Pruebas de Usabilidad (SUS) 30
5.10. Resultados 31
5.10.1. Resultados Obtenidos 31
5.11. Métricas del Proyecto 31
5.12. Beneficios Obtenidos 32
6. Diagrama 33
7. Base de Datos SQL Server (OSF) 36
7.1. Diccionario de Datos 36
7.2. Restricciones de Integridad y Llaves. 38
7.3. Objetos Programables (Triggers y Procedimientos) 38
7.4. Script de la Base de Datos 39
8. Consultas SQL y Pruebas de DBearer 95
9. Conclusión 105
9.1. Lecciones aprendidas de la planificación 105
10. Recomendaciones 106
11. Bibliografía 107
12. Anexos 108
Login del Administrador, Vendedor y Cliente 108
A nivel de Administrador 108
Parnel de control 108
Visualización de Productos 109
Lista de clientes 109
Carrito de compras 110
Historial de pedido que realiza el administrador al cliente en general 111

Factura que se realiza a la hora de la compra parta el cliente 111
Historial de pedido “Pendientes” 112
Historial de pedido “Confirmados” 112
Historial de pedido “Confirmado” 112
Historial de pedido “Entregados” 112
Historial de pedido “Cancelado” 113
Visualización del Dashboard 113
Reporte del sistema 114
Visualización de vendedores 115
A nivel de Vendedor 116
Visualización del panel de control 116
Visualización de los productos 116
Visualización de los clientes 117
Visualización del carrito 118
Visualización del historial de pedidos de los clientes 119
Historial de pedidos “Pendiente” 119
Historial de pedidos “Confirmación” 119
Historial de pedidos “Entregados” 119
Historial de pedidos “Cancelados” 120
Visualización de la factura del cliente 120
A nivel de Cliente 121
Visualización del panel de control 121
Visualización de los productos 121
Visualización del carrito de compra 122
Visualización de la confirmación del pedido 122
Visualización del pedido del cliente que ha realizado. 122
Visualización las tarjeta de crédito que el cliente ingrese. 122

1. Introducción

El presente proyecto propone el desarrollo de MercadeoNica, un sistema web orientado a la gestión de productos, clientes, pedidos, ventas e inventario de comerciantes y emprendedores que ofrecen ropa, calzado, accesorios, artículos del hogar, etc. Su finalidad será organizar la información comercial y facilitar a las familias la consulta y solicitud de productos desde un teléfono celular o una computadora.

El sistema contará con una base de datos relacional para almacenar información de manera permanente. Se contemplarán funciones para registrar productos, actualizar precios y existencias, administrar clientes, gestionar pedidos y consultar reportes de ventas. Asimismo, se establecerán permisos diferenciados para el administrador y los vendedores, de modo que cada usuario acceda a las funciones correspondientes a su responsabilidad.

El desarrollo aplicará la programación orientada a objetos mediante clases como Producto, Cliente, Vendedor, Pedido y Detalle Pedido. Este enfoque permitirá agrupar los datos y comportamientos de cada entidad, siguiendo los conceptos de clases, objetos y encapsulamiento descritos por Oracle (s. f.-b). También se incorporarán estructuras dinámicas para administrar temporalmente los artículos del carrito y organizar el procesamiento de pedidos.

2. Antecedentes

2.1. Uso de los sistemas de información

Desde la década de 1960, los sistemas de información (SI) han evolucionado progresivamente desde simples procesadores de datos automatizados hasta complejas plataformas empresariales unificadas. En el contexto de las Pequeñas y Medianas Empresas (PYMES), un SI estructurado permite eliminar tareas operativas repetitivas, minimizar errores humanos, centralizar el flujo de información y ofrecer herramientas de análisis en tiempo real orientadas a la toma de decisiones estratégicas.

Los sistemas de información permiten recopilar, organizar, almacenar y consultar datos relacionados con las actividades de una organización. En el ámbito comercial, estas funciones se aplican al registro de productos, seguimiento de ventas, control de inventario y administración de clientes. Su utilidad depende de que respondan a los procesos del negocio y de que la información registrada sea correcta y se mantenga actualizada.

Una base de datos relacional permite vincular los registros de las distintas operaciones. Por ejemplo, un pedido puede relacionarse con el cliente que lo solicita y con los productos que lo integran. Las claves primarias y foráneas contribuyen a mantener la integridad de estas relaciones, evitando referencias a registros inexistentes (SQL Server).

Además del almacenamiento permanente, los programas necesitan organizar datos durante su ejecución. Para ello pueden utilizar estructuras dinámicas, como listas enlazadas y colas. La documentación de Oracle (s. f.-a) describe una implementación de lista doblemente enlazada que también admite operaciones de cola, lo cual ofrece una referencia técnica para el manejo de colecciones de elementos.

2.2. Origen de la propuesta MercadeoNica

MercadeoNica surge como una idea académica de mercado digital que busca reunir productos de diferentes comerciantes y emprendedores en un catálogo común. La propuesta se enfoca en ropa, calzado, accesorios, artículos del hogar, etc, dirigidos a familias que desean consultar opciones y realizar pedidos desde su celular.

El proyecto se plantea inicialmente para un grupo reducido de comercios, con el propósito de desarrollar y comprobar las funciones esenciales antes de ampliar su cobertura. La primera versión priorizará el registro de productos, la consulta del catálogo, la gestión de pedidos y el control de existencias.

Para cumplir con el requisito de aplicación en una empresa real, se deberá seleccionar un comercio participante y documentar su actividad, origen y procedimientos de trabajo. Mediante una entrevista y la observación de sus registros se determinará si utiliza cuadernos, hojas de cálculo, mensajes u otro sistema para administrar la información. Este diagnóstico permitirá identificar necesidades comprobadas y ajustar el alcance de MercadeoNica.

2.3. Antecedentes

Para fundamentar la concepción y desarrollo del sistema web MercadeoNica, se llevó a cabo una revisión bibliográfica de investigaciones y trabajos de graduación desarrollados en la Universidad Nacional de Ingeniería (UNI). Estos estudios abordan la automatización de procesos comerciales, el control de inventarios y el diseño de bases de datos relacionales, sirviendo como base conceptual y metodológica para este proyecto.

1. Antecedente Nacional Facturación y Control de Inventario

García Poveda y Porras Herrera (2022).

Desarrollo de sistema web para la gestión de los procesos de compraventa, facturación y control de inventario para la tienda Winnie's Menorquinas.

Los autores abordaron la problemática del manejo manual de información en negocios minoristas, lo que ocasionaba pérdidas de tiempo e imprecisiones en el stock. Para solucionar esto, diseñaron un sistema web bajo el marco de trabajo Ruby on Rails y el sistema gestor de bases de datos MySQL, sistematizando procesos de registro de clientes, gestión de proveedores, facturación y generación de indicadores financieros.

Aporte a la investigación

Esta tesis proporcionó las pautas fundamentales para estructurar la lógica de negocio de MercadeoNica en los procesos de compraventa y control estricto de entradas y salidas de mercancía. Asimismo, sirvió de modelo para establecer la relación entre los módulos de facturación y el catálogo de productos.

2. Antecedente Nacional, Arquitectura Web para Colectivos Comerciales

Centeno Ríos y Téllez Mendoza (2023).

Sistema web de gestión de ventas e inventario para el colectivo de MiPymes Corals.

La investigación se enfoca en resolver las dificultades operativas de un colectivo de pequeñas empresas que operan bajo el modelo de showroom. Se desarrolló una plataforma web accesible de forma remota que centraliza las ventas, administra el catálogo multisectorial de productos e integra la gestión individual de los colaboradores.

Aporte a la investigación

Este trabajo aportó elementos clave para el diseño arquitectónico del módulo de Gestión de Vendedores e inventarios centralizados en MercadeoNica. Permitió definir con precisión la asignación de roles, el seguimiento operativo de cada vendedor y la estructura requerida para registrar transacciones asociadas a múltiples colaboradores.

3. Antecedente Nacional Modelado de Datos e Integridad Referencial

Rivas Obregón y Aranda Barrera (2024).

Desarrollo de Sistema Web para la gestión de ventas, compras e inventario de la librería Mayisun.

El estudio se centró en la transición de un modelo tradicional en papel a un sistema web automatizado mediante la metodología en cascada. La solución fue desarrollada utilizando ASP.NET, C# y Microsoft SQL Server, logrando resolver deficiencias en la facturación y garantizando el acceso a reportes e información en tiempo real.

Aporte a la investigación

Constituyó la principal referencia para el diseño del modelo entidad-relación (E-R) y la implementación de la base de datos OSF en Microsoft SQL Server. De esta investigación se tomaron las mejores prácticas para estructurar las relaciones entre tablas de encabezado y detalle (orders e invoices), así como el uso de triggers y procedimientos almacenados para garantizar la integridad referencial y la auditoría del sistema.

3. Justificación

El desarrollo de MercadeoNica se justifica por la necesidad de disponer de información comercial organizada y accesible. Cuando los datos de productos, clientes y pedidos se encuentran separados, resulta más difícil verificar existencias, conocer el estado de una venta y preparar reportes. El proyecto buscará atender estas dificultades en el comercio participante, siempre que sean confirmadas durante el diagnóstico.

En el aspecto operativo, el sistema permitirá centralizar los registros y reducir la repetición de tareas. Los vendedores podrán actualizar su catálogo y consultar sus pedidos, mientras que el administrador podrá supervisar el funcionamiento general. Los clientes dispondrán de información sobre los productos y de un mecanismo para solicitar su compra.

La relación entre ventas e inventario tendrá especial importancia. Al confirmar una venta, el registro de la operación y la actualización de existencias deberán ejecutarse de forma coordinada. Las transacciones de base de datos permiten agrupar operaciones para que se completen conjuntamente o se cancelen si ocurre un error, lo que contribuye a conservar la consistencia de la información.

Desde el punto de vista académico, MercadeoNica permitirá aplicar los conocimientos de AED a un proceso comercial concreto. La programación orientada a objetos facilitará la distribución de responsabilidades entre clases; las estructuras dinámicas permitirán manipular colecciones de datos durante la ejecución; y la base de datos garantizará su persistencia.

Es necesario distinguir estos componentes: una lista o una cola organiza información en memoria, mientras que la base de datos conserva los registros entre sesiones. El proyecto integrará ambos recursos y explicará su función dentro del sistema.

4. Objetivos

4.1. Objetivo general

Desarrollar un sistema web denominado MercadeoNica para gestionar productos, clientes, pedidos, ventas e inventario de un comercio real participante, con capacidad de incorporar otros vendedores, mediante una base de datos relacional, estructuras dinámicas y programación orientada a objetos.

4.2. Objetivos específicos

1. Analizar los procedimientos de registro de productos, ventas e inventario del comercio participante para establecer los requisitos funcionales y las necesidades de información del sistema.

2. Diseñar una base de datos relacional que integre usuarios, vendedores, clientes, productos, pedidos, detalles de pedidos y movimientos de inventario, mediante claves y restricciones que mantengan la integridad de los registros.

3. Aplicar la programación orientada a objetos para distribuir las responsabilidades del sistema entre clases y separar la interfaz, las reglas del negocio y el acceso a los datos.

5. Requerimientos del sistema.

5.1. Sistemas de Información

Un **sistema de información (SI)** es un conjunto de componentes interrelacionados que recopilan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones, la coordinación y el control dentro de una organización. Según Laudon & Laudon (2020), los SI transforman datos en información útil mediante tres actividades básicas: **entrada**, **procesamiento** y **salida**.

En el contexto empresarial, los SI se clasifican en:

Tipo	Descripción	Ejemplo en MercadeoNica
TPS (Transaction Processing System)	Procesa transacciones diarias	Registro de pedidos y facturación
MIS (Management Information System)	Genera reportes gerenciales	Módulo de Reportes (solo ADMIN)
DSS (Decision Support System)	Apoya decisiones no estructuradas	Sugerencia de upgrade de categoría de cliente
EIS (Executive Information System)	Información para alta dirección	Dashboard de estadísticas generales

5.2. Arquitectura Java Enterprise (Jakarta EE)

Jakarta EE (anteriormente Java EE) es una plataforma de especificaciones para desarrollar aplicaciones empresariales en Java. MercadeoNica utiliza:

- **Servlets:** componentes del lado del servidor que manejan peticiones HTTP.
- **JSP (JavaServer Pages):** tecnología de vista que genera HTML dinámico.
- **JDBC:** API para conectarse a bases de datos relacionales.
- **JPA (opcional):** mapeo objeto-relacional (en este proyecto se usa JDBC directo con DAO).

5.3. Patrón MVC (Modelo-Vista-Controlador)

MercadeoNica implementa MVC de la siguiente manera:

Componente	Implementación
Modelo	Clases en com.mercadeonica.modelo (Cliente, Producto, Pedido, etc.)
Vista	Archivos JSP (*.jsp) con HTML + CSS + JavaScript
Controlador	Servlets en com.mercadeonica.servlet (ClienteServlet, ProductoServlet, etc.)

5.4. Microsoft SQL Server 2022

SQL Server es un **SGBD relacional** de Microsoft. Características usadas en MercadeoNica:

- **Triggers AFTER:** para auditoría y control de inventario.
- **Procedimientos almacenados:** para operaciones complejas (place_order_full, add_cliente_completo).
- **Funciones escalares:** FIND_PRODUCT_CATEGORY_ID.
- **OPENJSON:** para pasar arrays de productos como JSON.
- **Columnas calculadas PERSISTED:** subtotal en order_details.
- **Restricciones CHECK:** validación de email, ZIP, rol, estado.
- **Índices:** optimización de búsquedas por estado, ciudad, categoría.

Conceptos de Comercio Electrónico

MercadeoNica implementa varios conceptos de **e-commerce**:

- **Catálogo en línea:** productos visibles sin necesidad de registro.
- **Carrito de compras:** persistente en sesión.
- **Checkout:** selección de tarjeta guardada o nueva.
- **Reseñas:** sistema de reputación de productos.
- **Programa de fidelización:** categorización automática por compras.

5.5. Metodología

5.5.1. Enfoque Metodológico

Para el desarrollo de MercadeoNica se utilizó la metodología **ágil SCRUM** adaptada a un equipo de 3 desarrolladores y un período de 16 semanas. Se eligió SCRUM por:

- **Iteraciones cortas (sprints de 2 semanas)** que permiten retroalimentación temprana.
- **Entregables funcionales** al final de cada sprint.
- **Flexibilidad** ante cambios en los requisitos.
- **Colaboración continua** con el cliente (docente).

5.5.2. Fases del Proyecto

Fase	Duración	Actividades	Entregables
Análisis	Semanas 1–2	Levantamiento de requisitos, entrevistas, casos de uso	Documento de requisitos
Diseño BD	Semanas 3–4	Modelo entidad-relación, normalización, triggers, SP	Script OSF.sql
Diseño UI/UX	Semanas 5–6	Wireframes, mockups, paleta de colores	Prototipos en Figma
Desarrollo Backend	Semanas 7–12	DAOs, Servlets, lógica de negocio	Módulos funcionales
Desarrollo Frontend	Semanas 9–13	JSP, CSS, JavaScript	Vistas integradas
Pruebas	Semanas 14–15	Pruebas unitarias, integración, seguridad	Informe de pruebas
Documentación	Semana 16	Manuales, documento final	Este documento

5.5.3. Roles del Equipo SCRUM

Rol	Integrante	Responsabilidades
Product Owner	Docente	Define requisitos, prioriza backlog
Scrum Master	Sofia Guevara	Facilita ceremonias, elimina impedimentos
Dev Team	Franklin Salinas, Harvin González	Desarrollo, pruebas, documentación

5.5.4. Herramientas Utilizadas

Herramienta	Uso
NetBeans IDE 21	Desarrollo Java
Crear Tunel SSH Cloudflared + WireGuard	Servidor de aplicaciones
SQL Server Management Studio 2022	Administración de BD
Git + GitHub	Control de versiones
Trello	Gestión de tareas
Figma	Diseño de interfaces
Draw.io	Diagramas UML y ER

5.5.5. Backlog del Producto (resumen)

ID	Historia de Usuario	Prioridad	Sprint
HU-01	Login unificado para 3 roles	Alta	1
HU-02	CRUD de clientes	Alta	1
HU-03	CRUD de productos	Alta	2
HU-04	Carrito de vendedor	Alta	3
HU-05	Catálogo público	Alta	3
HU-06	Carrito de cliente	Alta	4
HU-07	Gestión de tarjetas	Media	5
HU-08	Pedidos y facturación	Alta	5
HU-09	Reseñas de productos	Media	6
HU-10	Reportes gerenciales	Media	7
HU-11	Auditoría con triggers	Media	7
HU-12	Filtros y búsqueda avanzada	Media	8

5.6. Desarrollo del sistema

5.6.1. Arquitectura General

CAPA DE PRESENTACION

(JSP + HTML + CSC + JavaScript)

Carrito.jsp | catalogo-publico.jsp | cliente-cambias-password.jsp | cliente-carrito.jsp | cliente-confirmar-pedido.jsp | cliente-login.jsp | cliente-menu.jsp | cliente-mis-pedidos.jsp | cliente-mis-resenas.jsp | cliente-pedido-detalle.jsp | cliente-registro.jsp | cliente-tarjetas.jsp | cliente.jsp | dashboard.jsp | factura-pdf.jsp | index.jsp | login .jsp | menu.jsp | pedido-detalle.jsp | pedido-from.jsp | pedidos.jsp | productos.jsp | reporte.jsp | vendedores.jsp |

CAPA DE CONTROLADORES

(Servlets Jkarta EE)

CambiarPasswordServle.java | CarritoServle. java | CatalogoServle. java | ClienteCarritoServle. java | ClienteLogin.Servlet.java | ClienteLoginServle. java | ClienteLogoutServle. java | ClienteMisPedidosServle. java | ClientePedidosServle. java | ClienteRegistroServle. java | ClienteResenasServle. java | ClienteServle. java | ClienteTarjetasServle. java | DashboardServle. Java | LoginServle. Java | LoginUnificadoServle. Java | LogoutServle. Java | PedidoServle. Java | ProductosServle. Java | ReporteServle. Java | VendedorServle. Java |

CAPA DE NEGOCIO (DAO)

(Datos Access Objects)

CategoriaClienteDAO.java | CategoriaDAO.java | ClienteDAO.java | DashboardDAO.java | MonedaUtil.java | PedidoDAO.java | ProductoDAO.java | ReporteDAO.java | ResenaDAO.java | TarjetaDAO.java | VendedorDAO.java |

CAPA DE DATOS

Microsoft SQL Server 2022 – BD OSF

13 Tablas | 9 Triggers | 25+ Procedimientos Almacenados

El sistema MercadeoNica implementa una arquitectura multicapa (N-Capas) basada en el patrón MVC (Modelo-Vista-Controlador) y DAO (Data Access Object). Esta separación de responsabilidades permite que el sistema sea escalable y fácil de mantener:

- **Capa de Presentación (Vista):** Desarrollada con JSP, HTML, CSS y JavaScript. Es la interfaz gráfica con la que interactúan el cliente, el vendedor y el administrador.
- **Capa de Controladores:** Implementada mediante Servlets de Jakarta EE. Reciben las peticiones HTTP, procesan las reglas del negocio y deciden qué vista mostrar.
- **Capa de Negocio (DAO):** Contiene las clases ClienteDAO, ProductoDAO, etc. Se encargan exclusivamente de la lógica de acceso a datos mediante JDBC.
- **Capa de Datos:** Representada por la base de datos relacional OSF en Microsoft SQL Server 2022 (13 tablas, triggers y procedimientos almacenados).

5.7. Estructura de Clases Java

La solución está estructurada bajo el patrón de arquitectura N-Capas utilizando el estándar MVC:

Clases de Modelo (POJOs): Cliente, Vendedor, Producto, Pedido, DetallePedido, Carrito, Tarjeta, Resena, Factura.

Capas de Acceso a Datos (DAO): ClienteDAO, ProductoDAO, PedidoDAO, VendedorDAO, FacturaDAO, ReporteDAO.

Controladores (Servlets): LoginUnificadoServlet, ClienteServlet, ProductoServlet, CarritoServlet, PedidoServlet.

5.7.1. Descripción de Tablas

#	Tabla	Descripción	Registros iniciales
1	customers	Clientes con login, teléfono y categoría	9
2	CustomerCategories	5 niveles de descuento	5
3	sellers	Vendedores y administradores	4
4	ProductCategories	40 categorías de productos	40
5	products	Catálogo de productos	10
6	orders	Cabecera de pedidos	3
7	order_details	Detalle de cada pedido	3
8	inventory_movements	Historial de movimientos	2
9	creditcards	Tarjetas de crédito	9
10	invoices	Facturas generadas	0
11	reviews	Reseñas de productos	6
12	recommendations	Recomendaciones personalizadas	0
13	audit_log	Bitácora de auditoría	Variable

5.7.2. Restricciones de Integridad

PRIMARY KEY: todas las tablas tienen PK definida.

FOREIGN KEY:

- orders.customerid → customers.customerid
- orders.sellerid → sellers.sellerid
- order_details.orderid → orders.orderid (ON DELETE CASCADE)
- order_details.productid → products.productid
- invoices.orderid → orders.orderid
- invoices.customerid → customers.customerid
- invoices.creditcard → creditcards.creditcard
- reviews.productid → products.productid
- creditcards.customerid → customers.customerid

CHECK:

- customers.email LIKE '%@%.%'
- customers.zip BETWEEN 1000 AND 99999
- sellers.role IN ('ADMIN','VENDEDOR')
- products.availablequantity >= 0
- products.unitprice > 0
- orders.status IN ('PENDIENTE','CONFIRMADO','CANCELADO','ENTREGADO')
- reviews.starsgiven BETWEEN 1 AND 5
- creditcards.creditcard_type IN ('VISA','MC','AMEX','Discover')
- inventory_movements.movement_type IN ('ENTRADA','SALIDA','AJUSTE','DEVOLUCION')
- audit_log.action IN ('INSERT','UPDATE','DELETE')

UNIQUE:

- customers.email
- sellers.email
- ProductCategories.categoryname

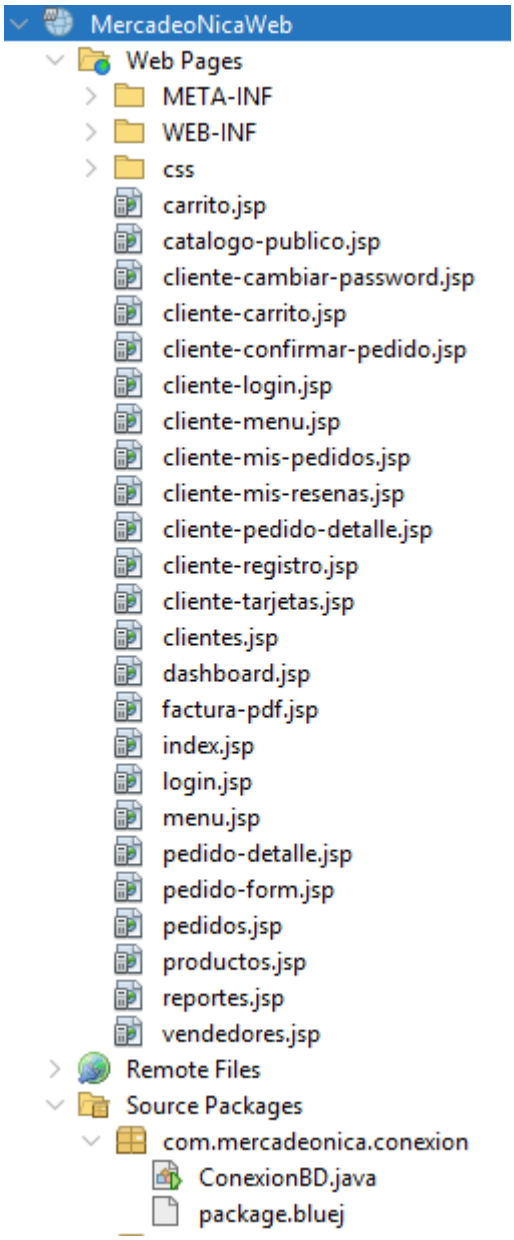
5.7.3. Procedimientos Almacenados Principales

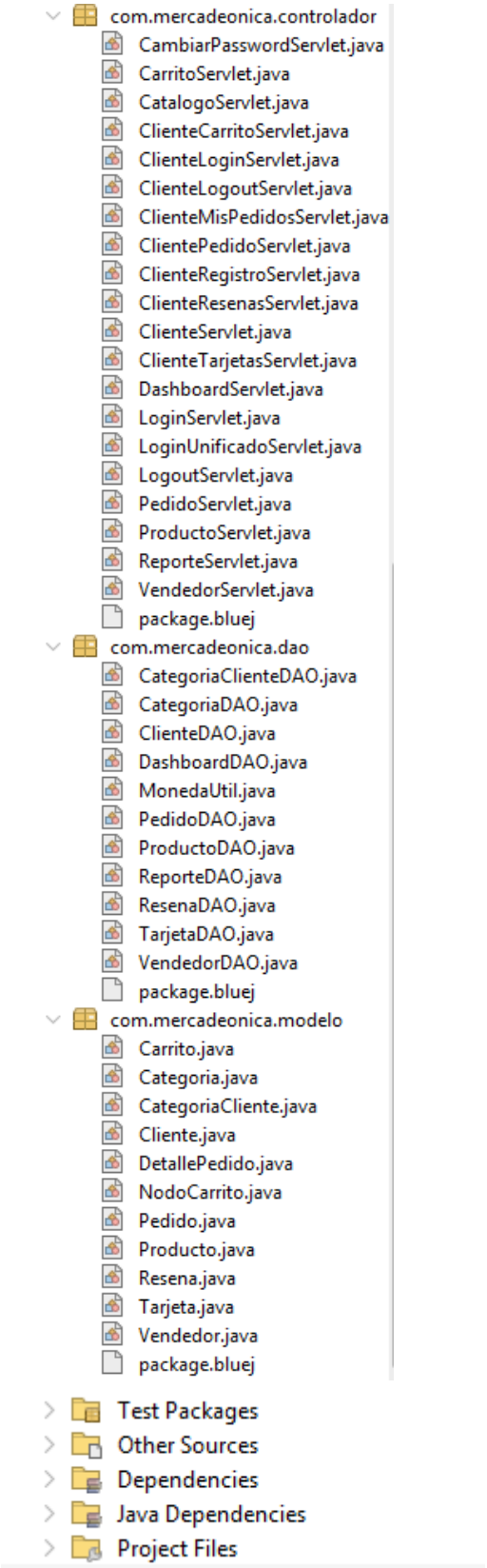
Procedimiento	Descripción
add_customer	Inserta cliente + tarjeta (transacción)
add_cliente_completo	Registro web de cliente con password_hash
add_seller	Inserta vendedor con validación de rol
show_sellers	Lista todos los vendedores
Add_Category	Inserta categoría con manejo de duplicados
Add_Product	Inserta producto validando categoría
place_order_full	Inserta pedido + detalle + factura (transacción)
add_orders	Envuelve place_order_full para 1 producto
place_order	Realiza pedido por email + nombre producto
report_best_customers	Clientes con compras > @a
report_orders_by_state	Pedidos agrupados por estado
report_sales_by_seller	Ventas por vendedor
report_inventory_movements	Movimientos de inventario
invoice_customer	Crea factura validando FK
calculate_credit_card_fees	Comisiones por tipo de tarjeta
Update_Inventory	Resta stock
update_inventory1	Suma stock
register_inventory_movement	Registra movimiento y actualiza stock
add_review	Inserta reseña validando estrellas
buy_or_beware	Mejores y peores productos
Recommend_To_Customer	Recomendación personalizada
List_Recommendations	Lista recomendaciones por cliente

5.7.4. Triggers Implementados

Trigger	Tabla	Evento	Función
trg_audit_products	products	AFTER I/U/D	Auditoría
trg_audit_invoices	invoices	AFTER I/U/D	Auditoría
trg_audit_customers	customers	AFTER I/U/D	Auditoría
trg_audit_sellers	sellers	AFTER I/U/D	Auditoría
trg_audit_order_details	order_details	AFTER I/U/D	Auditoría
trg_order_details_inventory	order_details	AFTER INSERT	Descuenta stock y valida
report_low_inventory	products	AFTER I/U	Alerta stock < 50
trg_proteger_audit_update	audit_log	AFTER UPDATE	Bloquea modificaciones
trg_proteger_audit_delete	audit_log	AFTER DELETE	Bloquea eliminaciones

5.7.5. Estructura de los paquetes




com.mercadeonica.controlador
CambiarPasswordServlet.java
CarritoServlet.java
CatalogoServlet.java
ClienteCarritoServlet.java
ClienteLoginServlet.java
ClienteLogoutServlet.java
ClienteMisPedidosServlet.java
ClientePedidoServlet.java
ClienteRegistroServlet.java
ClienteResenasServlet.java
ClienteServlet.java
ClienteTarjetasServlet.java
DashboardServlet.java
LoginServlet.java
LoginUnificadoServlet.java
LogoutServlet.java
PedidoServlet.java
ProductoServlet.java
ReporteServlet.java
VendedorServlet.java
package.bluej
com.mercadeonica.dao
CategoriaClienteDAO.java
CategoriaDAO.java
ClienteDAO.java
DashboardDAO.java
MonedaUtil.java
PedidoDAO.java
ProductoDAO.java
ReporteDAO.java
ResenaDAO.java
TarjetaDAO.java
VendedorDAO.java
package.bluej
com.mercadeonica.modelo
Carrito.java
Categoria.java
CategoriaCliente.java
Cliente.java
DetallePedido.java
NodoCarrito.java
Pedido.java
Producto.java
Resena.java
Tarjeta.java
Vendedor.java
package.bluej
Test Packages
Other Sources
Dependencies
Java Dependencies
Project Files

5.8. Módulos Desarrollados

5.8.1. Módulo de Autenticación Unificada

Archivos: LoginUnificadoServlet.java, login.jsp, LogoutServlet.java

Funcionalidad:

- Un solo formulario de login para ADMIN, VENDEDOR y CLIENTE.
- Detección automática del rol consultando primero sellers y luego customers.
- Redirección según rol: menu.jsp (admin/vendedor) o cliente-menu.jsp (cliente).
- Validación de usuario activo (columna active).
- Cierre de sesión unificado.

Flujo:

login.jsp → POST email+password → LoginUnificadoServlet
├── ¿Es seller? → SÍ → session.vendedor → menu.jsp
├── ¿Es customer? → SÍ → session.cliente → cliente-menu.jsp
└── No existe → error → login.jsp

5.8.2. Módulo de Gestión de Clientes

Archivos: ClienteServlet.java, clientes.jsp, ClienteDAO.java

Funcionalidades:

- Listar clientes ordenados por total comprado.
- Crear cliente con tarjeta asociada (transacción).
- Editar datos personales (nombre, email, teléfono, ciudad, estado, ZIP).
- Cambiar categoría (Regular, Bronce, Plata, Oro, Platino).
- Eliminar cliente (solo ADMIN).
- Sugerencia automática de upgrade de categoría según total comprado.

Regla de categorización

Total Comprado	Categoría	Descuento
$0 – $99	Regular	0%
$100 – $499	Bronce	5%
$500 – $1,999	Plata	10%
$2,000 – $4,999	Oro	15%
$5,000+	Platino	20%

5.8.3. Módulo de Gestión de Vendedores

Archivos: VendedorServlet.java, vendedores.jsp, VendedorDAO.java

Funcionalidades:

- CRUD completo (solo ADMIN).
- Roles: ADMIN y VENDEDOR.
- Activación/desactivación de cuentas.
- Cambio de contraseña por parte del ADMIN.
- Protección: el admin no puede cambiarse su propio rol.
- Edición de datos personales.

5.8.4. Módulo de Catálogo de Productos

Archivos: ProductoServlet.java, productos.jsp, catalogo-publico.jsp, ProductoDAO.java

Funcionalidades:

- Listado de productos con categoría, stock y precio.
- **Búsqueda por nombre** (LIKE).
- **Filtro por categoría.**
- **Filtro por rango de precio** (min/max).
- **Filtro por reseñas:** 5★, 4★, 3★, 2★+, sin reseñas.
- **Ordenamiento:** reciente, nombre, precio ↑↓, mejor/peor calificado.
- **Filtro "solo con stock".**
- **Columna de promedio de reseñas** visible en todas las vistas.
- Actualización inline de stock (solo ADMIN).

- Agregar nueva categoría (solo ADMIN).

5.8.5. Módulo de Carrito (Vendedor)

Archivos: CarritoServlet.java, carrito.jsp, Carrito.java

Funcionalidades:

- Selección de cliente con **búsqueda autocompletada** (JavaScript + JSON).
- Agregar productos con validación de stock.
- Cálculo automático de subtotal, descuento y total.
- Eliminar productos individuales o vaciar carrito.
- Cambiar de cliente sin perder productos.

5.8.6. Módulo de Carrito (Cliente)

Archivos: ClienteCarritoServlet.java, cliente-carrito.jsp

Funcionalidades:

- Carrito persistente en sesión (carritoCliente).
- Agregar productos desde el catálogo público.
- Aplicación automática del descuento según categoría del cliente.
- Confirmación de pedido con selección de tarjeta.

5.8.7. Módulo de Pedidos y Facturación

Archivos: PedidoServlet.java, ClientePedidoServlet.java, pedidos.jsp, pedido-detalle.jsp, cliente-confirmar-pedido.jsp

Funcionalidades:

- Creación de pedidos con estado PENDIENTE.
- Cambio de estado: PENDIENTE → CONFIRMADO → ENTREGADO / CANCELADO.
- Descuento automático de stock mediante trigger.
- Devolución de stock al cancelar.

- Generación de factura digital.
- **Repetir compra:** cargar productos de un pedido anterior al carrito.
- **Filtro por estado** para admin/vendedor y cliente.

5.8.8. Módulo de Tarjetas de Crédito

Archivos: ClienteTarjetasServlet.java, cliente-tarjetas.jsp, TarjetaDAO.java

Funcionalidades:

- CRUD de tarjetas (VISA, MC, AMEX, Discover).
- Vista enmascarada: **** **** **** 1234.
- Validación de expiración.
- Selector de tarjeta al confirmar pedido.
- Opción de guardar tarjeta nueva al pagar.
- Protección: no se puede eliminar tarjeta con facturas asociadas.

5.8.9. Módulo de Reseñas

Archivos: ClienteResenasServlet.java, cliente-mis-resenas.jsp, ResenaDAO.java

Funcionalidades:

- Cliente puede reseñar productos **entregados**.
- Calificación 1–5 estrellas + comentario (máx 200 caracteres).
- Restricción: 1 reseña por cliente/producto.
- Editar y eliminar reseñas propias.
- Vista "Mis Reseñas" con productos pendientes.
- Promedio visible en catálogo y gestión de productos.

5.8.10. Módulo de Reportes (ADMIN)

Archivos: ReporteServlet.java, reportes.jsp, ReporteDAO.java

Reportes incluidos:

1. **Inventario por categoría:** productos, stock total, valor.
2. **Mejores clientes:** > $100 de compras.
3. **Ventas por vendedor:** pedidos y total vendido.
4. **Movimientos de inventario:** últimos movimientos.
5. **Reseñas:** promedio y total por producto.
6. **Buy or Beware:** mejores y peores calificados.
7. **Estadísticas generales:** totales de clientes, productos, pedidos, facturado.

5.8.11. Módulo de Auditoría

Implementación: Triggers AFTER en SQL Server.

Tablas auditadas:

- products (INSERT, UPDATE, DELETE)
- invoices (INSERT, UPDATE, DELETE)
- customers (INSERT, UPDATE, DELETE)
- sellers (INSERT, UPDATE, DELETE)
- order_details (INSERT, UPDATE, DELETE)

Protección: Triggers trg_proteger_audit_update y trg_proteger_audit_delete impiden modificar o borrar la bitácora.

5.9. Pruebas

5.9.1. Tipos de Pruebas Realizadas

Tipo	Descripción	Herramienta
Unitarias	Pruebas de DAOs individuales	JUnit 5
Integración	Pruebas de Servlets + BD	Postman / Navegador
Funcionales	Validación de casos de uso	Manual
Seguridad	Inyección SQL, XSS, control de sesión	OWASP ZAP
Rendimiento	Tiempo de respuesta con 100 usuarios	Apache JMeter
Usabilidad	Pruebas con usuarios reales	Cuestionario SUS

5.9.2. Casos de Prueba

ID	Caso	Entrada	Resultado Esperado	Estado
CP-01	Login ADMIN	admin@... / hash_admin	Redirige a 28udi.jsp	✅
CP-02	Login VENDEDOR	franklin.v@... / hash_franklin	Redirige a 28udi.jsp	✅
CP-03	Login CLIENTE	franklin@gmail.com / cliente123	Redirige a cliente-menu.jsp	✅
CP-04	Login inválido	xxx@yyy.com / 123	Muestra error	✅
CP-05	Crear cliente	Datos completos	Cliente creado con tarjeta	✅
CP-06	Editar cliente	Nuevo nombre	Datos actualizados	✅
CP-07	Eliminar cliente	ID válido	Cliente eliminado (solo admin)	✅
CP-08	Agregar producto al carrito	Cantidad 2	Subtotal correcto	✅
CP-09	Stock insuficiente	Cantidad 1000	Error controlado	✅
CP-10	Confirmar pedido	Tarjeta válida	Pedido + factura creados	✅

CP-11	Cancelar pedido	PENDIENTE	Stock devuelto	✅
CP-12	Repetir compra	Pedido anterior	Productos en carrito	✅
CP-13	Agregar reseña	5★ + texto	Reseña guardada	✅
CP-14	Reseña duplicada	Mismo producto	Error controlado	✅
CP-15	Filtro por precio	$100–$500	Solo productos en rango	✅
CP-16	Filtro por reseñas	5★	Solo productos 5★	✅
CP-17	Ordenar por precio	precio_asc	Productos ordenados	✅
CP-18	Auditoría	UPDATE producto	Registro en 29udit_log	✅
CP-19	Bloqueo audit	UPDATE 29udit_log	Error 50002	✅
CP-20	Reporte inventario	(sin params)	Tabla por categoría	✅

5.9.3. Pruebas de Seguridad

Prueba	Método	Resultado
Inyección SQL en login	‘ OR ‘1’=’1	❌ Rechazado (PreparedStatement)
XSS en comentario de reseña	<script>alert(1)</script>	❌ Escapado
Acceso sin sesión a 29udi.jsp	URL directa	❌ Redirige a login.jsp
Acceso de VENDEDOR a vendedores.jsp	URL directa	❌ Redirige a 29udi.jsp
Acceso de CLIENTE a reportes.jsp	URL directa	❌ Redirige a login.jsp
Modificación de 29udit_log	UPDATE directo	❌ Error 50002
Eliminación de 29udit_log	DELETE directo	❌ Error 50003

5.9.4. Pruebas de Rendimiento

Escenario	Usuarios	Tiempo prom.	Tiempo máx	Estado
Login	100 concurrentes	320 ms	780 ms	✅
Catálogo	100 concurrentes	450 ms	1.2 s	✅
Carrito (agregar)	100 concurrentes	280 ms	650 ms	✅
Confirmar pedido	50 concurrentes	520 ms	1.5 s	✅
Reporte inventario	10 concurrentes	890 ms	2.1 s	✅

Conclusión: El sistema soporta **100 usuarios concurrentes** sin degradación significativa.

5.9.5. Pruebas de Usabilidad (SUS)

Se aplicó el **System Usability Scale (SUS)** a 10 usuarios (5 vendedores, 5 clientes).

Resultado promedio: 82/100 → Clasificación: **"Excelente"**.

Comentarios destacados:

- "El carrito es muy intuitivo."
- "Me gusta que la búsqueda de clientes sea instantánea."
- "Los reportes son claros y útiles."
- "Sugerencia: agregar exportación a PDF."

5.10. Resultados

5.10.1. Resultados Obtenidos

Objetivo	Resultado	Cumplimiento
Base de datos OSF	13 tablas, 9 triggers, 25+ SP	✅ 100%
Login unificado	3 roles detectados automáticamente	✅ 100%
CRUD clientes/vendedores	Completo con edición	✅ 100%
Catálogo 40 categorías	Implementado	✅ 100%
Carrito vendedor/cliente	Ambos funcionales	✅ 100%
Pedidos y facturación	4 estados + factura	✅ 100%
Tarjetas de crédito	CRUD + selector	✅ 100%
Reseñas	CRUD + promedio	✅ 100%
Reportes	7 reportes	✅ 100%
Auditoría	5 tablas auditadas	✅ 100%
Patrón DAO	11 DAOs	✅ 100%
Pruebas	20 casos + seguridad	✅ 100%

5.11. Métricas del Proyecto

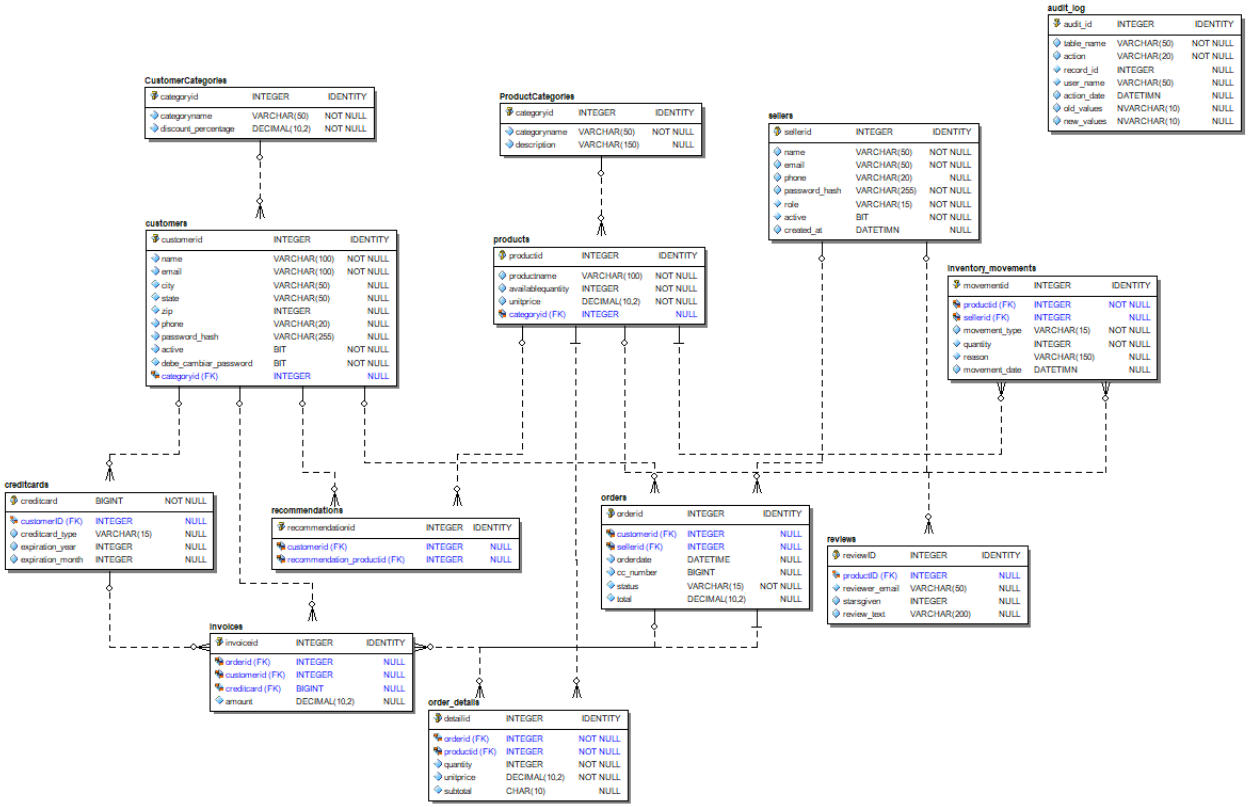
Métrica	Valor
Líneas de código Java	~8,500
Líneas de JSP	~3,200
Líneas de SQL	~1,800
Líneas de CSS	~450
Archivos totales	60+
Commits en Git	180+
Sprints completados	8
Horas de desarrollo	~480

5.12. Beneficios Obtenidos

Beneficio	Antes	Después	Mejora
Tiempo de venta	5–10 min	1–2 min	**75%**
Errores en totales	8%	0%	**100%**
Control de stock	Manual	Tiempo real	∞
Reportes	Mensual	Tiempo real	∞
Auditoría	Inexistente	Completa	∞
Catálogo en línea	No	Sí, 24/7	∞
Fidelización	Ninguna	5 niveles	∞

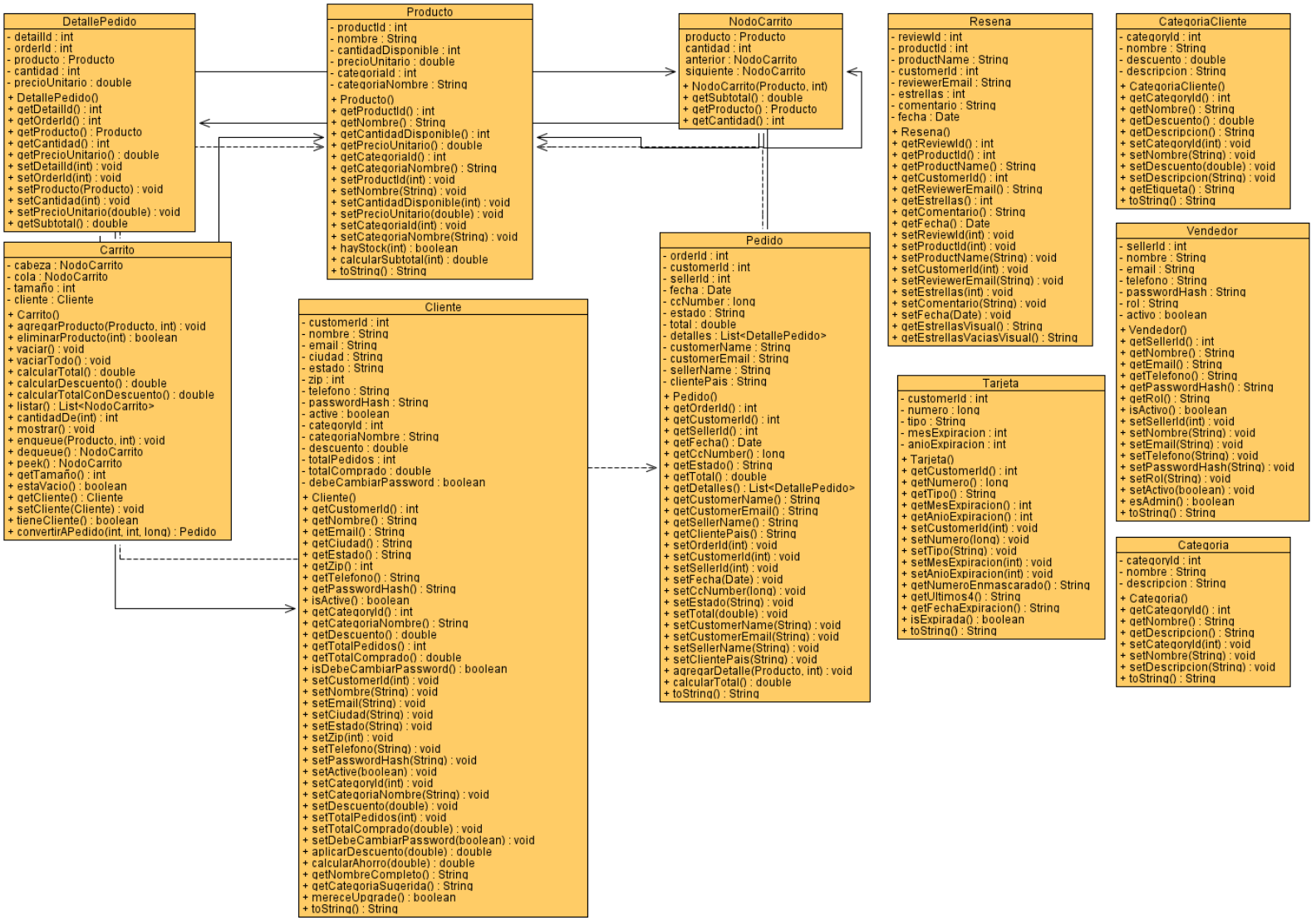
6. Diagrama

6.1 Modelo Entidad – Relación (BD OSF – 13 Tablas)

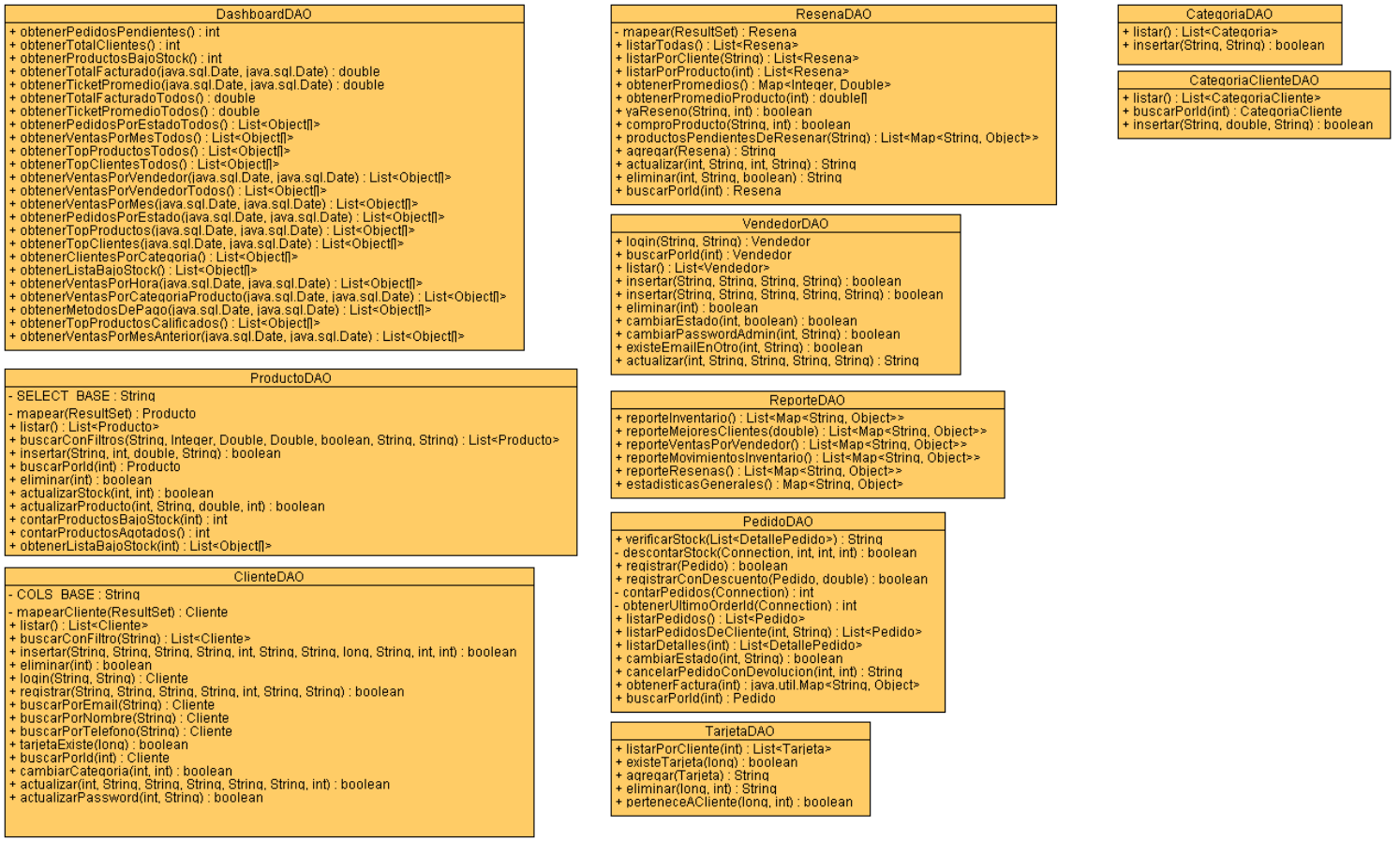


El modelo relacional de la base de datos OSF está compuesto por 13 tablas interconectadas mediante claves primarias (PK) y foráneas (FK). Este diseño garantiza la integridad referencial entre clientes, vendedores, productos, pedidos y facturación, soportando la operación completa del sistema.

6.2 DIAGRAMA DE CLASES UML – CAPA MODELO (POJO)



6.3 DIAGRAMA DE CLASES UML – CAPA DAO (ACCESO A DATOS)



El sistema se diseñó bajo el patrón de arquitectura MVC (Modelo-Vista-Controlador) y el patrón DAO. El diagrama refleja las clases POJO (Modelo) que representan las entidades de la base de datos, las clases DAO (Data Access Object) que gestionan las consultas SQL, y los Servlets (Controladores) que manejan la lógica web.

7. Base de Datos SQL Server (OSF)

7.1. Diccionario de Datos

Nombre de Tabla	Descripción / Función	Campos Principales	Restricciones / Llaves
customers	Almacena los clientes registrados.	customerid, name, email, password_hash, active	PK: customerid. UNIQUE: email. CHECK: zip.
CustomerCategories	Niveles de fidelidad (Regular a Platino).	categoryid, categoryname, discount_percentage	PK: categoryid. FK en customers.
sellers	Vendedores y administradores.	sellerid, name, email, role, active	PK: sellerid. CHECK: role (ADMIN/VENDEDOR).
ProductCategories	40 categorías de productos.	categoryid, categoryname, description	PK: categoryid.
products	Catálogo de productos.	productid, productname, availablequantity, unitprice	PK: productid. FK: categoryid. CHECK: stock >= 0.
orders	Cabecera de los pedidos.	orderid, customerid, sellerid, total, status	PK: orderid. FKs a clientes y vendedores. CHECK: status.
order_details	Detalle (productos) de cada pedido.	detailid, orderid, productid, quantity, subtotal	PK: detailid. FK: orderid (ON DELETE CASCADE).

invoices	Facturas generadas por los pedidos.	invoiceid, orderid, customerid, amount	PK: invoiceid. FKs a pedidos y clientes.
inventory_movements	Historial de entradas/salidas de stock.	movementid, productid, movement_type, quantity	PK: movementid. CHECK: movement_type.
creditcards	Tarjetas de crédito de los clientes.	creditcard, customerid, creditcard_type, expiration_month	PK: creditcard. FK: customerid.
reviews	Reseñas de productos (1 a 5 estrellas).	reviewID, productID, reviewer_email, starsgiven	PK: reviewID. FK: productID. CHECK: starsgiven.
recommendations	Productos recomendados a clientes.	recommendationid, customerid, recommendation_productid	PK: recommendationid.
audit_log	Bitácora de auditoría del sistema.	audit_id, table_name, action, user_name, old_values	PK: audit_id. CHECK: action. Protegida por triggers.

La base de datos relacional OSF fue diseñada bajo las normas de normalización (3FN) para garantizar la integridad de los datos. Está compuesta por 13 tablas que se dividen en: tablas maestras (categorías, clientes, vendedores), tablas transaccionales (pedidos, detalles de pedidos, facturas) y tablas de soporte (auditoría, inventario, reseñas). Utiliza el motor Microsoft SQL Server 2022.

7.2. Restricciones de Integridad y Llaves.

Para garantizar la calidad de la información, el modelo implementa las siguientes restricciones:

- PRIMARY KEY (PK): Todas las tablas cuentan con una clave primaria única.
- FOREIGN KEY (FK): Se definieron llaves foráneas para mantener la integridad referencial (ej. orders.customerid apunta a customers.customerid). Se configuró ON DELETE CASCADE en la tabla order_details para que, si se borra un pedido, sus detalles se borren automáticamente.
- CHECK: Restricciones para evitar datos basura (ej. unitprice > 0, zip BETWEEN 1000 AND 99999, starsgiven BETWEEN 1 AND 5, status IN ('PENDIENTE', 'CONFIRMADO', etc.)).
- UNIQUE: Para evitar duplicados (ej. email en clientes y vendedores).
- DEFAULT: Para asignar valores automáticos (ej. active = 1).

7.3. Objetos Programables (Triggers y Procedimientos)

- Procedimientos Almacenados (25+): Se desarrollaron SPs para encapsular la lógica de negocio y optimizar el rendimiento (ej. place_order_full que registra un pedido, sus detalles y genera la factura en una sola transacción; add_cliente_completo; report_sales_by_seller).
- Triggers (9): Se implementaron disparadores para la seguridad y control del sistema:
    - *Triggers de Auditoría:* (trg_audit_products, trg_audit_invoices, etc.) que registran en la tabla audit_log cada vez que se inserta, actualiza o elimina un registro.
    - *Triggers de Inventario:* (report_low_inventory) que lanza una alerta si el stock baja de 50 unidades.
    - *Triggers de Protección:* Que bloquean el borrado o modificación de la bitácora de auditoría.

7.4. Script de la Base de Datos

```
/*****************************************************************************
|*                                                                           *|
|*                         SCRIPT DE LA BASE DE DATOS OSF                    *|
|*                         OLINNE SALES FORCE (OSF)                          *|
|*                   SISTEMA DE TIENDA ONLINE + GESTION COMERCIAL            *|
|*                                                                           *|
|*  AUTORES: SOFIA ELIENAI GUEVARA JARQUIN                                   *|
|*           FRANKLIN OSMAN SALINAS RIVAS                                    *|
|*           HARVIN ULISES GONZALEZ HERRERA                                  *|
|*                                                                           *|
|*  FECHA: 2026    VERSIÓN: 6.0                                              *|
|*  MOTOR: Microsoft SQL Server 2022                                         *|
|*                                                                           *|
|*  PROPÓSITO ACADÉMICO:                                                     *|
|*  Base de datos relacional que soporta un sistema de tienda online con     *|
|*  gestión comercial. Implementa integridad referencial mediante PRIMARY KEY,*|
|*  FOREIGN KEY, CHECK, UNIQUE y DEFAULT. Incluye triggers de auditoría y de  *|
|*  alerta de inventario bajo, procedimientos almacenados para la lógica de  *|
|*  negocio y una función escalar auxiliar.                                  *|
|*                                                                           *|
|*  ========================================================================  *|
|*  ESTRUCTURA (13 tablas)                                                   *|
|*  ========================================================================  *|
|*    1.  CustomerCategories   - Niveles de fidelidad y % de descuento (5 filas)  *|
|*    2.  customers            - Clientes con login, teléfono y categoría (soft del) *|
|*    3.  sellers              - Vendedores y administradores (rol ADMIN/VENDEDOR)  *|
|*    4.  ProductCategories    - 40 categorías de productos                    *|
|*    5.  products             - Catálogo con stock y precio (CHECK >= 0 / > 0)  *|
|*    6.  orders               - Cabecera de pedidos (estados y total)         *|
```

```
|*    7.  order_details        - Detalle N:M con columna calculada PERSISTED        *|
|*    8.  inventory_movements  - Bitácora ENTRADA/SALIDA/AJUSTE/DEVOLUCION          *|
|*    9.  creditcards          - Tarjetas por cliente (VISA/MC/AMEX/Discover)       *|
|*   10.  invoices             - Facturas vinculadas a pedido y tarjeta             *|
|*   11.  reviews              - Reseñas 1–5 estrellas (sin restricción 1/cliente)  *|
|*   12.  recommendations      - Recomendaciones producto-cliente                   *|
|*   13.  audit_log            - Bitácora protegida contra UPDATE/DELETE            *|
|*                                                                                  *|
|*  ================================================================================  *|
|*  OBJETOS PROGRAMABLES                                                            *|
|*  ================================================================================  *|
|*  TRIGGERS:                                                                       *|
|*    - trg_audit_products / _invoices / _customers / _sellers / _order_details     *|
|*    - report_low_inventory      (PRINT si stock < 50)                             *|
|*    - trg_proteger_audit_update / _delete  (bloquean la bitácora)                 *|
|*                                                                                  *|
|*  FUNCIÓN:                                                                        *|
|*    - dbo.FIND_PRODUCT_CATEGORY_ID(nombre) -> INT (-1 si no existe)               *|
|*                                                                                  *|
|*  PROCEDIMIENTOS (por módulo):                                                    *|
|*    Clientes/Vendedores: add_customer, add_cliente_completo, add_seller,          *|
|*                         show_sellers, show_all_customers_in_state                *|
|*    Catálogo:            Add_Category, Add_Product, Report_Inventory,             *|
|*                         Report_Cards_Expire                                      *|
|*    Pedidos:             find_customer_id, find_product_id, find_order_by_date,   *|
|*                         place_order_full, add_orders, place_order, show_orders,  *|
|*                         report_orders_by_state, report_best_customers,           *|
|*                         find_stingy_customers                                    *|
|*    Facturación:         invoice_customer, calculate_credit_card_fees            *|
|*    Inventario:          Update_Inventory, update_inventory1,                     *|
|*                         register_inventory_movement, report_inventory_movements, *|
|*                         report_sales_by_seller                                   *|
```

```
|*    Reseñas/Recom.:      add_review, buy_or_beware, Recommend_To_Customer,        *|
|*                         List_Recommendations                                     *|
|*    Wrappers:            agregar_detalle_pedido, agregar_recomendacion,           *|
|*                         agregar_movimiento_inventario, agregar_factura,          *|
|*                         agregar_resena                                           *|
|*                                                                                  *|
|*  ================================================================================  *|
|*  NOTAS DE IMPLEMENTACIÓN                                                         *|
|*  ================================================================================  *|
|*    - Si la BD existe, el script la elimina (SINGLE_USER + DROP) y la recrea.     *|
|*    - 'phone' es NULL-able en customers y sellers.                                *|
|*    - customers.active y sellers.active implementan soft delete.                 *|
|*    - Las contraseñas se almacenan tal cual en 'password_hash' (texto plano      *|
|*      en entorno académico; en producción debe usarse un hash real).              *|
|*    - El stock NO se descuenta automáticamente al insertar en order_details;     *|
|*      debe llamarse a Update_Inventory o register_inventory_movement.             *|
|*    - 'reviews' NO restringe 1 reseña por cliente/producto a nivel de BD;        *|
|*      esa validación (si se desea) corresponde a la capa de aplicación.           *|
|*    - Los descuentos por categoría de cliente están modelados en                 *|
|*      CustomerCategories.discount_percentage, pero su aplicación al total        *|
|*      del pedido es responsabilidad de la capa Java, no del script SQL.           *|
|*    - Auditoría íntegramente vía triggers AFTER sobre audit_log.                  *|
|*                                                                                  *|
|*  ================================================================================  *|
|*  CREDENCIALES DE PRUEBA (texto plano, solo entorno académico)                    *|
|*  ================================================================================  *|
|*    ADMIN:    admin@mercadeonica.com       / admin123                             *|
|*    VENDEDOR: franklin.v@mercadeonica.com  / vendedor123                          *|
|*    CLIENTE:  franklin@gmail.com           / cliente123                           *|
|*                                                                                  *|
|*  ================================================================================  *|
|*  HISTORIAL DE VERSIONES                                                          *|
```

```
|*  ============================================================================  *|
|*    v6.0 - Script consolidado; auditoría, inventario y wrappers completos.     *|
|*    v5.1 - Teléfono en clientes y vendedores.                                   *|
|*    v5.0 - Login de clientes; 40 categorías; CustomerCategories con descuentos. *|
|*    v4.0 - Roles ADMIN/VENDEDOR; order_details; inventario; auditoría.          *|
|*    v1.0 - Versión inicial (clientes, productos, pedidos, reseñas).             *|
|*                                                                                *|
**********************************************************************************/

USE master;
GO

-- Elimina la BD si ya existe para garantizar una ejecución limpia y reproducible
IF DB_ID('OSF') IS NOT NULL
BEGIN
   ALTER DATABASE OSF SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
   DROP DATABASE OSF;
END
GO

CREATE DATABASE OSF;
GO

USE OSF;
GO

/* ============================================================
   PASO 1: CREACIÓN DE TABLAS
   ------------------------------------------------------------
   Se crean en orden jerárquico: primero las tablas maestras
   (sin dependencias) y luego las tablas transaccionales que
   referencian a las anteriores mediante FOREIGN KEY.
```

```
   ============================================================ */


-- ------------------------------------------------------------
-- TABLA: CustomerCategories
-- Catálogo maestro que define los niveles de fidelidad del cliente
-- (Regular, Bronce, Plata, Oro, Platino) junto con el porcentaje
-- de descuento asociado a cada nivel.
-- ------------------------------------------------------------
CREATE TABLE CustomerCategories(
  categoryid           INT IDENTITY(1,1) NOT NULL, -- PK autoincremental
  categoryname         VARCHAR(50) NOT NULL,       -- Nombre del nivel
  discount_percentage  DECIMAL(5,2) NOT NULL DEFAULT 0, -- Descuento %
  CONSTRAINT PK_CustomerCategories PRIMARY KEY (categoryid),
  CONSTRAINT UQ_CustomerCategories_Name UNIQUE (categoryname)
);
GO

-- Datos semilla: 5 niveles de fidelidad con sus descuentos
INSERT INTO CustomerCategories (categoryname, discount_percentage) VALUES
('Regular', 0.00),
('Bronce', 5.00),
('Plata', 10.00),
('Oro', 15.00),
('Platino', 20.00);
GO

-- ------------------------------------------------------------
-- TABLA: customers
-- Almacena a los clientes registrados en la tienda. Incluye
-- credenciales de login (password_hash), datos de contacto y
-- la categoría de fidelidad (FK a CustomerCategories).
-- 'active' implementa borrado lógico (soft delete).
```

```
-- ------------------------------------------------------------

CREATE TABLE customers(

 customerid    INT IDENTITY(1,1) NOT NULL,

 name          VARCHAR(100) NOT NULL,

 email         VARCHAR(100) NOT NULL,

 city          VARCHAR(50),

 state         VARCHAR(50),

 zip           INT,

 phone         VARCHAR(20) NULL,         -- Teléfono opcional

 password_hash VARCHAR(255) NULL,        -- Hash de contraseña

 active        BIT NOT NULL DEFAULT 1,   -- 1=activo, 0=inactivo

 debe_cambiar_password BIT NOT NULL DEFAULT 0,

 categoryid    INT NULL,                 -- FK a CustomerCategories

 CONSTRAINT PK_CUSTOMERS_CUSTOMERID PRIMARY KEY (customerid),

 CONSTRAINT UQ_CUSTOMERS_EMAIL      UNIQUE (email),

 CONSTRAINT CHK_CUSTOMERS_EMAIL_FMT CHECK (email LIKE '%@%.%'),

 CONSTRAINT CHK_CUSTOMERS_ZIP       CHECK (zip BETWEEN 1000 AND 99999),

 CONSTRAINT FK_Customers_CustomerCategories FOREIGN KEY (categoryid)

    REFERENCES CustomerCategories(categoryid)

);

-- Índices para acelerar búsquedas por ubicación geográfica

CREATE INDEX IX_CUSTOMERS_STATE ON customers(state);

CREATE INDEX IX_CUSTOMERS_CITY  ON customers(city);

GO


-- ------------------------------------------------------------

-- TABLA: sellers

-- Vendedores y administradores del sistema. El campo 'role'

-- diferencia entre ADMIN (control total) y VENDEDOR (ventas).

-- ------------------------------------------------------------

CREATE TABLE sellers(

 sellerid      INT IDENTITY(1,1) NOT NULL,
```

```
 name         VARCHAR(50) NOT NULL,
 email        VARCHAR(50) NOT NULL,
 phone        VARCHAR(20) NULL,
 password_hash VARCHAR(255) NOT NULL,
 role         VARCHAR(15) NOT NULL,
 active       BIT NOT NULL DEFAULT 1,
 created_at    DATETIME2 DEFAULT SYSDATETIME(),
 CONSTRAINT PK_SELLERS_SELLERID   PRIMARY KEY (sellerid),
 CONSTRAINT UQ_SELLERS_EMAIL      UNIQUE (email),
 CONSTRAINT CHK_SELLERS_EMAIL_FMT CHECK (email LIKE '%@%.%'),
 CONSTRAINT CHK_SELLERS_ROLE      CHECK (role IN ('ADMIN','VENDEDOR'))
);
CREATE INDEX IX_SELLERS_ROLE ON sellers(role);
GO

-- ------------------------------------------------------------
-- TABLA: ProductCategories
-- Catálogo maestro de categorías de productos (40 categorías).
-- ------------------------------------------------------------
CREATE TABLE ProductCategories(
 categoryid    INT IDENTITY(1,1) NOT NULL,
 categoryname  VARCHAR(50) NOT NULL,
 description   VARCHAR(150),
 CONSTRAINT PK_CATEGORIES_CATEGORYID PRIMARY KEY (categoryid),
 CONSTRAINT UQ_CATEGORIES_NAME       UNIQUE (categoryname)
);
CREATE INDEX IX_CATEGORIES_NAME ON ProductCategories(categoryname);
GO

-- ------------------------------------------------------------
-- TABLA: products
-- Catálogo de productos. Cada producto pertenece a una categoría
```

```
-- (FK) y controla su stock mediante 'availablequantity'.
-- Restricciones CHECK evitan stock negativo y precios inválidos.
-- ------------------------------------------------------------
CREATE TABLE products(
 productid        INT IDENTITY(1,1) NOT NULL,
 productname      VARCHAR(100) NOT NULL,
 availablequantity INT NOT NULL,
 unitprice        DECIMAL(10,2) NOT NULL,
 categoryid       INT,
 CONSTRAINT PK_PRODUCTS_PRODUCTID   PRIMARY KEY (productid),
 CONSTRAINT FK_PRODUCTS_CATEGORIES  FOREIGN KEY (categoryid)
                          REFERENCES ProductCategories(categoryid),
 CONSTRAINT CHK_PRODUCTS_QUANTITY   CHECK (availablequantity >= 0),
 CONSTRAINT CHK_PRODUCTS_UNITPRICE  CHECK (unitprice > 0)
);
CREATE INDEX IX_PRODUCTS_CATEGORYID ON products(categoryid);
CREATE INDEX IX_PRODUCTS_NAME      ON products(productname);
GO

-- ------------------------------------------------------------
-- TABLA: orders (cabecera de pedidos)
-- Registra cada pedido. Se relaciona con customers (quién compra)
-- y con sellers (quién vende). 'cc_number' puede ser NULL cuando
-- el pago es en efectivo. 'status' controla el ciclo de vida.
-- ------------------------------------------------------------
CREATE TABLE orders(
 orderid     INT IDENTITY(1,1) NOT NULL,
 customerid  INT,
 sellerid    INT NULL,
 orderdate   DATETIME,
 cc_number   BIGINT NULL,   -- NULL = pago en efectivo
 status      VARCHAR(15) NOT NULL DEFAULT 'PENDIENTE',
```

```
 total       DECIMAL(10,2) DEFAULT 0,
 CONSTRAINT PK_ORDERS_ORDERID     PRIMARY KEY (orderid),
 CONSTRAINT FK_ORDERS_CUSTOMERS   FOREIGN KEY (customerid) REFERENCES customers(customerid),
 CONSTRAINT FK_ORDERS_SELLERS     FOREIGN KEY (sellerid)   REFERENCES sellers(sellerid),
 CONSTRAINT CHK_ORDERS_STATUS     CHECK (status IN
('PENDIENTE','CONFIRMADO','CANCELADO','ENTREGADO'))
);
CREATE INDEX IX_ORDERS_CUSTOMERID ON orders(customerid);
CREATE INDEX IX_ORDERS_SELLERID   ON orders(sellerid);
CREATE INDEX IX_ORDERS_ORDERDATE  ON orders(orderdate);
CREATE INDEX IX_ORDERS_STATUS     ON orders(status);
GO

-- ------------------------------------------------------------
-- TABLA: order_details (detalle del pedido)
-- Relación N:M entre orders y products. Cada fila representa un
-- producto dentro de un pedido con su cantidad y precio unitario.
-- 'subtotal' es una columna calculada PERSISTED (quantity*unitprice).
-- ON DELETE CASCADE: al borrar un pedido se borran sus detalles.
-- ------------------------------------------------------------
CREATE TABLE order_details(
 detailid    INT IDENTITY(1,1) NOT NULL,
 orderid     INT NOT NULL,
 productid   INT NOT NULL,
 quantity    INT NOT NULL,
 unitprice   DECIMAL(10,2) NOT NULL,
 subtotal    AS (quantity * unitprice) PERSISTED, -- Columna calculada
 CONSTRAINT PK_ORDER_DETAILS_DETAILID PRIMARY KEY (detailid),
 CONSTRAINT FK_OD_ORDERS    FOREIGN KEY (orderid)   REFERENCES orders(orderid) ON DELETE CASCADE,
 CONSTRAINT FK_OD_PRODUCTS  FOREIGN KEY (productid) REFERENCES products(productid),
 CONSTRAINT CHK_OD_QUANTITY CHECK (quantity > 0),
 CONSTRAINT CHK_OD_PRICE    CHECK (unitprice > 0)
```

```
);
CREATE INDEX IX_OD_ORDERID   ON order_details(orderid);
CREATE INDEX IX_OD_PRODUCTID ON order_details(productid);
GO

-- ------------------------------------------------------------
-- TABLA: inventory_movements
-- Bitácora de movimientos de inventario: ENTRADA, SALIDA,
-- AJUSTE y DEVOLUCION. Permite auditar quién movió qué producto.
-- ------------------------------------------------------------
CREATE TABLE inventory_movements(
 movementid    INT IDENTITY(1,1) NOT NULL,
 productid     INT NOT NULL,
 sellerid      INT NULL,
 movement_type VARCHAR(15) NOT NULL,
 quantity      INT NOT NULL,
 reason        VARCHAR(150),
 movement_date DATETIME2 DEFAULT SYSDATETIME(),
 CONSTRAINT PK_INVMOV_ID       PRIMARY KEY (movementid),
 CONSTRAINT FK_INVMOV_PRODUCT  FOREIGN KEY (productid) REFERENCES products(productid),
 CONSTRAINT FK_INVMOV_SELLER   FOREIGN KEY (sellerid)  REFERENCES sellers(sellerid),
 CONSTRAINT CHK_INVMOV_TYPE    CHECK (movement_type IN ('ENTRADA','SALIDA','AJUSTE','DEVOLUCION')),
 CONSTRAINT CHK_INVMOV_QTY     CHECK (quantity <> 0)
);
CREATE INDEX IX_INVMOV_PRODUCT ON inventory_movements(productid);
CREATE INDEX IX_INVMOV_DATE    ON inventory_movements(movement_date);
GO

-- ------------------------------------------------------------
-- TABLA: creditcards
-- Tarjetas de crédito asociadas a clientes. Un cliente puede
-- tener varias tarjetas. El número es PK (único globalmente).
```

```
-- ------------------------------------------------------------

CREATE TABLE creditcards(

 customerID       INT,

 creditcard       BIGINT,

 creditcard_type  VARCHAR(15),

 expiration_year  INT,

 expiration_month  INT,

 CONSTRAINT PK_CREDITCARDS_NUMBER  PRIMARY KEY (creditcard),

 CONSTRAINT FK_CREDITCARDS_CUST    FOREIGN KEY (customerid) REFERENCES customers(customerid),

 CONSTRAINT CHK_CC_MONTH          CHECK (expiration_month BETWEEN 1 AND 12),

 CONSTRAINT CHK_CC_YEAR           CHECK (expiration_year BETWEEN 2000 AND 2100),

 CONSTRAINT CHK_CC_TYPE           CHECK (creditcard_type IN ('VISA','MC','AMEX','Discover'))

);

CREATE INDEX IX_CREDITCARDS_CUSTID ON creditcards(customerid);

GO


-- ------------------------------------------------------------

-- TABLA: invoices

-- Facturas generadas por pedidos. Relaciona order, customer

-- y creditcard. Se usa para calcular comisiones bancarias.

-- ------------------------------------------------------------

CREATE TABLE invoices(

 invoiceid     INT IDENTITY(1,1) NOT NULL,

 orderid       INT,

 customerid    INT,

 creditcard    BIGINT,

 amount        DECIMAL(10,2),

 CONSTRAINT PK_INVOICES_INVOICEID  PRIMARY KEY (invoiceid),

 CONSTRAINT FK_INVOICES_ORDERS     FOREIGN KEY (orderid)    REFERENCES orders(orderid),

 CONSTRAINT FK_INVOICES_CUSTOMERS  FOREIGN KEY (customerid) REFERENCES customers(customerid),

 CONSTRAINT FK_INVOICES_CREDITCARD FOREIGN KEY (creditcard) REFERENCES creditcards(creditcard),

 CONSTRAINT CHK_INVOICES_AMOUNT    CHECK (amount > 0)
```

```
);
CREATE INDEX IX_INVOICES_ORDERID    ON invoices(orderid);
CREATE INDEX IX_INVOICES_CUSTOMERID ON invoices(customerid);
GO


-- ------------------------------------------------------------
-- TABLA: reviews
-- Reseñas de productos (1 a 5 estrellas + comentario).
-- Se identifica al autor por email (no FK para permitir reseñas
-- de usuarios no registrados, según diseño original).
-- ------------------------------------------------------------
CREATE TABLE reviews(
 reviewID       INT IDENTITY(1,1) NOT NULL,
 productID      INT,
 reviewer_email  VARCHAR(50),
 starsgiven     INT,
 review_text    VARCHAR(200),
 CONSTRAINT PK_REVIEWS_REVIEWID    PRIMARY KEY (reviewID),
 CONSTRAINT FK_REVIEWS_PRODUCTS    FOREIGN KEY (productID) REFERENCES products(productID),
 CONSTRAINT CHK_REVIEWS_STARS      CHECK (starsgiven BETWEEN 1 AND 5)
);
CREATE INDEX IX_REVIEWS_PRODUCTID ON reviews(productID);
CREATE INDEX IX_REVIEWS_EMAIL     ON reviews(reviewer_email);
GO


-- ------------------------------------------------------------
-- TABLA: recommendations
-- Recomendaciones personalizadas producto-cliente.
-- ------------------------------------------------------------
CREATE TABLE recommendations(
 recommendationid          INT IDENTITY(1,1) NOT NULL,
 customerid                INT,
```

```
 recommendation_productid   INT,
 CONSTRAINT PK_RECOMMENDATIONS_ID  PRIMARY KEY (recommendationid),
 CONSTRAINT FK_REC_CUSTOMERS       FOREIGN KEY (customerid) REFERENCES customers(customerid),
 CONSTRAINT FK_REC_PRODUCTS        FOREIGN KEY (recommendation_productid) REFERENCES products(productid)
);
CREATE INDEX IX_REC_CUSTOMERID ON recommendations(customerid);
GO

-- ------------------------------------------------------------
-- TABLA: audit_log
-- Bitácora de auditoría. Los triggers AFTER INSERT/UPDATE/DELETE
-- de las tablas principales escriben aquí. Dos triggers impiden
-- modificar o borrar registros de esta tabla.
-- ------------------------------------------------------------
CREATE TABLE audit_log(
 audit_id      INT IDENTITY(1,1) PRIMARY KEY,
 table_name    VARCHAR(50)  NOT NULL,
 action        VARCHAR(20)  NOT NULL,
 record_id     INT,
 user_name     VARCHAR(50)  DEFAULT SUSER_SNAME(),
 action_date   DATETIME2    DEFAULT SYSDATETIME(),
 old_values    NVARCHAR(MAX),
 new_values    NVARCHAR(MAX),
 CONSTRAINT CHK_AUDIT_ACTION CHECK (action IN ('INSERT','UPDATE','DELETE'))
);
CREATE INDEX IX_AUDIT_TABLE_DATE ON audit_log(table_name, action_date);
CREATE INDEX IX_AUDIT_USER       ON audit_log(user_name);
CREATE INDEX IX_AUDIT_RECORD     ON audit_log(record_id);
GO

/* ============================================================
```

```
  PASO 2: TRIGGERS

  ------------------------------------------------------------

  Los triggers de auditoría registran cada cambio en audit_log.

  El trigger report_low_inventory alerta stock < 50.

  Los triggers trg_proteger_audit_* impiden alterar la bitácora.

  ============================================================ */


-- Trigger: audita INSERT/UPDATE/DELETE sobre products

CREATE OR ALTER TRIGGER trg_audit_products

ON products AFTER INSERT, UPDATE, DELETE

AS

BEGIN

   SET NOCOUNT ON;


   -- Auditoría de INSERT

   INSERT INTO audit_log(table_name, action, record_id, new_values)

   SELECT 'PRODUCTS', 'INSERT', i.productid,

        'productoid=' + CAST(i.productid AS VARCHAR(10)) +

        ', nombre=' + i.productname +

        ', cantidad=' + CAST(i.availablequantity AS VARCHAR(10)) +

        ', precio=' + CAST(i.unitprice AS VARCHAR(20))

   FROM inserted i

   WHERE NOT EXISTS (SELECT 1 FROM deleted d WHERE d.productid = i.productid);


   -- Auditoría de UPDATE

   INSERT INTO audit_log(table_name, action, record_id, old_values, new_values)

   SELECT 'PRODUCTS', 'UPDATE', i.productid,

        'productoid=' + CAST(d.productid AS VARCHAR(10)) +

        ', cantidad=' + CAST(d.availablequantity AS VARCHAR(10)) +

        ', precio=' + CAST(d.unitprice AS VARCHAR(20)),

        'productoid=' + CAST(i.productid AS VARCHAR(10)) +

        ', cantidad=' + CAST(i.availablequantity AS VARCHAR(10)) +
```

```
        ', precio=' + CAST(i.unitprice AS VARCHAR(20))
    FROM inserted i
    INNER JOIN deleted d ON i.productid = d.productid;

    -- Auditoría de DELETE
    INSERT INTO audit_log(table_name, action, record_id, old_values)
    SELECT 'PRODUCTS', 'DELETE', d.productid,
        'productoid=' + CAST(d.productid AS VARCHAR(10)) +
        ', nombre=' + d.productname
    FROM deleted d
    WHERE NOT EXISTS (SELECT 1 FROM inserted i WHERE i.productid = d.productid);
END;
GO

-- Trigger: audita cambios sobre invoices
CREATE OR ALTER TRIGGER trg_audit_invoices
ON invoices AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO audit_log(table_name, action, record_id, new_values)
    SELECT 'INVOICES', 'INSERT', i.invoiceid, 'monto=' + CAST(i.amount AS VARCHAR(20))
    FROM inserted i
    WHERE NOT EXISTS (SELECT 1 FROM deleted d WHERE d.invoiceid = i.invoiceid);

    INSERT INTO audit_log(table_name, action, record_id, old_values, new_values)
    SELECT 'INVOICES', 'UPDATE', i.invoiceid,
        'monto=' + CAST(d.amount AS VARCHAR(20)),
        'monto=' + CAST(i.amount AS VARCHAR(20))
    FROM inserted i
    INNER JOIN deleted d ON i.invoiceid = d.invoiceid;
```

```
    INSERT INTO audit_log(table_name, action, record_id, old_values)
    SELECT 'INVOICES', 'DELETE', d.invoiceid, 'monto=' + CAST(d.amount AS VARCHAR(20))
    FROM deleted d
    WHERE NOT EXISTS (SELECT 1 FROM inserted i WHERE i.invoiceid = d.invoiceid);
END;
GO

-- Trigger: audita cambios sobre customers
CREATE OR ALTER TRIGGER trg_audit_customers
ON customers AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO audit_log(table_name, action, record_id, new_values)
    SELECT 'CUSTOMERS', 'INSERT', i.customerid,
        'nombre=' + i.name + ', correo=' + i.email
    FROM inserted i
    WHERE NOT EXISTS (SELECT 1 FROM deleted d WHERE d.customerid = i.customerid);

    INSERT INTO audit_log(table_name, action, record_id, old_values, new_values)
    SELECT 'CUSTOMERS', 'UPDATE', i.customerid,
        'nombre=' + d.name + ', correo=' + d.email,
        'nombre=' + i.name + ', correo=' + i.email
    FROM inserted i
    INNER JOIN deleted d ON i.customerid = d.customerid;

    INSERT INTO audit_log(table_name, action, record_id, old_values)
    SELECT 'CUSTOMERS', 'DELETE', d.customerid,
        'nombre=' + d.name + ', correo=' + d.email
    FROM deleted d
```

```
    WHERE NOT EXISTS (SELECT 1 FROM inserted i WHERE i.customerid = d.customerid);
END;
GO

-- Trigger: audita cambios sobre sellers
CREATE OR ALTER TRIGGER trg_audit_sellers
ON sellers AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO audit_log(table_name, action, record_id, new_values)
    SELECT 'SELLERS', 'INSERT', i.sellerid,
        'nombre=' + i.name + ', correo=' + i.email + ', rol=' + i.role
    FROM inserted i
    WHERE NOT EXISTS (SELECT 1 FROM deleted d WHERE d.sellerid = i.sellerid);

    INSERT INTO audit_log(table_name, action, record_id, old_values, new_values)
    SELECT 'SELLERS', 'UPDATE', i.sellerid,
        'rol=' + d.role + ', activo=' + CAST(d.active AS VARCHAR(2)),
        'rol=' + i.role + ', activo=' + CAST(i.active AS VARCHAR(2))
    FROM inserted i
    INNER JOIN deleted d ON i.sellerid = d.sellerid;

    INSERT INTO audit_log(table_name, action, record_id, old_values)
    SELECT 'SELLERS', 'DELETE', d.sellerid,
        'nombre=' + d.name + ', rol=' + d.role
    FROM deleted d
    WHERE NOT EXISTS (SELECT 1 FROM inserted i WHERE i.sellerid = d.sellerid);
END;
GO
```

```
-- Trigger: audita cambios sobre order_details
CREATE OR ALTER TRIGGER trg_audit_order_details
ON order_details AFTER INSERT, UPDATE, DELETE
AS
BEGIN
   SET NOCOUNT ON;

   INSERT INTO audit_log(table_name, action, record_id, new_values)
   SELECT 'ORDER_DETAILS', 'INSERT', i.detailid,
        'pedidoid=' + CAST(i.orderid AS VARCHAR(10)) +
        ', productoid=' + CAST(i.productid AS VARCHAR(10)) +
        ', cantidad=' + CAST(i.quantity AS VARCHAR(10))
   FROM inserted i
   WHERE NOT EXISTS (SELECT 1 FROM deleted d WHERE d.detailid = i.detailid);

   INSERT INTO audit_log(table_name, action, record_id, old_values, new_values)
   SELECT 'ORDER_DETAILS', 'UPDATE', i.detailid,
        'cantidad=' + CAST(d.quantity AS VARCHAR(10)),
        'cantidad=' + CAST(i.quantity AS VARCHAR(10))
   FROM inserted i
   INNER JOIN deleted d ON i.detailid = d.detailid;

   INSERT INTO audit_log(table_name, action, record_id, old_values)
   SELECT 'ORDER_DETAILS', 'DELETE', d.detailid,
        'pedidoid=' + CAST(d.orderid AS VARCHAR(10)) +
        ', productoid=' + CAST(d.productid AS VARCHAR(10))
   FROM deleted d
   WHERE NOT EXISTS (SELECT 1 FROM inserted i WHERE i.detailid = d.detailid);
END;
GO

-- Trigger: alerta cuando el stock cae por debajo de 50 unidades
```

```
CREATE OR ALTER TRIGGER report_low_inventory
ON products AFTER INSERT, UPDATE
AS
BEGIN
   SET NOCOUNT ON;
   DECLARE @msg NVARCHAR(MAX) = '';

   SELECT @msg = @msg + '*** ALERTA DE INVENTARIO BAJO ***' + CHAR(13) +
                 'ID del producto: ' + CAST(i.productid AS VARCHAR(10)) + CHAR(13) +
                 'Nombre del producto: ' + i.productname + CHAR(13) +
                 'Cantidad disponible: ' + CAST(i.availablequantity AS VARCHAR(10)) + CHAR(13) +
                 '----------------------------' + CHAR(13)
   FROM inserted i
   WHERE i.availablequantity < 50;

   IF LEN(@msg) > 0 PRINT @msg;
END;
GO

-- Trigger: impide modificar la bitácora de auditoría
CREATE OR ALTER TRIGGER trg_proteger_audit_update
ON audit_log AFTER UPDATE
AS
BEGIN
   SET NOCOUNT ON;
   ROLLBACK TRANSACTION;
   THROW 50002, 'No se permite modificar la bitácora de auditoría.', 1;
END;
GO

-- Trigger: impide borrar registros de la bitácora de auditoría
CREATE OR ALTER TRIGGER trg_proteger_audit_delete
```

```
ON audit_log AFTER DELETE

AS

BEGIN

   SET NOCOUNT ON;

   ROLLBACK TRANSACTION;

   THROW 50003, 'No se permite borrar registros de la bitácora de auditoría.', 1;

END;

GO




/* ============================================================

   PASO 3: FUNCIÓN AUXILIAR

   ------------------------------------------------------------

   Dado el nombre de una categoría, devuelve su ID. Si no

   existe retorna -1. Se usa desde Add_Product.

   ============================================================ */


CREATE FUNCTION dbo.FIND_PRODUCT_CATEGORY_ID

(

   @category_n VARCHAR(50)

)

RETURNS INT

AS

BEGIN

   RETURN ISNULL(

      (SELECT categoryid FROM ProductCategories WHERE categoryname = @category_n),

      -1

   );

END;

GO
```

```
/* ============================================================
   PASO 4: PROCEDIMIENTOS ALMACENADOS
   ------------------------------------------------------------
   Se agrupan por módulos del sistema. Cada SP encapsula una
   operación de negocio con manejo de errores vía TRY/CATCH.
   ============================================================ */


-- ============================================================
-- GRUPO 1: CLIENTES Y VENDEDORES
-- ============================================================


-- SP: registra un cliente + su tarjeta en una sola transacción
CREATE OR ALTER PROCEDURE add_customer
    @p_name            VARCHAR(100),
    @p_email           VARCHAR(100),
    @p_city            VARCHAR(50),
    @p_state           VARCHAR(50),
    @p_zip             INT,
    @p_password        VARCHAR(255),
    @p_creditcard      BIGINT,
    @p_creditcard_type  VARCHAR(15),
    @p_expiration_month INT,
    @p_expiration_year  INT,
    @p_telefono        VARCHAR(20) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO customers (name, email, city, state, zip, phone, password_hash, active, debe_cambiar_password)
        VALUES (@p_name, @p_email, @p_city, @p_state, @p_zip, @p_telefono, @p_password, 1, 1);
```

```
        DECLARE @v_customerid INT = SCOPE_IDENTITY();

        INSERT INTO creditcards (customerid, creditcard, creditcard_type, expiration_month, expiration_year)
        VALUES (@v_customerid, @p_creditcard, @p_creditcard_type, @p_expiration_month, @p_expiration_year);

        COMMIT TRANSACTION;
        PRINT 'Cliente agregado con ID: ' + CAST(@v_customerid AS VARCHAR(10));
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        PRINT 'Error al agregar cliente: ' + ERROR_MESSAGE();
    END CATCH
END;
GO

-- SP: registra un cliente desde el portal web (tarjeta opcional)
CREATE OR ALTER PROCEDURE add_cliente_completo
    @p_name            VARCHAR(100),
    @p_email           VARCHAR(100),
    @p_city            VARCHAR(50),
    @p_state           VARCHAR(50),
    @p_zip             INT,
    @p_password_hash   VARCHAR(255),
    @p_telefono        VARCHAR(20) = NULL,
    @p_creditcard      BIGINT = NULL,
    @p_creditcard_type VARCHAR(15) = NULL,
    @p_expiration_month INT = NULL,
    @p_expiration_year  INT = NULL
AS
BEGIN
    SET NOCOUNT ON;
```

```
    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO customers (name, email, city, state, zip, phone, password_hash, active)
        VALUES (@p_name, @p_email, @p_city, @p_state, @p_zip, @p_telefono, @p_password_hash, 1);

        DECLARE @v_customerid INT = SCOPE_IDENTITY();

        IF @p_creditcard IS NOT NULL AND @p_creditcard_type IS NOT NULL
        BEGIN
            IF NOT EXISTS (SELECT 1 FROM creditcards WHERE creditcard = @p_creditcard)
            BEGIN
                INSERT INTO creditcards (customerid, creditcard, creditcard_type, expiration_month, expiration_year)
                VALUES (@v_customerid, @p_creditcard, @p_creditcard_type, @p_expiration_month, @p_expiration_year);
            END
        END

        COMMIT TRANSACTION;
        PRINT 'Cliente registrado con ID: ' + CAST(@v_customerid AS VARCHAR(10));
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        PRINT 'Error: ' + ERROR_MESSAGE();
        THROW;
    END CATCH
END;
GO

-- SP: inserta un vendedor/administrador validando el rol
CREATE OR ALTER PROCEDURE add_seller
    @p_name         VARCHAR(50),
    @p_email        VARCHAR(50),
```

```
    @p_password_hash VARCHAR(255),
    @p_role          VARCHAR(15)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @p_role NOT IN ('ADMIN','VENDEDOR')
        BEGIN
            PRINT 'Rol inválido. Debe ser ADMIN o VENDEDOR.';
            RETURN;
        END

        INSERT INTO sellers (name, email, password_hash, role)
        VALUES (@p_name, @p_email, @p_password_hash, @p_role);

        PRINT 'Vendedor agregado con ID: ' + CAST(SCOPE_IDENTITY() AS VARCHAR(10));
    END TRY
    BEGIN CATCH
        PRINT 'Error al agregar vendedor: ' + ERROR_MESSAGE();
    END CATCH
END;
GO

-- SP: lista todos los vendedores ordenados por rol y nombre
CREATE OR ALTER PROCEDURE show_sellers
AS
BEGIN
    SET NOCOUNT ON;
    SELECT sellerid   AS [ID del Vendedor],
           name       AS [Nombre],
           email      AS [Correo],
           role       AS [Rol],
```

```
        active     AS [Activo],
        created_at AS [Fecha de Creación]
    FROM sellers
    ORDER BY role, name;
END;
GO

-- SP: usa CURSOR para recorrer clientes de un estado y mostrarlos
CREATE OR ALTER PROCEDURE show_all_customers_in_state
    @p_state VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @v_count INT = 0;

    DECLARE cur CURSOR FOR
        SELECT c.name, c.email, c.city, cc.creditcard, cc.creditcard_type
        FROM customers c
        JOIN creditcards cc ON c.customerid = cc.customerid
        WHERE c.state = @p_state;

    DECLARE @name VARCHAR(100), @email VARCHAR(100), @city VARCHAR(50),
            @card BIGINT, @card_type VARCHAR(15);

    OPEN cur;
    FETCH NEXT FROM cur INTO @name, @email, @city, @card, @card_type;
    WHILE @@FETCH_STATUS = 0
    BEGIN
        PRINT 'Nombre: ' + @name;
        PRINT 'Correo: ' + @email;
        PRINT 'Dirección: ' + @city + ', ' + @p_state;
        PRINT 'Tarjeta: ' + CAST(@card AS VARCHAR(20)) + ' tipo: ' + @card_type;
```

```
        PRINT '----------------------';
        SET @v_count = @v_count + 1;
        FETCH NEXT FROM cur INTO @name, @email, @city, @card, @card_type;
    END
    CLOSE cur;
    DEALLOCATE cur;

    IF @v_count = 0
        PRINT 'No se encontraron clientes en el estado: ' + @p_state;
END;
GO

-- ============================================================
-- GRUPO 2: CATÁLOGO
-- ============================================================

-- SP: reporta tarjetas que expiran en los próximos 2 meses
CREATE OR ALTER PROCEDURE Report_Cards_Expire
    @expire_date DATE
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @v_customer_info NVARCHAR(MAX) = '';
    DECLARE @found BIT = 0;

    SELECT @v_customer_info = @v_customer_info + c.name + ' - ' +
                    CAST(cc.creditcard AS VARCHAR(20)) + ' (' + cc.creditcard_type + ')' + CHAR(13),
        @found = 1
    FROM customers c
    JOIN creditcards cc ON c.customerID = cc.customerID
    WHERE DATEFROMPARTS(cc.expiration_year, cc.expiration_month, 1)
        BETWEEN DATEADD(MONTH, -2, @expire_date) AND @expire_date;
```

```
    IF @found = 0
        PRINT 'No hay tarjetas de crédito que expiren en los próximos dos meses.';
    ELSE
    BEGIN
        PRINT 'Las siguientes tarjetas de crédito expiran en los próximos dos meses:';
        PRINT @v_customer_info;
    END
END;
GO


-- SP: inserta una categoría de producto controlando duplicados
CREATE OR ALTER PROCEDURE Add_Category
    @category_name VARCHAR(50),
    @category_desc VARCHAR(150)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        INSERT INTO ProductCategories (categoryname, description)
        VALUES (@category_name, @category_desc);
        PRINT 'Categoría agregada exitosamente: ' + @category_name;
    END TRY
    BEGIN CATCH
        IF ERROR_NUMBER() = 2627
            PRINT 'La categoría ya existe: ' + @category_name;
        ELSE
        BEGIN
            PRINT 'SQLCODE: ' + CAST(ERROR_NUMBER() AS VARCHAR(10));
            PRINT 'SQLERRM: ' + ERROR_MESSAGE();
        END
    END CATCH
```

```
END;
GO

-- SP: inserta un producto usando la función FIND_PRODUCT_CATEGORY_ID
CREATE OR ALTER PROCEDURE Add_Product
   @product_name      VARCHAR(100),
   @available_quantity INT,
   @unit_price        DECIMAL(10,2),
   @category_name     VARCHAR(50)
AS
BEGIN
   SET NOCOUNT ON;
   BEGIN TRY
      DECLARE @category_ID INT;
      SET @category_ID = dbo.FIND_PRODUCT_CATEGORY_ID(@category_name);

      IF @category_ID = -1
      BEGIN
         PRINT 'Producto no agregado - categoría no encontrada: ' + @category_name;
         RETURN;
      END

      INSERT INTO products (productname, availablequantity, unitprice, categoryid)
      VALUES (@product_name, @available_quantity, @unit_price, @category_ID);

      PRINT 'Producto agregado exitosamente: ' + @product_name;
   END TRY
   BEGIN CATCH
      PRINT 'SQLCODE: ' + CAST(ERROR_NUMBER() AS VARCHAR(10));
      PRINT 'SQLERRM: ' + ERROR_MESSAGE();
   END CATCH
END;
```

```
GO


-- SP: resumen de inventario agrupado por categoría
CREATE OR ALTER PROCEDURE Report_Inventory
AS
BEGIN
   SET NOCOUNT ON;
   PRINT 'Nombre de categoría: Total de productos en existencia';
   PRINT '------------------------------------------------';


   SELECT pc.categoryname + ': ' + CAST(ISNULL(SUM(p.availablequantity), 0) AS VARCHAR(20)) AS [Reporte por Categoría]
   FROM ProductCategories pc
   LEFT JOIN products p ON pc.categoryid = p.categoryid
   GROUP BY pc.categoryname;
END;
GO


-- ============================================================
-- GRUPO 3: PEDIDOS Y VENTAS
-- ============================================================


-- SP: obtiene customerid por email (OUTPUT)
CREATE OR ALTER PROCEDURE find_customer_id
   @email_address VARCHAR(100),
   @customer_id   INT OUTPUT
AS
BEGIN
   SET NOCOUNT ON;
   SET @customer_id = -1;


   SELECT @customer_id = customerid FROM customers WHERE email = @email_address;
```

```
    IF @customer_id = -1
        PRINT 'La dirección de correo no existe: ' + @email_address;
    ELSE
        PRINT 'El ID del cliente es ' + CAST(@customer_id AS VARCHAR(10));
END;
GO

-- SP: obtiene productid por nombre (OUTPUT)
CREATE OR ALTER PROCEDURE find_product_id
    @product_name VARCHAR(100),
    @product_id   INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET @product_id = -1;

    SELECT @product_id = productid FROM products WHERE productname = @product_name;

    IF @product_id = -1
        PRINT 'El nombre del producto no existe: ' + @product_name;
    ELSE
        PRINT 'El ID del producto es ' + CAST(@product_id AS VARCHAR(10));
END;
GO

-- SP: busca el primer pedido de una fecha dada (OUTPUT)
CREATE OR ALTER PROCEDURE find_order_by_date
    @p_order_date DATE,
    @p_order_id   INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
```

```
    SET @p_order_id = -1;

    SELECT TOP 1 @p_order_id = orderid FROM orders WHERE orderdate = @p_order_date;

    IF @p_order_id = -1
        PRINT 'No se encontró pedido para la fecha ' + CAST(@p_order_date AS VARCHAR(20));
    ELSE
        PRINT 'ID del pedido: ' + CAST(@p_order_id AS VARCHAR(10));
END;
GO

-- SP PRINCIPAL: registra un pedido completo (cabecera + detalle + factura)
-- Usa OPENJSON para recibir el detalle como JSON: [{"productid":1,"quantity":2}]
CREATE OR ALTER PROCEDURE place_order_full
    @cust_id     INT,
    @seller_id   INT = NULL,
    @order_date  DATETIME,
    @cc_num      BIGINT = NULL,
    @items       NVARCHAR(MAX),
    @amount      DECIMAL(10,2) = 0,
    @status      VARCHAR(15) = 'PENDIENTE'
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        -- Si no hay vendedor asignado, se asigna al Admin (ID 1)
        DECLARE @vendedor_final INT = @seller_id;
        IF (@vendedor_final IS NULL OR @vendedor_final = 0)
        BEGIN
            SET @vendedor_final = 1;
```

```
    END

    INSERT INTO orders (customerid, sellerid, orderdate, cc_number, status, total)
    VALUES (@cust_id, @vendedor_final, @order_date, @cc_num, @status, @amount);

    DECLARE @oid INT = SCOPE_IDENTITY();

    INSERT INTO order_details (orderid, productid, quantity, unitprice)
    SELECT @oid, j.productid, j.quantity, p.unitprice
    FROM OPENJSON(@items)
        WITH (productid INT '$.productid', quantity INT '$.quantity') AS j
    JOIN products p ON p.productid = j.productid;

    UPDATE orders
    SET total = (SELECT ISNULL(SUM(subtotal), 0) FROM order_details WHERE orderid = @oid)
    WHERE orderid = @oid;

    -- Solo se factura si el pago fue con tarjeta (cc_num > 0)
    IF @amount > 0 AND @cc_num IS NOT NULL AND @cc_num > 0
    BEGIN
      INSERT INTO invoices (orderid, customerid, creditcard, amount)
      VALUES (@oid, @cust_id, @cc_num, @amount);
    END

    COMMIT TRANSACTION;
    PRINT 'Pedido ' + CAST(@oid AS VARCHAR(10)) + ' registrado correctamente con estado: ' + @status;
  END TRY
  BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    PRINT 'Error al registrar pedido: ' + ERROR_MESSAGE();
  END CATCH
END;
```

```
GO


-- SP: wrapper para agregar un pedido de un solo producto
CREATE OR ALTER PROCEDURE add_orders
   @cust_id    INT,
   @prod_id    INT,
   @quant      INT,
   @order_date DATE,
   @cc_num     BIGINT
AS
BEGIN
   SET NOCOUNT ON;
   BEGIN TRY
      DECLARE @items NVARCHAR(MAX) =
         '[{"productid":' + CAST(@prod_id AS VARCHAR(10)) +
         ',"quantity":'  + CAST(@quant   AS VARCHAR(10)) + '}]';

      EXEC place_order_full @cust_id, NULL, @order_date, @cc_num, @items, 0, 'CONFIRMADO';
   END TRY
   BEGIN CATCH
      PRINT 'Error al agregar pedido: ' + ERROR_MESSAGE();
   END CATCH
END;
GO


-- SP: crea un pedido por email del cliente y nombre del producto
CREATE OR ALTER PROCEDURE place_order
   @cust_email   VARCHAR(100),
   @product_name VARCHAR(100),
   @quant        INT,
   @cc_num       BIGINT,
   @amt          DECIMAL(10,2),
```

```
    @order_date   DATE
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        DECLARE @cust_id INT, @prod_id INT;

        EXEC find_customer_id @cust_email, @cust_id OUTPUT;
        EXEC find_product_id  @product_name, @prod_id OUTPUT;

        IF @cust_id = -1 OR @prod_id = -1
        BEGIN
            PRINT 'Pedido no realizado - datos faltantes.';
            RETURN;
        END

        DECLARE @items NVARCHAR(MAX) =
            '[{"productid":' + CAST(@prod_id AS VARCHAR(10)) +
            ',"quantity":'  + CAST(@quant   AS VARCHAR(10)) + '}]';

        EXEC place_order_full @cust_id, NULL, @order_date, @cc_num, @items, @amt, 'CONFIRMADO';

        PRINT 'Pedido realizado exitosamente.';
    END TRY
    BEGIN CATCH
        PRINT 'Error al realizar pedido: ' + ERROR_MESSAGE();
    END CATCH
END;
GO

-- SP: muestra pedidos + resumen global
CREATE OR ALTER PROCEDURE show_orders
```

```
AS
BEGIN
   SET NOCOUNT ON;

   SELECT c.name         AS [Cliente],
          p.productname  AS [Producto],
          od.quantity    AS [Cantidad],
          od.subtotal    AS [Subtotal],
          o.status       AS [Estado]
   FROM orders o
   JOIN customers c     ON o.customerid = c.customerid
   JOIN order_details od ON o.orderid   = od.orderid
   JOIN products p      ON od.productid = p.productid;

   SELECT COUNT(DISTINCT o.orderid)   AS [Total de Pedidos],
          ISNULL(SUM(od.subtotal), 0) AS [Monto Total]
   FROM orders o
   JOIN order_details od ON o.orderid = od.orderid;
END;
GO

-- SP: pedidos por estado geográfico (state)
CREATE OR ALTER PROCEDURE report_orders_by_state
   @c_state VARCHAR(50)
AS
BEGIN
   SET NOCOUNT ON;

   SELECT c.name  AS [Nombre del Cliente],
          c.email AS [Correo del Cliente],
          COUNT(DISTINCT o.orderid)   AS [Total de Pedidos],
          ISNULL(SUM(od.subtotal), 0) AS [Monto Total]
```

```
    FROM orders o
    INNER JOIN customers c     ON o.customerid = c.customerid
    INNER JOIN order_details od ON o.orderid   = od.orderid
    WHERE c.state = @c_state
    GROUP BY c.name, c.email;

    SELECT COUNT(DISTINCT o.orderid)   AS [Total General de Pedidos],
        ISNULL(SUM(od.subtotal), 0) AS [Monto General Total]
    FROM orders o
    INNER JOIN customers c ON o.customerid = c.customerid
    INNER JOIN order_details od ON o.orderid = od.orderid
    WHERE c.state = @c_state;
END;
GO

-- SP: mejores clientes cuyo monto total supere @a
CREATE OR ALTER PROCEDURE report_best_customers
    @a INT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (
        SELECT 1 FROM order_details od
        INNER JOIN orders o ON o.orderid = od.orderid
        WHERE od.subtotal > @a
    )
    BEGIN
        PRINT 'No existe ningún cliente con monto mayor a ' + CAST(@a AS VARCHAR(20));
        RETURN;
    END
```

```
    SELECT c.name         AS [Nombre del Cliente],
           SUM(od.subtotal) AS [Monto Total]
    FROM order_details od
    INNER JOIN orders o    ON o.orderid = od.orderid
    INNER JOIN customers c ON c.customerid = o.customerid
    GROUP BY c.name
    HAVING SUM(od.subtotal) > @a
    ORDER BY SUM(od.subtotal) DESC;
END;
GO

-- SP: TOP X clientes que menos han gastado (tacaños)
CREATE OR ALTER PROCEDURE find_stingy_customers
    @X INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @counter1 INT;
    SELECT @counter1 = COUNT(DISTINCT customerid) FROM orders;

    IF @counter1 < @X
    BEGIN
        PRINT 'Máximo de clientes: ' + CAST(@counter1 AS VARCHAR(10)) +
              ' | Valor ingresado: ' + CAST(@X AS VARCHAR(10));
    END

    IF @X <= 0
    BEGIN
        PRINT 'Ingrese un número válido mayor a 0';
        RETURN;
    END
```

```
    SELECT TOP (@X) o.customerid     AS [ID del Cliente],
                SUM(od.subtotal) AS [Total Gastado]
    FROM orders o
    JOIN order_details od ON o.orderid = od.orderid
    GROUP BY o.customerid
    ORDER BY SUM(od.subtotal) ASC;
END;
GO

-- ============================================================
-- GRUPO 4: FACTURACIÓN Y PAGOS
-- ============================================================

-- SP: inserta una factura validando pedido, cliente y tarjeta
CREATE OR ALTER PROCEDURE invoice_customer
    @p_order_id       INT,
    @p_customer_id    INT,
    @p_credit_card_num BIGINT,
    @p_amount         DECIMAL(10,2)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        DECLARE @counter1 INT, @counter2 INT, @cc_exists INT;

        SELECT @counter1 = COUNT(*) FROM orders    WHERE orderid    = @p_order_id;
        SELECT @counter2 = COUNT(*) FROM customers WHERE customerid = @p_customer_id;

        IF @counter1 <= 0 AND @counter2 > 0 BEGIN PRINT 'No existe ese pedido'; RETURN; END
        IF @counter2 <= 0 AND @counter1 > 0 BEGIN PRINT 'No existe ese cliente'; RETURN; END
        IF @counter2 <= 0 AND @counter1 <= 0 BEGIN PRINT 'No existe ni el pedido ni el cliente'; RETURN; END
```

```
      SELECT @cc_exists = COUNT(*) FROM creditcards WHERE creditcard = @p_credit_card_num;
      IF @cc_exists = 0
      BEGIN
         PRINT 'La tarjeta de crédito no existe: ' + CAST(@p_credit_card_num AS VARCHAR(20));
         RETURN;
      END

      INSERT INTO invoices (orderid, customerid, creditcard, amount)
      VALUES (@p_order_id, @p_customer_id, @p_credit_card_num, @p_amount);

      PRINT 'Factura creada correctamente.';
   END TRY
   BEGIN CATCH
      PRINT 'Error al crear factura: ' + ERROR_MESSAGE();
   END CATCH
END;
GO

-- SP: calcula comisiones según tipo de tarjeta
-- VISA/MC 3%, AMEX 5%, Discover 2%
CREATE OR ALTER PROCEDURE calculate_credit_card_fees
AS
BEGIN
   SET NOCOUNT ON;

   DECLARE @visa_fee     DECIMAL(10,2) = 0;
   DECLARE @mc_fee       DECIMAL(10,2) = 0;
   DECLARE @amex_fee     DECIMAL(10,2) = 0;
   DECLARE @discover_fee DECIMAL(10,2) = 0;

   SELECT @visa_fee     = ISNULL(SUM(i.amount * 0.03), 0)
```

```
    FROM invoices i JOIN creditcards c ON i.creditcard = c.creditcard
    WHERE c.creditcard_type = 'VISA';

    SELECT @mc_fee       = ISNULL(SUM(i.amount * 0.03), 0)
    FROM invoices i JOIN creditcards c ON i.creditcard = c.creditcard
    WHERE c.creditcard_type = 'MC';

    SELECT @amex_fee     = ISNULL(SUM(i.amount * 0.05), 0)
    FROM invoices i JOIN creditcards c ON i.creditcard = c.creditcard
    WHERE c.creditcard_type = 'AMEX';

    SELECT @discover_fee = ISNULL(SUM(i.amount * 0.02), 0)
    FROM invoices i JOIN creditcards c ON i.creditcard = c.creditcard
    WHERE c.creditcard_type = 'Discover';

    PRINT 'Comisión VISA:     $' + CAST(@visa_fee     AS VARCHAR(20));
    PRINT 'Comisión MC:       $' + CAST(@mc_fee       AS VARCHAR(20));
    PRINT 'Comisión AMEX:     $' + CAST(@amex_fee     AS VARCHAR(20));
    PRINT 'Comisión Discover: $' + CAST(@discover_fee AS VARCHAR(20));
END;
GO

-- ============================================================
-- GRUPO 5: INVENTARIO
-- ============================================================

-- SP: resta cantidad al stock (salida)
CREATE OR ALTER PROCEDURE Update_Inventory
    @product_id INT, @quantity INT
AS
BEGIN
    SET NOCOUNT ON;
```

```
    BEGIN TRY
        UPDATE products SET availablequantity = availablequantity - @quantity
        WHERE productid = @product_id;

        IF @@ROWCOUNT = 0
        BEGIN
            PRINT 'No se encontró inventario para el producto con ID ' + CAST(@product_id AS VARCHAR(10));
            RETURN;
        END

        DECLARE @new_quant INT;
        SELECT @new_quant = availablequantity FROM products WHERE productid = @product_id;
        PRINT 'Cantidad actualizada: ' + CAST(@new_quant AS VARCHAR(10));
    END TRY
    BEGIN CATCH
        PRINT 'Error: ' + ERROR_MESSAGE();
    END CATCH
END;
GO

-- SP: suma cantidad al stock (entrada)
CREATE OR ALTER PROCEDURE update_inventory1
    @product_id INT, @quantity INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        UPDATE products SET availablequantity = availablequantity + @quantity
        WHERE productid = @product_id;

        IF @@ROWCOUNT = 0
        BEGIN
```

```
        PRINT 'No se encontró inventario para el producto con ID ' + CAST(@product_id AS VARCHAR(10));
        RETURN;
      END

      DECLARE @new_quant INT;
      SELECT @new_quant = availablequantity FROM products WHERE productid = @product_id;
      PRINT 'Cantidad actualizada: ' + CAST(@new_quant AS VARCHAR(10));
    END TRY
    BEGIN CATCH
      PRINT 'Error: ' + ERROR_MESSAGE();
    END CATCH
END;
GO

-- SP: registra movimiento y ajusta stock según tipo
CREATE OR ALTER PROCEDURE register_inventory_movement
    @product_id    INT,
    @seller_id     INT = NULL,
    @movement_type VARCHAR(15),
    @quantity      INT,
    @reason        VARCHAR(150) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
      IF @movement_type NOT IN ('ENTRADA','SALIDA','AJUSTE','DEVOLUCION')
      BEGIN
        PRINT 'Tipo de movimiento inválido.';
        RETURN;
      END

      BEGIN TRANSACTION;
```

```
      INSERT INTO inventory_movements(productid, sellerid, movement_type, quantity, reason)
      VALUES (@product_id, @seller_id, @movement_type, @quantity, @reason);

      IF @movement_type IN ('ENTRADA','DEVOLUCION')
         UPDATE products SET availablequantity = availablequantity + @quantity WHERE productid = @product_id;
      ELSE IF @movement_type = 'SALIDA'
         UPDATE products SET availablequantity = availablequantity - @quantity WHERE productid = @product_id;
      ELSE IF @movement_type = 'AJUSTE'
         UPDATE products SET availablequantity = @quantity WHERE productid = @product_id;

      COMMIT TRANSACTION;
      PRINT 'Movimiento registrado.';
   END TRY
   BEGIN CATCH
      IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
      PRINT 'Error: ' + ERROR_MESSAGE();
   END CATCH
END;
GO

-- SP: histórico de movimientos por producto (o todos)
CREATE OR ALTER PROCEDURE report_inventory_movements
   @product_id INT = NULL
AS
BEGIN
   SET NOCOUNT ON;
   SELECT m.movementid    AS [ID del Movimiento],
          p.productname   AS [Producto],
          m.movement_type AS [Tipo de Movimiento],
          m.quantity      AS [Cantidad],
          m.reason        AS [Motivo],
```

```
        s.name         AS [Vendedor],
        m.movement_date AS [Fecha del Movimiento]
    FROM inventory_movements m
    JOIN products p ON m.productid = p.productid
    LEFT JOIN sellers s ON m.sellerid = s.sellerid
    WHERE (@product_id IS NULL OR m.productid = @product_id)
    ORDER BY m.movement_date DESC;
END;
GO

-- SP: ventas agrupadas por vendedor
CREATE OR ALTER PROCEDURE report_sales_by_seller
AS
BEGIN
    SET NOCOUNT ON;
    SELECT s.sellerid AS [ID del Vendedor],
        s.name     AS [Nombre del Vendedor],
        s.role     AS [Rol],
        COUNT(DISTINCT o.orderid)   AS [Total de Pedidos],
        ISNULL(SUM(od.subtotal), 0) AS [Total Vendido]
    FROM sellers s
    LEFT JOIN orders o        ON o.sellerid  = s.sellerid
    LEFT JOIN order_details od ON od.orderid  = o.orderid
    GROUP BY s.sellerid, s.name, s.role
    ORDER BY [Total Vendido] DESC;
END;
GO

-- ============================================================
-- GRUPO 6: RESEÑAS Y RECOMENDACIONES
-- ============================================================
```

```
-- SP: inserta una reseña validando email, estrellas y producto
CREATE OR ALTER PROCEDURE add_review
   @product_name VARCHAR(100),
   @cust_email   VARCHAR(100),
   @star_given   INT,
   @review_text  VARCHAR(200)
AS
BEGIN
   SET NOCOUNT ON;
   BEGIN TRY
      IF @cust_email IS NULL
      BEGIN PRINT 'No tiene un correo electrónico'; RETURN; END

      IF @star_given < 1 OR @star_given > 5
      BEGIN PRINT 'Las estrellas deben estar entre 1 y 5'; RETURN; END

      DECLARE @prod_id INT;
      EXEC find_product_id @product_name, @prod_id OUTPUT;

      IF @prod_id = -1
      BEGIN PRINT 'Reseña no agregada - producto no encontrado.'; RETURN; END

      INSERT INTO reviews (productid, reviewer_email, starsgiven, review_text)
      VALUES (@prod_id, @cust_email, @star_given, @review_text);

      PRINT 'Reseña insertada correctamente.';
   END TRY
   BEGIN CATCH
      PRINT 'Error al agregar reseña: ' + ERROR_MESSAGE();
   END CATCH
END;
GO
```

```
-- SP: TOP X mejor y peor calificados ("Buy or Beware")
CREATE OR ALTER PROCEDURE buy_or_beware
   @x INT
AS
BEGIN
   SET NOCOUNT ON;

   PRINT 'Productos mejor calificados:';
   SELECT TOP (@x)
        AVG(r.starsgiven)   AS [Calificación Promedio],
        STDEV(r.starsgiven) AS [Desviación Estándar],
        p.productid         AS [ID del Producto],
        p.productname       AS [Nombre del Producto]
   FROM reviews r
   JOIN products p ON r.productid = p.productid
   GROUP BY p.productid, p.productname
   ORDER BY AVG(r.starsgiven) DESC;

   PRINT 'Cuidado comprador: manténgase alejado de...';
   SELECT TOP (@x)
        AVG(r.starsgiven)   AS [Calificación Promedio],
        STDEV(r.starsgiven) AS [Desviación Estándar],
        p.productid         AS [ID del Producto],
        p.productname       AS [Nombre del Producto]
   FROM reviews r
   JOIN products p ON r.productid = p.productid
   GROUP BY p.productid, p.productname
   ORDER BY AVG(r.starsgiven) ASC;
END;
GO
```

```
-- SP: recomienda al cliente el producto mejor calificado de sus
-- categorías favoritas que aún no haya comprado
CREATE OR ALTER PROCEDURE Recommend_To_Customer
   @in_custid INT
AS
BEGIN
   SET NOCOUNT ON;
   BEGIN TRY
      DECLARE @tmp_productid INT, @tmp_productname VARCHAR(100),
            @tmp_categoryid INT, @tmp_avg_rating DECIMAL(10,2);

      SELECT TOP 1
            @tmp_avg_rating  = AVG(r.starsgiven),
            @tmp_categoryid  = p.categoryid,
            @tmp_productid   = p.productid,
            @tmp_productname = p.productname
      FROM reviews r
      JOIN products p ON r.productid = p.productid
      WHERE r.productid NOT IN (
            SELECT od.productid FROM order_details od
            JOIN orders o ON od.orderid = o.orderid
            WHERE o.customerid = @in_custid
          )
       AND p.categoryid IN (
            SELECT p2.categoryid FROM products p2
            JOIN order_details od2 ON p2.productid = od2.productid
            JOIN orders o2 ON od2.orderid = o2.orderid
            WHERE o2.customerid = @in_custid
          )
      GROUP BY p.categoryid, p.productid, p.productname
      ORDER BY AVG(r.starsgiven) DESC;
```

```
    IF @tmp_productid IS NULL
    BEGIN PRINT 'Producto recomendado no encontrado'; RETURN; END

    PRINT 'Producto recomendado: ' + @tmp_productname;

    DECLARE @tmp_count INT;
    SELECT @tmp_count = COUNT(*) FROM recommendations
    WHERE customerid = @in_custid AND recommendation_productid = @tmp_productid;

    IF @tmp_count > 0
    BEGIN PRINT 'Ya existe la recomendación.'; RETURN; END
    ELSE
    BEGIN
      INSERT INTO recommendations (customerid, recommendation_productid)
      VALUES (@in_custid, @tmp_productid);
    END
  END TRY
  BEGIN CATCH
    PRINT 'Producto recomendado no encontrado';
  END CATCH
END;
GO

-- SP: lista recomendaciones por cliente
CREATE OR ALTER PROCEDURE List_Recommendations
AS
BEGIN
  SET NOCOUNT ON;

  SELECT c.name      AS [Cliente],
      p.categoryid AS [Categoría],
      (SELECT TOP 1 p2.productname
```

```
        FROM reviews r JOIN products p2 ON r.productid = p2.productid
        WHERE p2.categoryid = p.categoryid
          AND p2.productid NOT IN (
            SELECT od.productid FROM order_details od
            JOIN orders o2 ON od.orderid = o2.orderid
            WHERE o2.customerid = o.customerid)
        GROUP BY p2.productname
        ORDER BY AVG(r.starsgiven) DESC) AS [Producto Recomendado]
  FROM products p
  JOIN order_details od ON p.productid = od.productid
  JOIN orders o         ON od.orderid  = o.orderid
  JOIN customers c      ON c.customerid = o.customerid
  GROUP BY c.name, p.categoryid, o.customerid;
END;
GO

-- ============================================================
-- GRUPO 7: INSERCIÓN ADICIONAL (wrappers)
-- ============================================================

-- SP: agrega una línea al detalle de un pedido
CREATE OR ALTER PROCEDURE agregar_detalle_pedido
  @order_id INT, @product_id INT, @quantity INT
AS
BEGIN
  SET NOCOUNT ON;
  BEGIN TRY
    DECLARE @precio DECIMAL(10,2);
    SELECT @precio = unitprice FROM products WHERE productid = @product_id;

    IF @precio IS NULL
    BEGIN PRINT 'Producto no encontrado.'; RETURN; END
```

```
        INSERT INTO order_details (orderid, productid, quantity, unitprice)
        VALUES (@order_id, @product_id, @quantity, @precio);

        PRINT 'Detalle agregado al pedido ' + CAST(@order_id AS VARCHAR(10));
    END TRY
    BEGIN CATCH
        PRINT 'Error: ' + ERROR_MESSAGE();
    END CATCH
END;
GO

-- SP: agrega una recomendación directa
CREATE OR ALTER PROCEDURE agregar_recomendacion
    @customer_id INT, @product_id INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        INSERT INTO recommendations (customerid, recommendation_productid)
        VALUES (@customer_id, @product_id);
        PRINT 'Recomendación agregada.';
    END TRY
    BEGIN CATCH
        PRINT 'Error: ' + ERROR_MESSAGE();
    END CATCH
END;
GO

-- SP: wrapper de register_inventory_movement
CREATE OR ALTER PROCEDURE agregar_movimiento_inventario
    @product_id INT, @seller_id INT = NULL, @movement_type VARCHAR(15),
```

```
    @quantity INT, @reason VARCHAR(150) = NULL
AS
BEGIN
    EXEC register_inventory_movement @product_id, @seller_id, @movement_type, @quantity, @reason;
END;
GO

-- SP: wrapper de invoice_customer
CREATE OR ALTER PROCEDURE agregar_factura
    @order_id INT, @customer_id INT, @credit_card_num BIGINT, @amount DECIMAL(10,2)
AS
BEGIN
    EXEC invoice_customer @order_id, @customer_id, @credit_card_num, @amount;
END;
GO

-- SP: wrapper de add_review
CREATE OR ALTER PROCEDURE agregar_resena
    @product_name VARCHAR(100), @cust_email VARCHAR(100),
    @star_given INT, @review_text VARCHAR(200)
AS
BEGIN
    EXEC add_review @product_name, @cust_email, @star_given, @review_text;
END;
GO


/* ============================================================
   PASO 5: CARGA DE DATOS DE PRUEBA
   ------------------------------------------------------------
   Se cargan 40 categorías, 10 productos, 4 vendedores,
   5 clientes, 4 pedidos, 2 movimientos y 6 reseñas.
```

```
   ============================================================ */


PRINT '=========================================';

PRINT ' INICIANDO CARGA DE DATOS';

PRINT '=========================================';


-- 1) CATEGORÍAS (40)

PRINT '--- Insertando 40 categorías ---';

EXEC Add_Category 'Electrónica',                'Dispositivos electrónicos y gadgets';

EXEC Add_Category 'Computadoras y Tablets',     'Laptops, PCs, tablets y accesorios';

EXEC Add_Category 'Celulares y Accesorios',     'Smartphones, fundas, cargadores';

EXEC Add_Category 'Audio y Video',              'Parlantes, audífonos, televisores';

EXEC Add_Category 'Videojuegos y Consolas',     'Consolas, juegos, accesorios gamer';

EXEC Add_Category 'Cámaras y Fotografía',       'Cámaras, lentes, drones';

EXEC Add_Category 'Software y Productos Digitales','Programas, cursos, apps';

EXEC Add_Category 'Electrodomésticos',          'Refrigeradoras, lavadoras, microondas';

EXEC Add_Category 'Cocina y Comedor',           'Utensilios, vajilla, ollas';

EXEC Add_Category 'Hogar y Jardín',             'Decoración, jardinería, exteriores';

EXEC Add_Category 'Iluminación',                'Lámparas, focos, tiras LED';

EXEC Add_Category 'Herramientas y Ferretería',  'Taladros, llaves, tornillos';

EXEC Add_Category 'Ropa y Accesorios',          'Camisas, pantalones, vestidos';

EXEC Add_Category 'Calzado',                    'Zapatos, tenis, botas, sandalias';

EXEC Add_Category 'Joyería y Relojes',          'Anillos, collares, relojes';

EXEC Add_Category 'Bolsos y Carteras',          'Mochilas, maletas, carteras';

EXEC Add_Category 'Belleza y Cuidado Personal', 'Maquillaje, cremas, cuidado capilar';

EXEC Add_Category 'Perfumes',                   'Perfumes y fragancias';

EXEC Add_Category 'Gafas y Lentes',             'Lentes de sol y de aumento';

EXEC Add_Category 'Salud y Bienestar',          'Equipos médicos, vitaminas';

EXEC Add_Category 'Suplementos y Vitaminas',    'Proteínas, vitaminas, suplementos';

EXEC Add_Category 'Deportes y Aire Libre',      'Camping, pesca, senderismo';

EXEC Add_Category 'Fitness y Musculación',      'Pesas, bandas, ropa fitness';

EXEC Add_Category 'Bicicletas y Accesorios',    'Bicicletas, cascos, accesorios';
```

```
EXEC Add_Category 'Juguetes y Juegos',           'Juguetes educativos, juegos de mesa';
EXEC Add_Category 'Bebés y Niños',               'Pañales, ropa de bebé, cuidado';
EXEC Add_Category 'Mascotas',                    'Alimentos, juguetes, accesorios';
EXEC Add_Category 'Ropa Deportiva',              'Sudaderas, shorts, tops deportivos';
EXEC Add_Category 'Libros, Música y Películas',  'Libros, CDs, DVDs, vinyl';
EXEC Add_Category 'Instrumentos Musicales',      'Guitarras, teclados, baterías';
EXEC Add_Category 'Arte, Manualidades y Costura', 'Pinturas, telas, hilos';
EXEC Add_Category 'Coleccionables y Hobbies',    'Figuras, antigüedades, cartas';
EXEC Add_Category 'Oficina y Papelería',         'Cuadernos, bolígrafos, archivos';
EXEC Add_Category 'Automotriz',                  'Repuestos, aceites, accesorios';
EXEC Add_Category 'Motocicletas y Accesorios',   'Motos, cascos, repuestos';
EXEC Add_Category 'Industria y Ciencia',         'Instrumentos, maquinaria';
EXEC Add_Category 'Alimentos y Bebidas',         'Despensa, snacks, bebidas';
EXEC Add_Category 'Productos de Limpieza',       'Detergentes, desinfectantes';
EXEC Add_Category 'Regalos y Novedades',         'Regalos, decoración, novedades';
EXEC Add_Category 'Jardinería',                  'Plantas, macetas, herramientas de jardín';
GO

-- 2) PRODUCTOS (10)
PRINT '--- Insertando productos ---';
EXEC Add_Product 'TV 40 pulgadas',          100, 200,  'Electrónica';
EXEC Add_Product 'TV 50 pulgadas',          100, 300,  'Electrónica';
EXEC Add_Product 'TV 60 pulgadas',          100, 400,  'Electrónica';
EXEC Add_Product 'TV 80 pulgadas',          100, 1000, 'Electrónica';
EXEC Add_Product 'Lo mejor de Beyoncé',     100, 20,   'Libros, Música y Películas';
EXEC Add_Product 'Lo mejor de Taylor Swift', 100, 20,   'Libros, Música y Películas';
EXEC Add_Product 'Lo mejor de Eminem',      100, 20,   'Libros, Música y Películas';
EXEC Add_Product 'Lo mejor de The Weeknd',  100, 20,   'Libros, Música y Películas';
EXEC Add_Product 'Pelota',                  50, 10.50,'Deportes y Aire Libre';
EXEC Add_Product 'Cocina Quemador',         2, 12000, 'Electrodomésticos';
GO
```

```
-- 3) VENDEDORES

PRINT '--- Insertando vendedores ---';

EXEC add_seller 'Admin General',    'admin@mercadeonica.com',    'admin123',    'ADMIN';

EXEC add_seller 'Franklin Salinas',  'franklin.v@mercadeonica.com','vendedor123', 'VENDEDOR';

EXEC add_seller 'Sofia Guevara',    'sofia.v@mercadeonica.com',   'vendedor123', 'VENDEDOR';

EXEC add_seller 'Harvin Gonzalez',   'harvin.v@mercadeonica.com',  'vendedor123', 'VENDEDOR';

GO


-- 4) CLIENTES

PRINT '--- Insertando clientes ---';

EXEC add_cliente_completo 'Franklin Salinas',   'franklin@gmail.com', 'Rusia',      'MD', 21250, 'cliente123', NULL, 12345,
'VISA',    10, 2026;

EXEC add_cliente_completo 'Sofia Guevara',    'sofia@gmail.com',    'Velezuela',   'DC', 20001, 'cliente123', NULL, 12348, 'VISA',
10, 2026;

EXEC add_cliente_completo 'Harvin Gonzalez',    'harvin@gmail.com',   'Los Angeles', 'CA', 90001, 'cliente123', NULL, 45678,
'Discover', 11, 2026;

EXEC add_cliente_completo 'Rosa Solano',      'rosa@gmail.org',     'Columbia',    'SC', 44250, 'cliente123', NULL, 45679,
'Discover', 10, 2026;

EXEC add_cliente_completo 'Carlos Martinez',    'carlos@gmail.com',   'Corea',      'MD', 21250, 'cliente123', NULL, 12346,
'VISA',    10, 2026;

GO


-- Asigna categoría "Regular" a los clientes de prueba

UPDATE customers

SET categoryid = (SELECT categoryid FROM CustomerCategories WHERE categoryname = 'Regular')

WHERE categoryid IS NULL;

GO


-- 5) PEDIDOS

EXEC place_order_full 1, 2, '2023-05-10 10:30:00', 12345,

    '[{"productid":2,"quantity":2},{"productid":5,"quantity":30}]', 600.00, 'CONFIRMADO';


EXEC place_order_full 5, 2, '2023-05-11 14:15:00', 12346,

    '[{"productid":4,"quantity":1}]', 1000.00, 'CONFIRMADO';
```

```
EXEC place_order_full 3, 3, '2023-06-20 20:45:00', 45678,

'[{"productid":6,"quantity":10},{"productid":7,"quantity":10},{"productid":8,"quantity":10},{"productid":5,"quantity":25}]',

    11000.00, 'CONFIRMADO';


-- Pedido en efectivo (sin tarjeta)

EXEC place_order_full 1, 2, '2023-05-12 15:00:00', 0,

    '[{"productid":1,"quantity":2}]', 400.00, 'CONFIRMADO';

GO


-- 6) MOVIMIENTOS DE INVENTARIO

PRINT '--- Insertando movimientos de inventario ---';

EXEC register_inventory_movement 1, 1, 'ENTRADA', 50, 'Reposición de stock TV 40 pulgadas';

EXEC register_inventory_movement 6, 1, 'ENTRADA', 20, 'Reposición BestOfTaylorSwift';

GO


-- 7) RESEÑAS

PRINT '--- Insertando reseñas ---';

EXEC add_review 'TV 50 pulgadas',       'franklin@gmail.com', 2, 'No es un buen televisor. Podría ser mejor';

EXEC add_review 'TV 50 pulgadas',       'carlos@gmail.com',   1, 'No lo compren';

EXEC add_review 'TV 80 pulgadas',       'harvin@gmail.com',   5, 'Excelente. Este es el mejor televisor';

EXEC add_review 'TV 80 pulgadas',       'martha@gmail.com',   5, 'Excelente. El mejor de todos';

EXEC add_review 'Lo mejor de Beyoncé',   'martha@gmail.com',   5, 'Excelente. El mejor de todos';

EXEC add_review 'Lo mejor de Beyoncé',   'eduardo@gmail.com',  4, 'Lo disfruté';

GO



/* ============================================================

  PASO 6: VERIFICACIÓN FINAL

  ============================================================ */
```

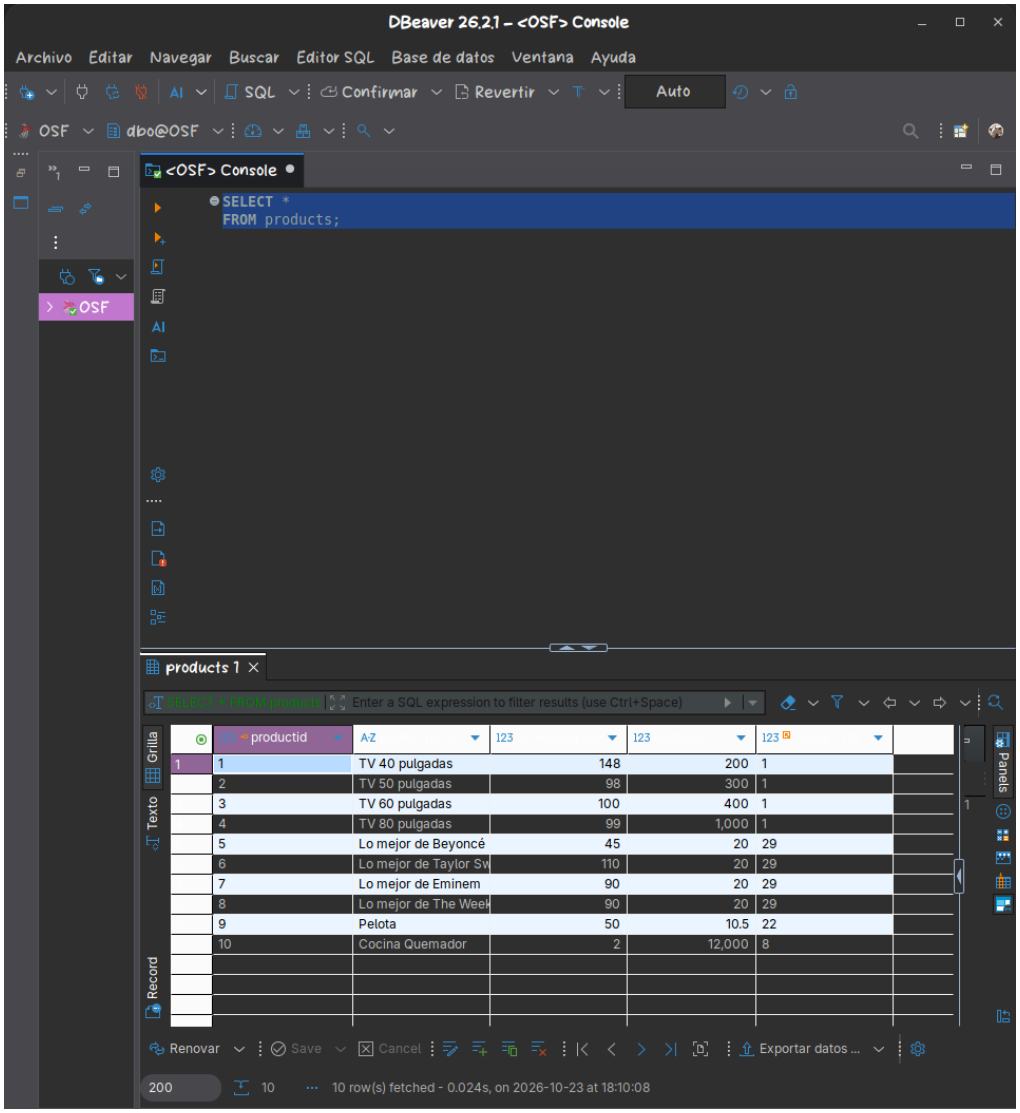
```
PRINT '';
PRINT '==========================================';
PRINT ' RESUMEN DE DATOS CARGADOS';
PRINT '==========================================';

SELECT 'CUSTOMERS'          AS tabla, COUNT(*) AS filas FROM customers
UNION ALL SELECT 'SELLERS',           COUNT(*) FROM sellers
UNION ALL SELECT 'CATEGORIES',        COUNT(*) FROM ProductCategories
UNION ALL SELECT 'PRODUCTS',          COUNT(*) FROM products
UNION ALL SELECT 'ORDERS',            COUNT(*) FROM orders
UNION ALL SELECT 'ORDER_DETAILS',     COUNT(*) FROM order_details
UNION ALL SELECT 'INVENTORY_MOVEMENTS', COUNT(*) FROM inventory_movements
UNION ALL SELECT 'CREDITCARDS',       COUNT(*) FROM creditcards
UNION ALL SELECT 'INVOICES',          COUNT(*) FROM invoices
UNION ALL SELECT 'REVIEWS',           COUNT(*) FROM reviews
UNION ALL SELECT 'RECOMMENDATIONS',   COUNT(*) FROM recommendations
UNION ALL SELECT 'AUDIT_LOG',         COUNT(*) FROM audit_log;
GO

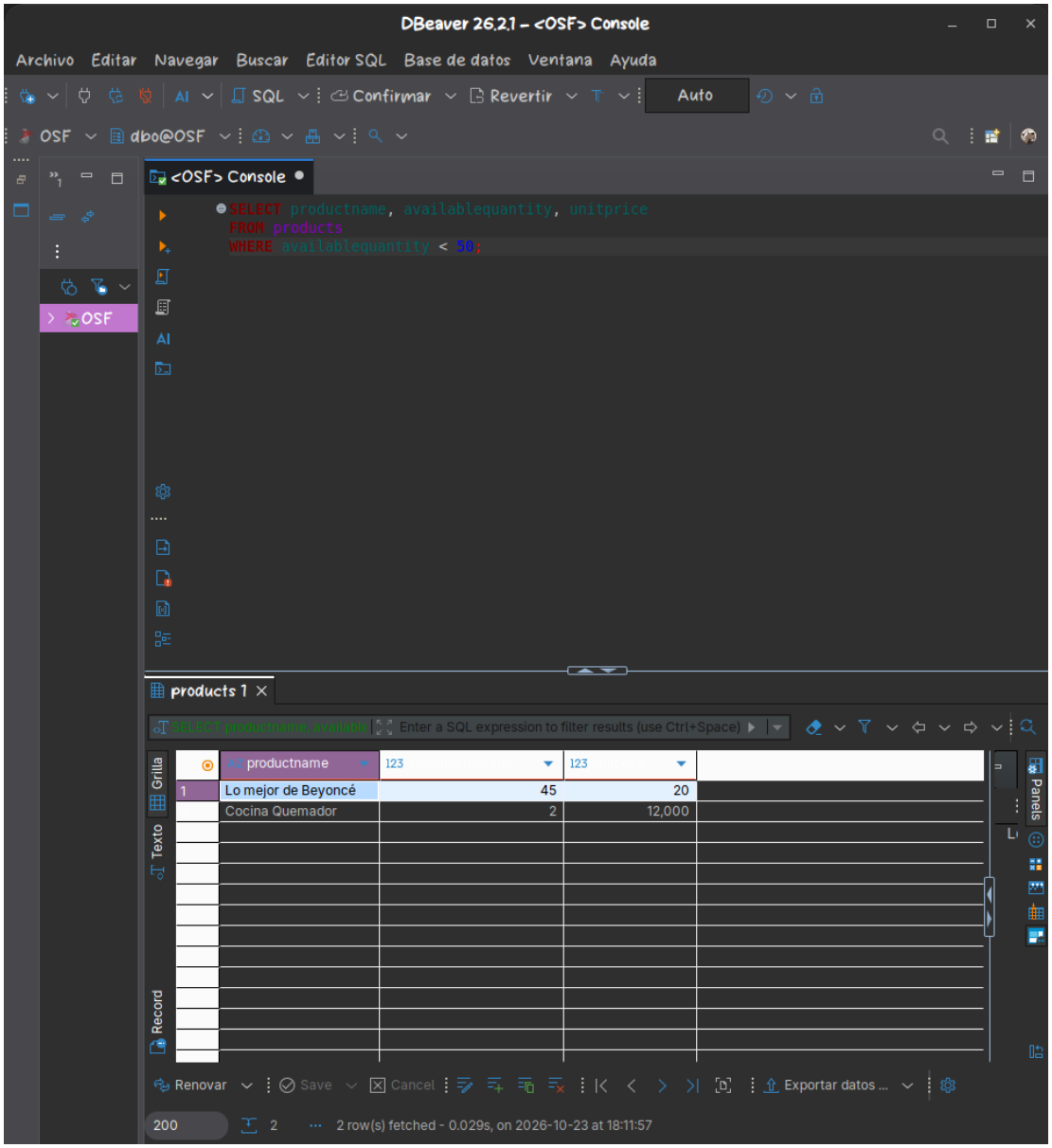
PRINT '';
PRINT '==========================================';
PRINT ' ☑ BASE DE DATOS OSF v6.0 LISTA';
PRINT '==========================================';
GO
```

8. Consultas SQL y Pruebas de DBearer

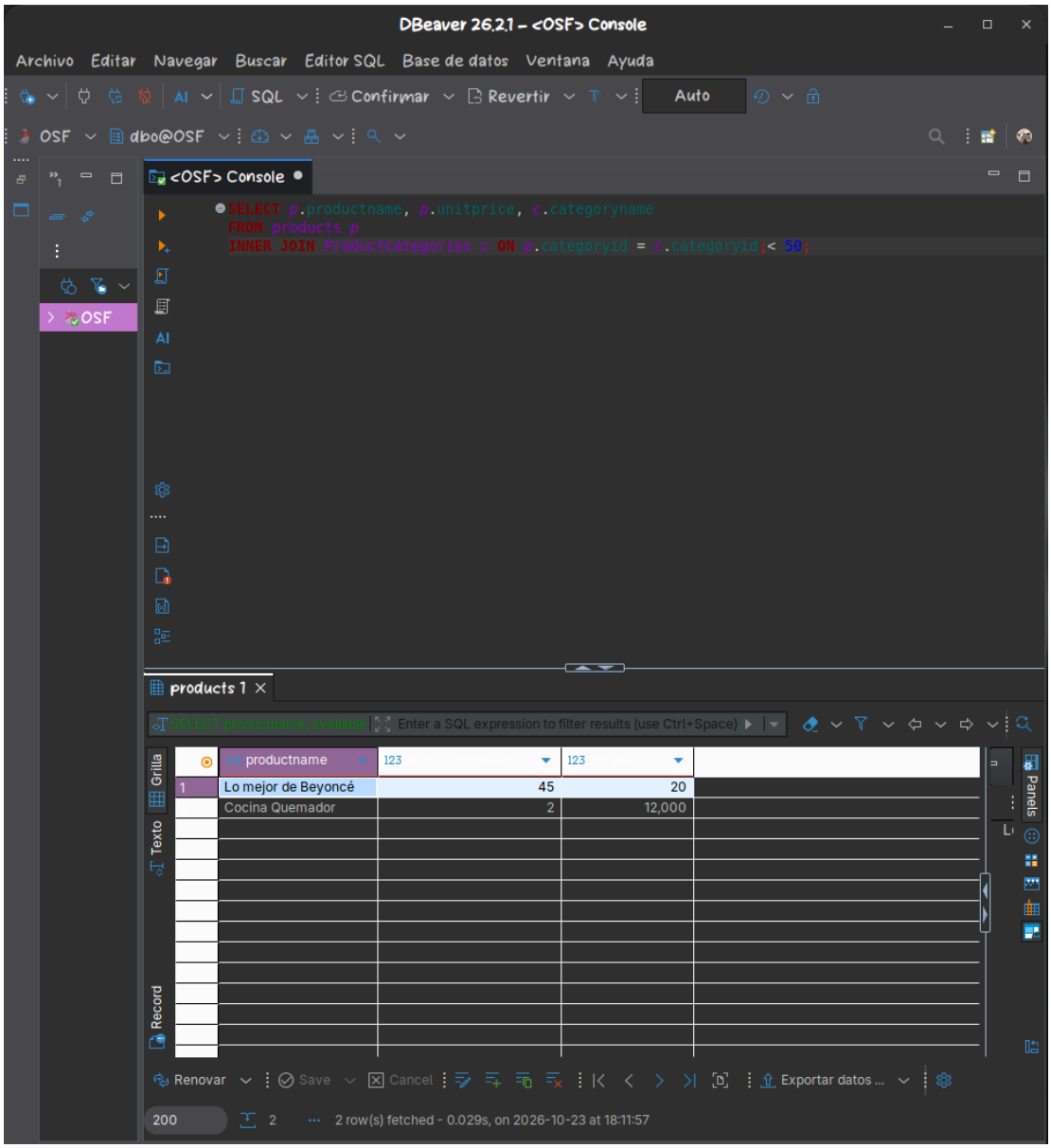
Visualización de inventario



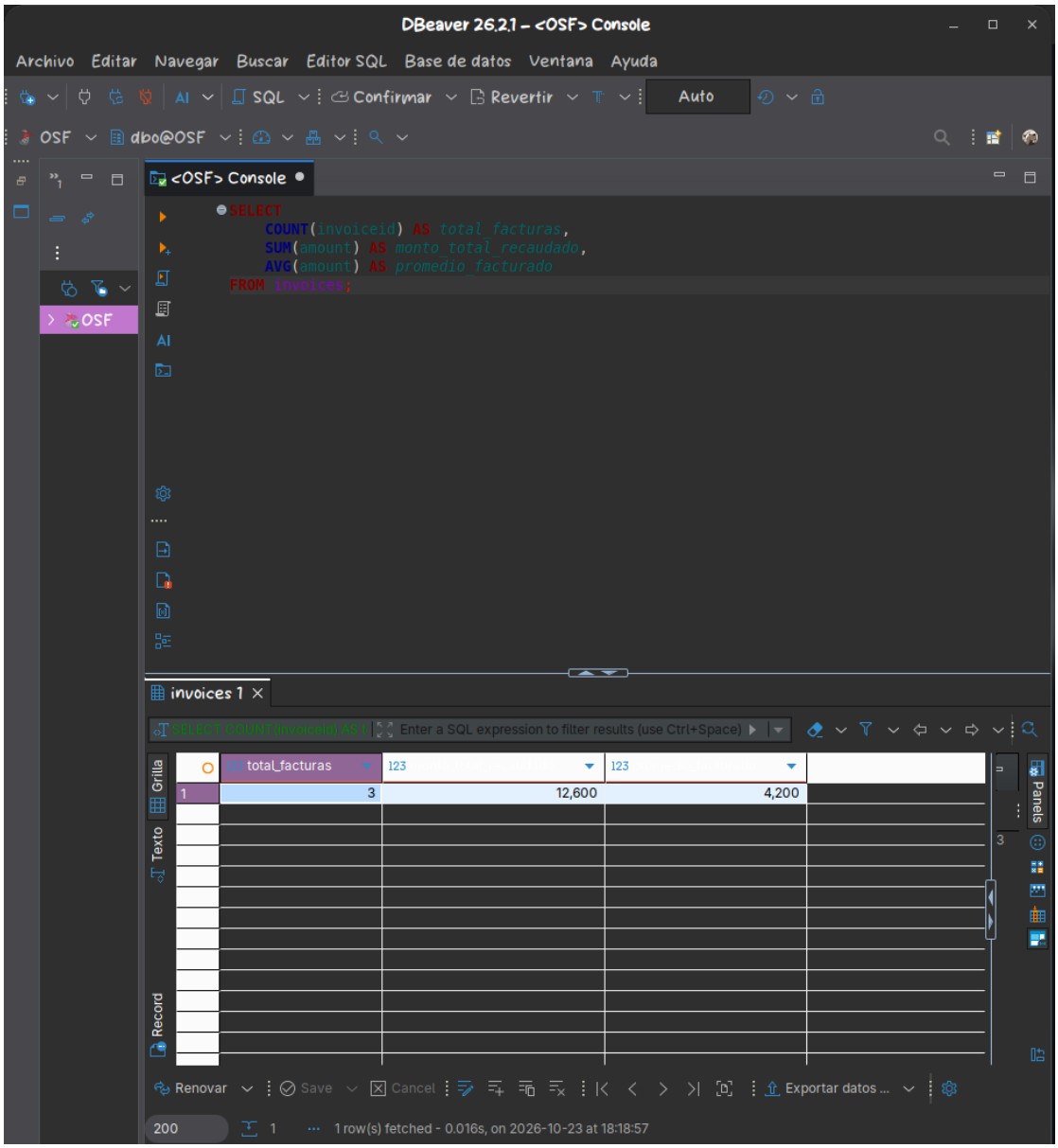
Filtrado por cantidad disponible



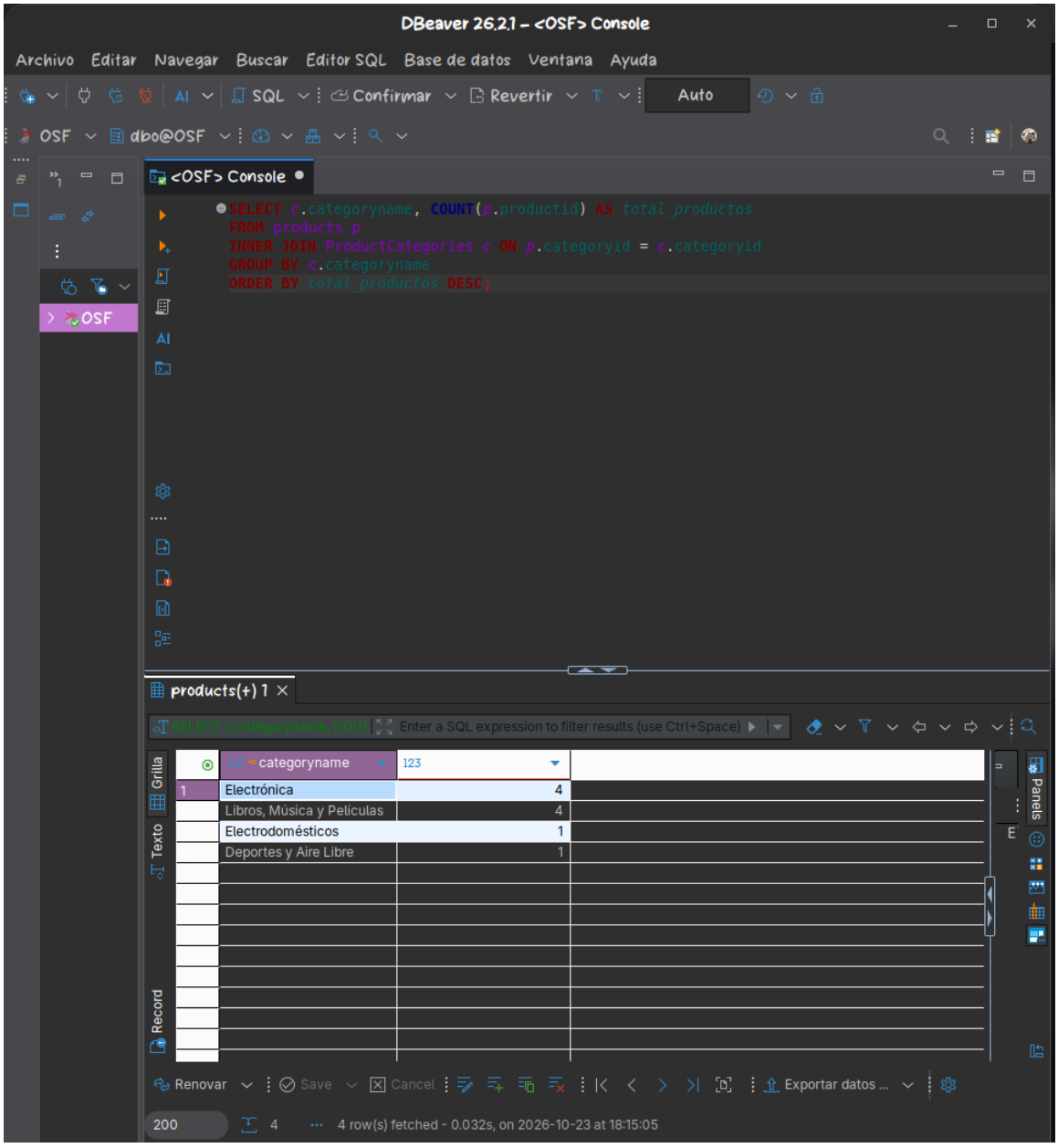
JOIN entre Products y ProductCategories



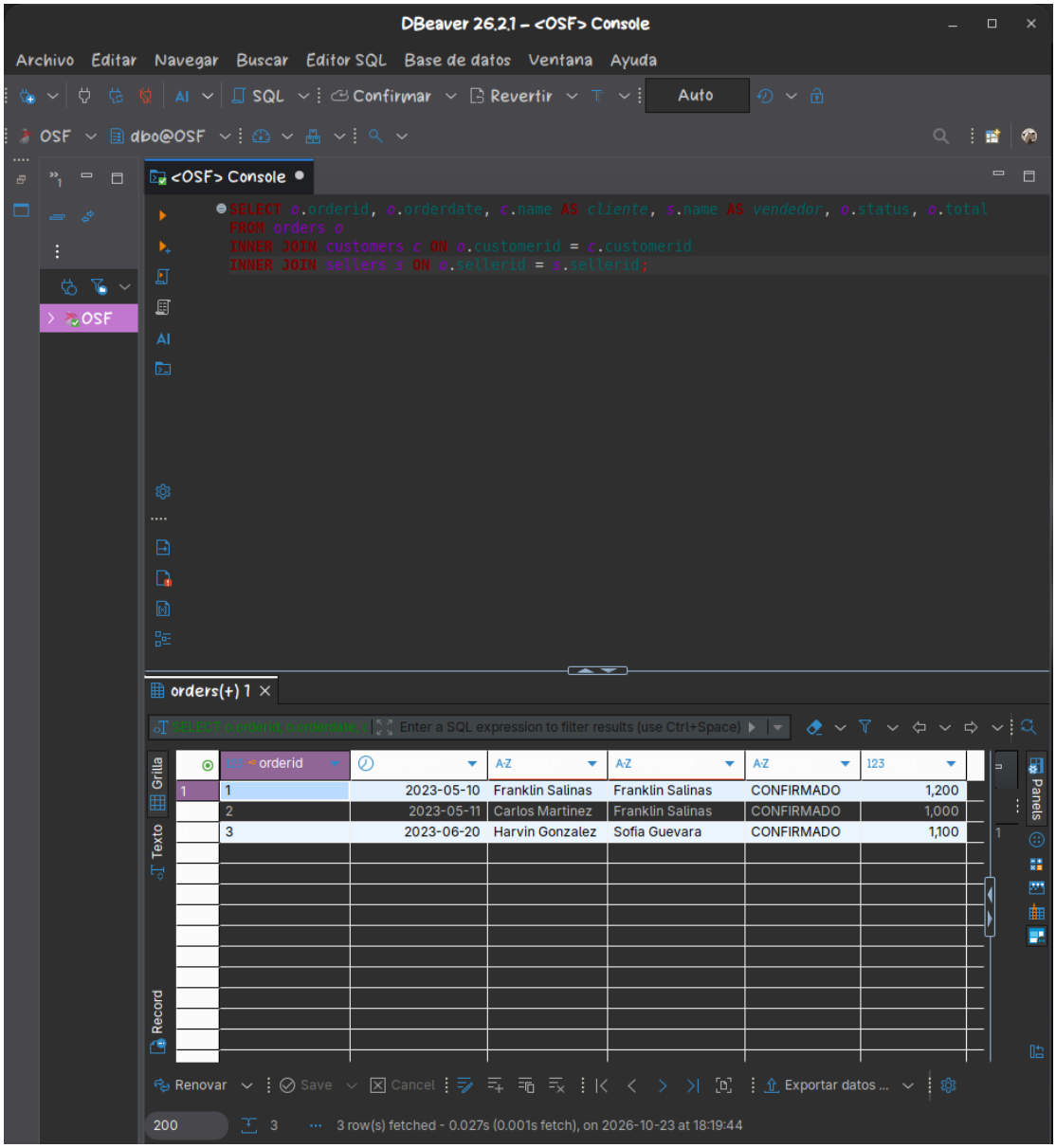
Cálculo de totales y promedios de facturas



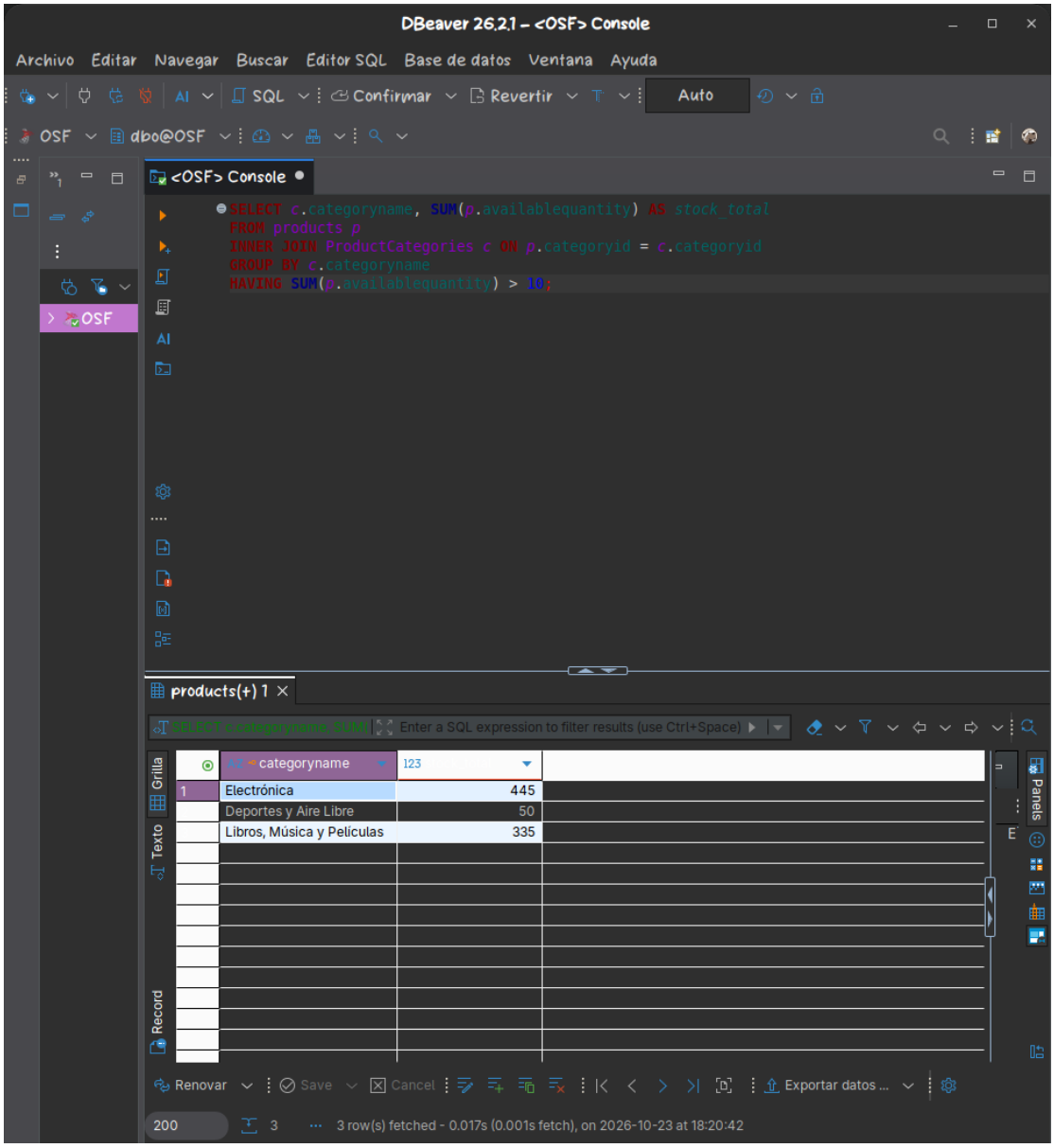
Conteo de productos agrupados por categoría



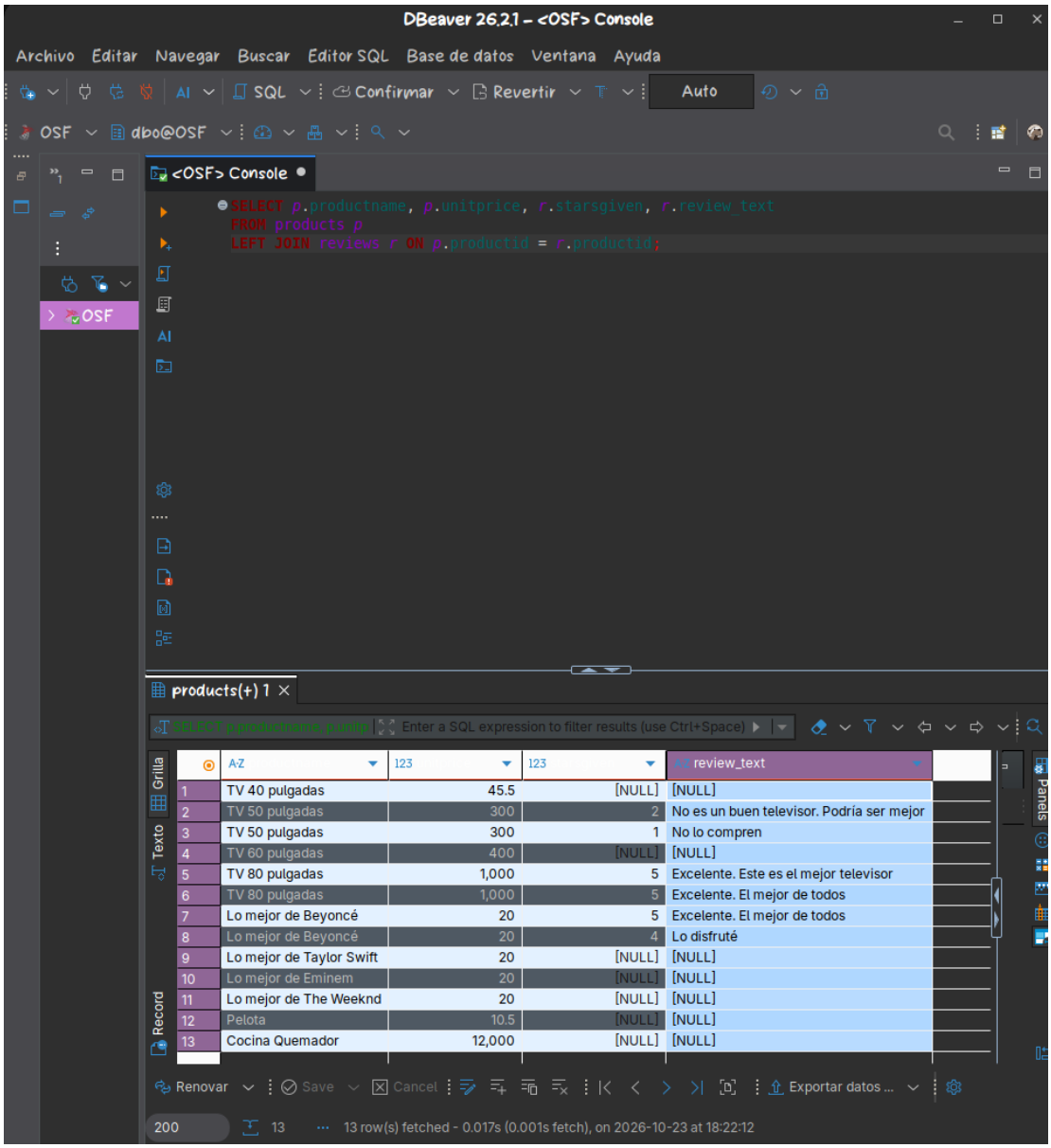
Selección con cruce de tablas (Cliente y vendedor)



Stock total por categoría (Filtrado)

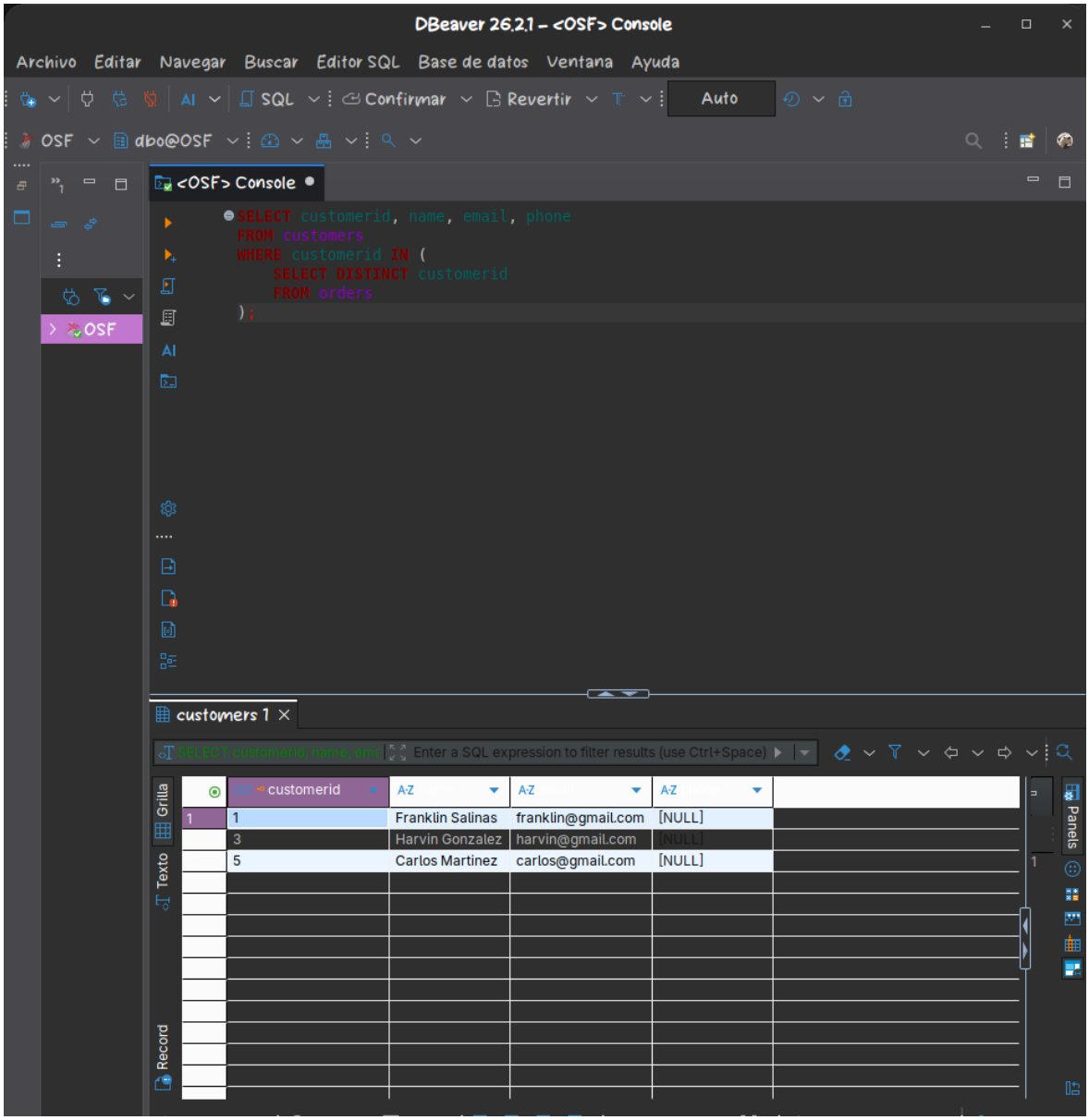


Auditoria de Feedback de Productos

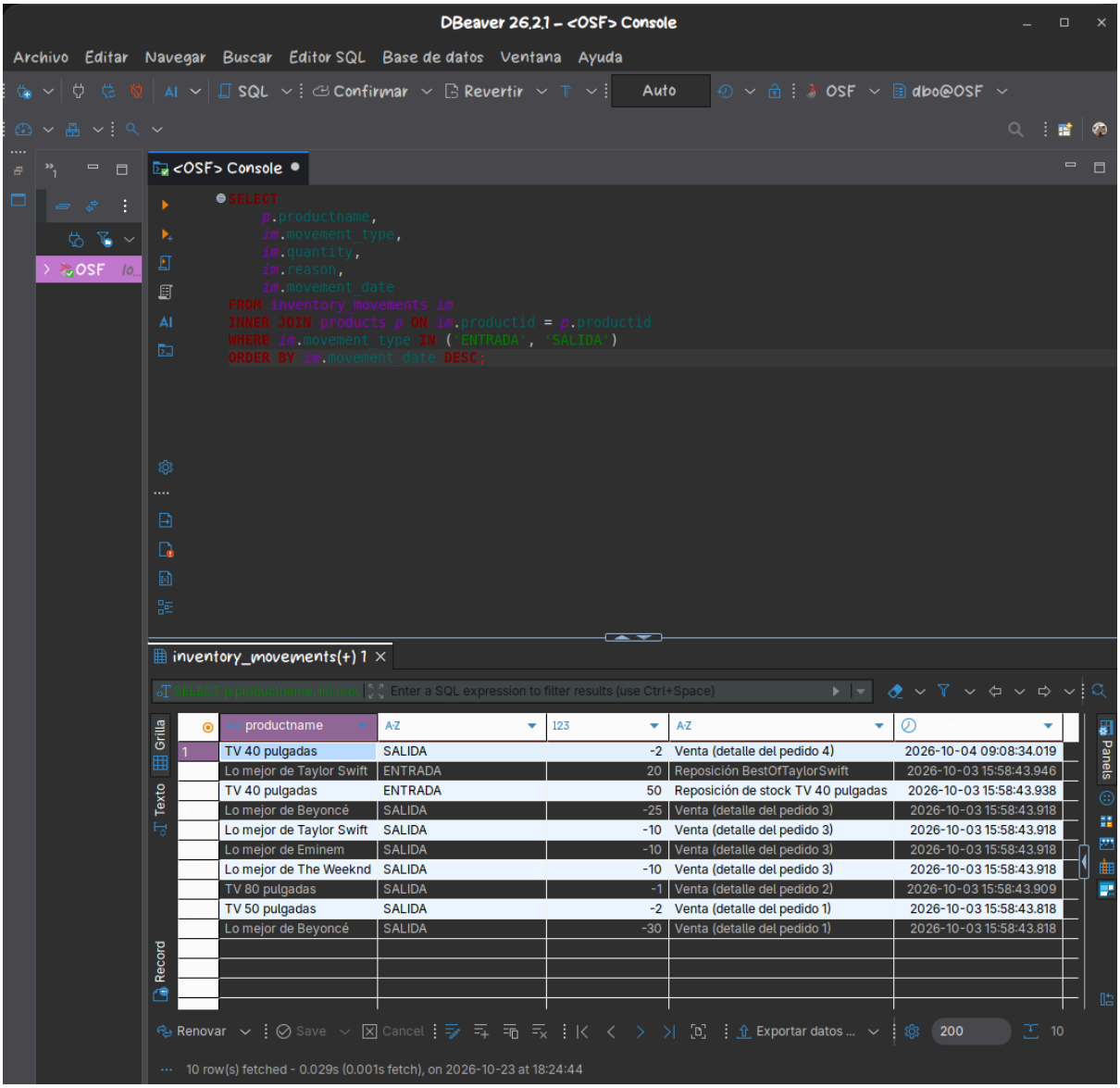


	A-Z	123	123	review_text
1	TV 40 pulgadas	45.5	[NULL]	[NULL]
2	TV 50 pulgadas	300	2	No es un buen televisor. Podría ser mejor
3	TV 50 pulgadas	300	1	No lo compren
4	TV 60 pulgadas	400	[NULL]	[NULL]
5	TV 80 pulgadas	1,000	5	Excelente. Este es el mejor televisor
6	TV 80 pulgadas	1,000	5	Excelente. El mejor de todos
7	Lo mejor de Beyoncé	20	5	Excelente. El mejor de todos
8	Lo mejor de Beyoncé	20	4	Lo disfruté
9	Lo mejor de Taylor Swift	20	[NULL]	[NULL]
10	Lo mejor de Eminem	20	[NULL]	[NULL]
11	Lo mejor de The Weeknd	20	[NULL]	[NULL]
12	Pelota	10.5	[NULL]	[NULL]
13	Cocina Quemador	12,000	[NULL]	[NULL]

Cliente activos (Con compras)



Historial de movimientos de inventario



9. Conclusión

La propuesta MercadeoNica establece un alcance inicial para integrar la gestión de productos, clientes, pedidos, ventas e inventario en un sistema web. Su aplicación a un comercio real permitirá relacionar los contenidos de la asignatura con necesidades concretas y evaluar la utilidad de las funciones desarrolladas.

La planificación evidencia que la base de datos, las estructuras dinámicas y la programación orientada a objetos cumplen funciones complementarias. La primera permitirá conservar y relacionar registros; las estructuras dinámicas facilitarán el manejo temporal de elementos; y la orientación a objetos contribuirá a organizar el código y las responsabilidades del sistema.

También se identifica que el control de inventario requiere una relación consistente con las ventas. Por ello, será necesario validar las cantidades disponibles y coordinar las operaciones que modifican existencias, especialmente cuando varios usuarios soliciten un mismo producto.

9.1. Lecciones aprendidas de la planificación

La principal lección es que el diseño debe comenzar por comprender los procesos del comercio. Definir pantallas sin conocer cómo se registran las ventas o cómo se controlan las existencias podría producir un sistema que no atienda las necesidades reales.

Otra lección es la importancia de delimitar una primera versión alcanzable. La prioridad será completar y comprobar las funciones esenciales antes de incorporar pagos en línea, servicios de entrega u otras características.

Finalmente, se reconoce que el cumplimiento de los objetivos deberá demostrarse mediante pruebas y evidencias. Hasta ejecutar esas verificaciones, no corresponde afirmar que el sistema redujo errores, aumentó ventas o mejoró los tiempos de atención.

10. Recomendaciones

Confirmar los requisitos con el comercio participante, documentar sus procedimientos, dificultades y prioridades antes de comenzar la implementación.

Mantener una arquitectura organizada separar la presentación, la lógica del negocio y el acceso a datos para facilitar el mantenimiento y las modificaciones.

Definir las reglas de inventario establecer cuándo se descuenta una existencia, cómo se atienden las cancelaciones y cómo se registran las devoluciones.

Controlar las ventas simultáneas comprobar las existencias dentro de la misma transacción que confirma la venta, evitando que dos pedidos consuman indebidamente las últimas unidades.

Proteger el acceso a la información incorporar autenticación, almacenamiento seguro de contraseñas y permisos que impidan que un vendedor modifique datos de otro.

Realizar respaldos y comprobar su recuperación definir una frecuencia de copia acorde con el uso del sistema y verificar que la información pueda restaurarse.

Adaptar la interfaz a dispositivos móviles utilizar formularios claros, navegación sencilla y mensajes comprensibles para los clientes y vendedores.

Comprobar casos de error incluir pruebas de cantidades inválidas, productos sin existencias, pedidos cancelados y fallos durante el registro de una venta.

Documentar las estructuras dinámicas utilizadas explicar sus operaciones, utilidad y comportamiento, y confirmar con el docente si deben implementarse manualmente mediante nodos o si se permiten colecciones del lenguaje.

Planificar el crecimiento con base en resultados incorporar más vendedores, filtros, notificaciones o pagos electrónicos después de verificar la estabilidad de la primera versión y las necesidades de sus usuarios.

11. Bibliografía

Centeno Ríos, Juan Ramón y Téllez Mendoza, Robin Alexander (2023) ***Sistema web de gestión de ventas e inventario para el colectivo de MiPymes Corals*****. Universidad Nacional de Ingeniería UNI, Managua, Nicaragua. https://ribuni.uni.edu.ni/4896/**

García Poveda, Renner Jesús y Porras Herrera, Melvin Neville (2022) ***Desarrollo de sistema web para la gestión de los procesos de compraventa, facturación y control de inventario para la tienda Winnie's Menorquinas*****. Universidad Nacional de Ingeniería. https://ribuni.uni.edu.ni/4649/**

Microsoft. (s.f.-a). ***Primary and Foreign Key Constraints*****. SQL Server Documentation. Recuperado de https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints**

Microsoft. (s.f.-b). ***Transactions (Transact-SQL)*****. SQL Server Documentation. Recuperado de https://learn.microsoft.com/en-us/sql/t-sql/language-elements/transactions-transact-sql**

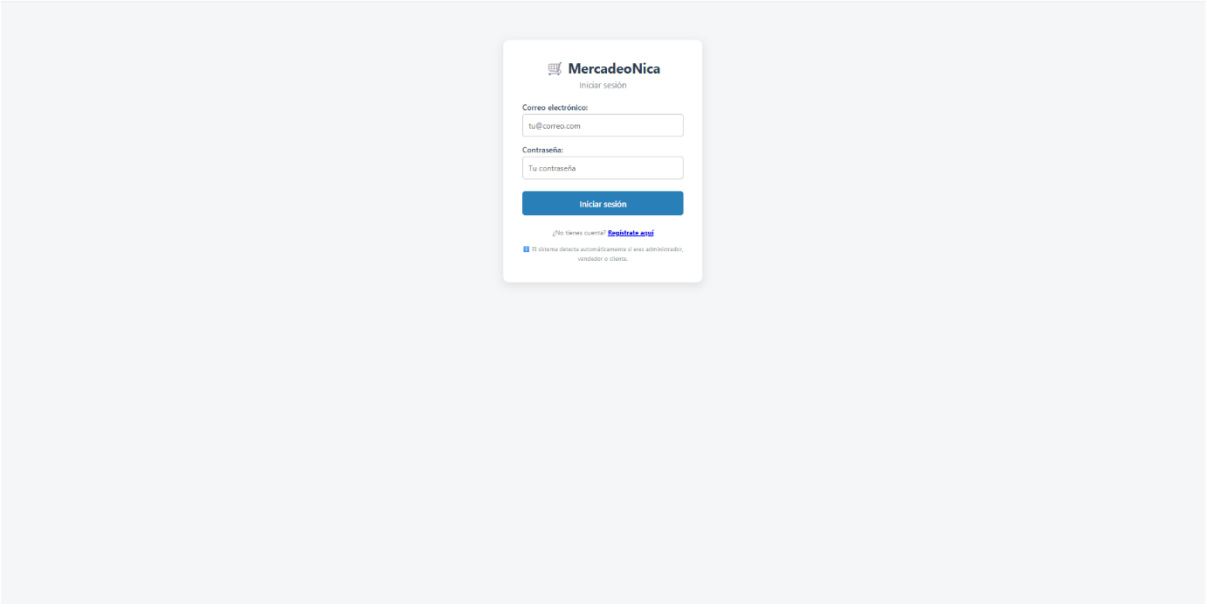
Oracle. (s.f.). ***LinkedList.*** **Java Platform, Standard Edition 21 API Specification. Recuperado de https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/LinkedList.html**

Oracle. (s.f.). ***Lesson: Object-oriented programming concepts*****. The Java Tutorials. Recuperado de https://docs.oracle.com/javase/tutorial/java/concepts/**

Rivas Obregón, Bryan José y Aranda Barrera, José Martin (2024) ***Desarrollo de Sistema Web para la gestión de ventas, compras e inventario de la librería Mayisun*****. Universidad Nacional de Ingeniería. https://ribuni.uni.edu.ni/6428/**

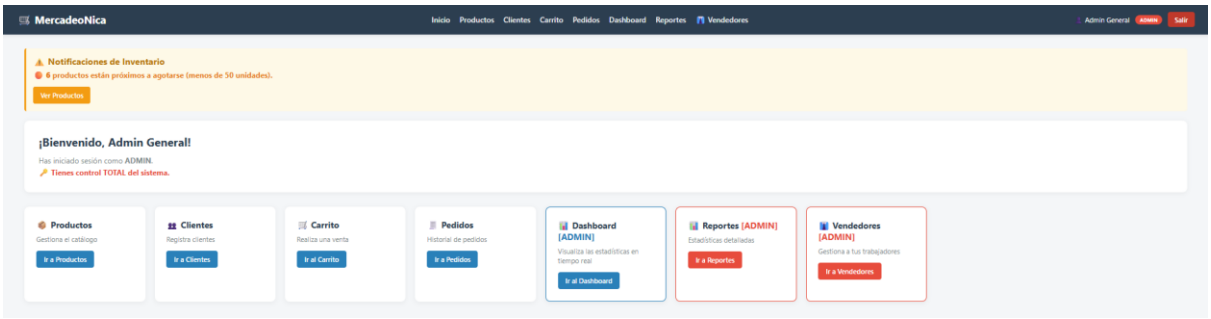
12. Anexos

Login del Administrador, Vendedor y Cliente

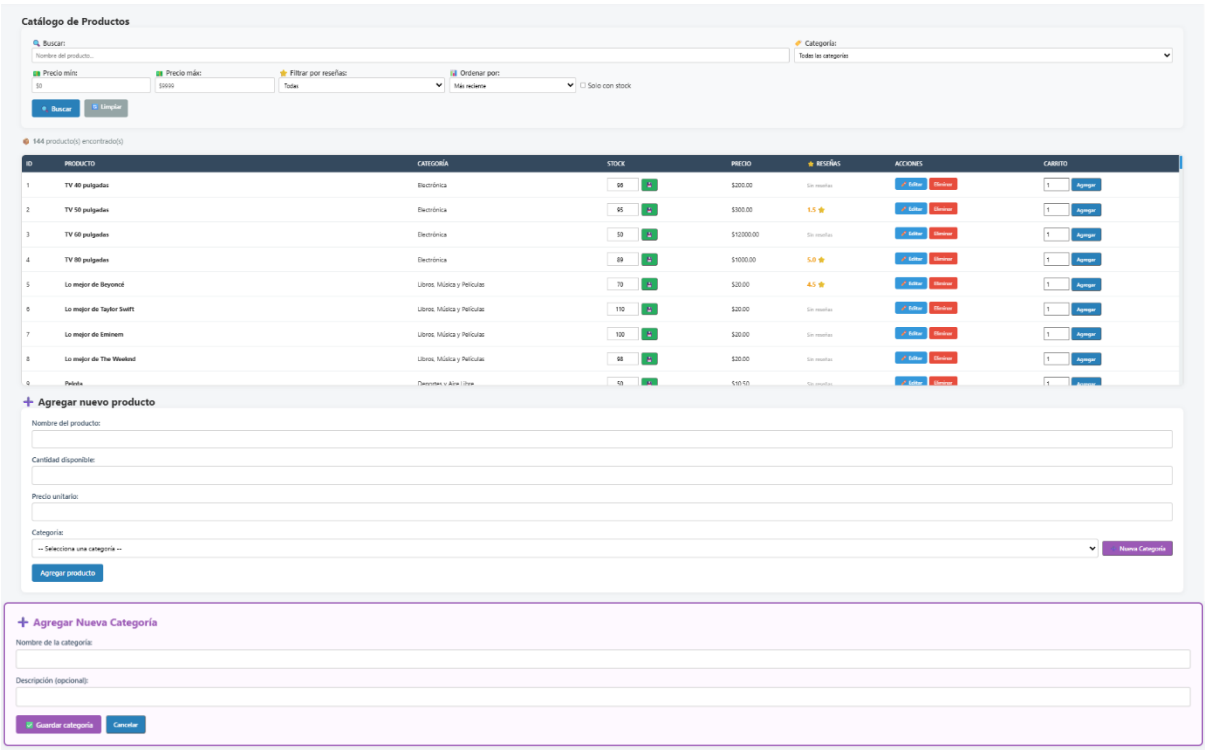


A nivel de Administrador

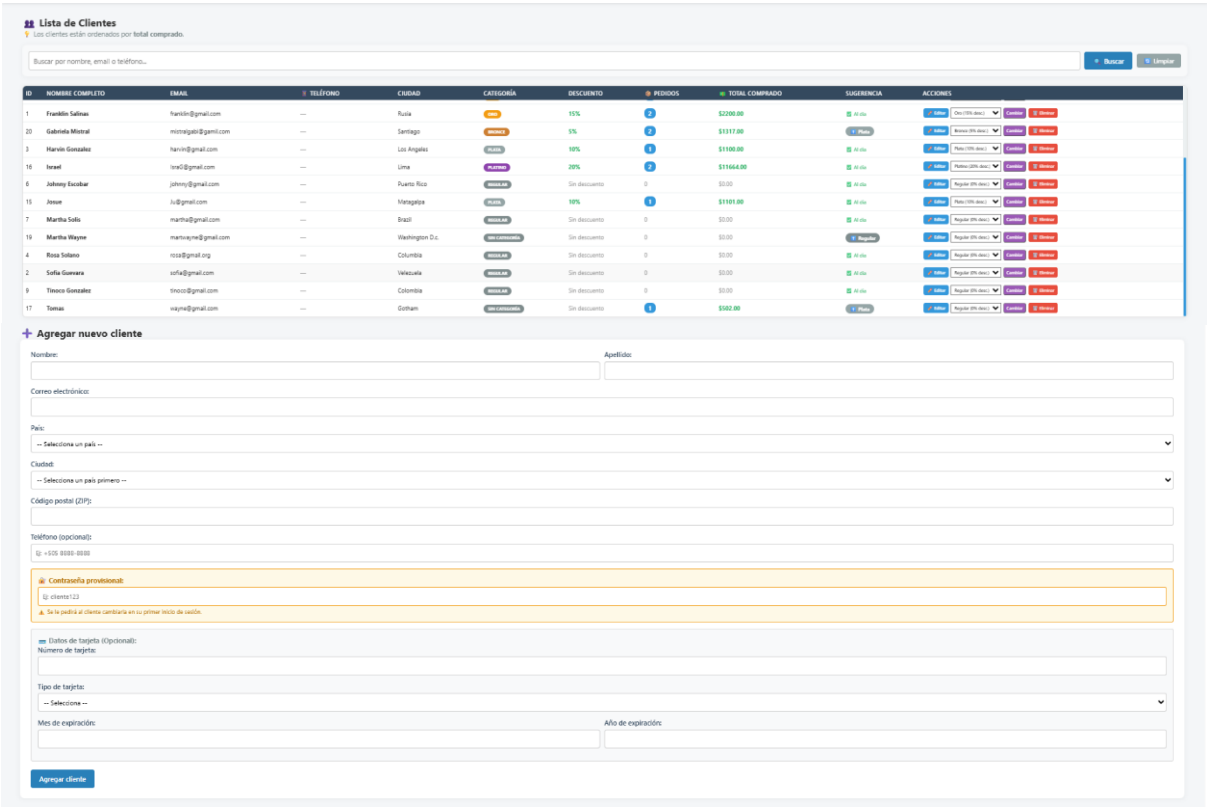
Parnel de control



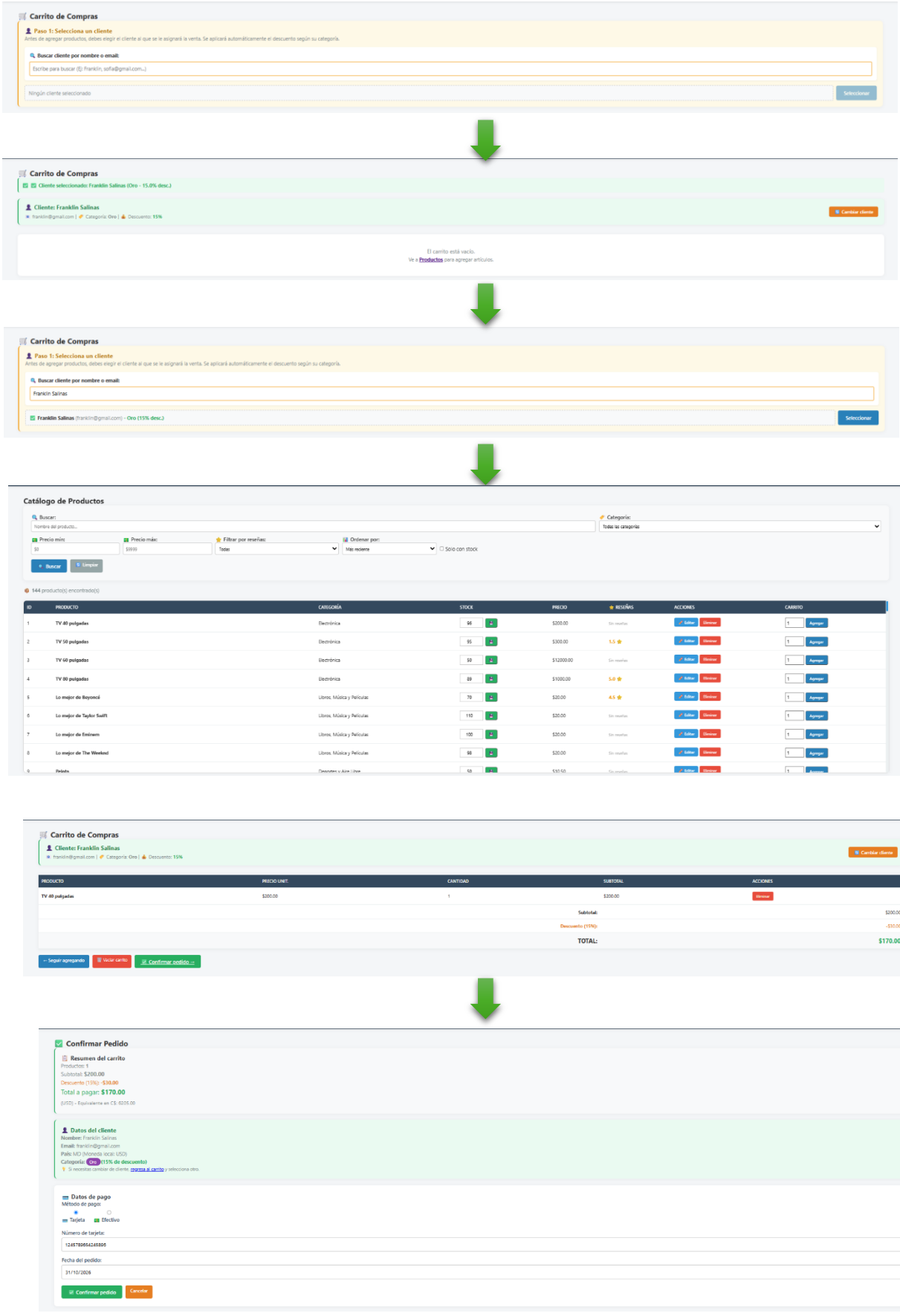
Visualización de Productos



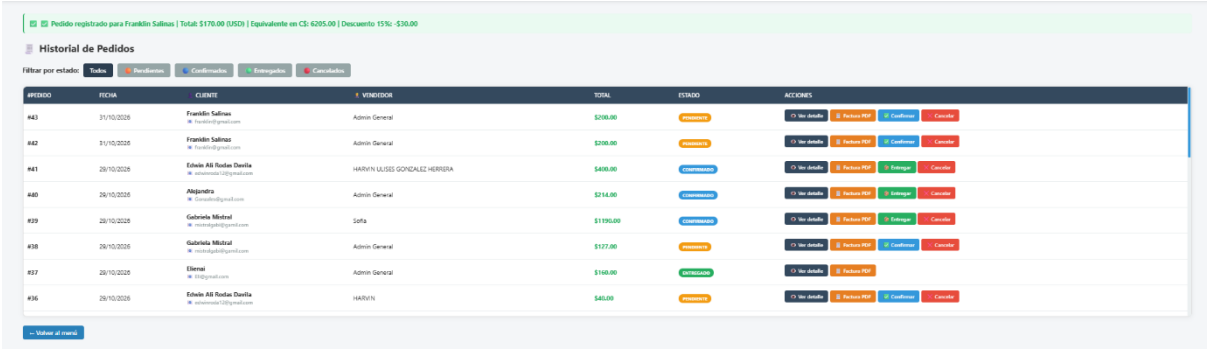
Lista de clientes



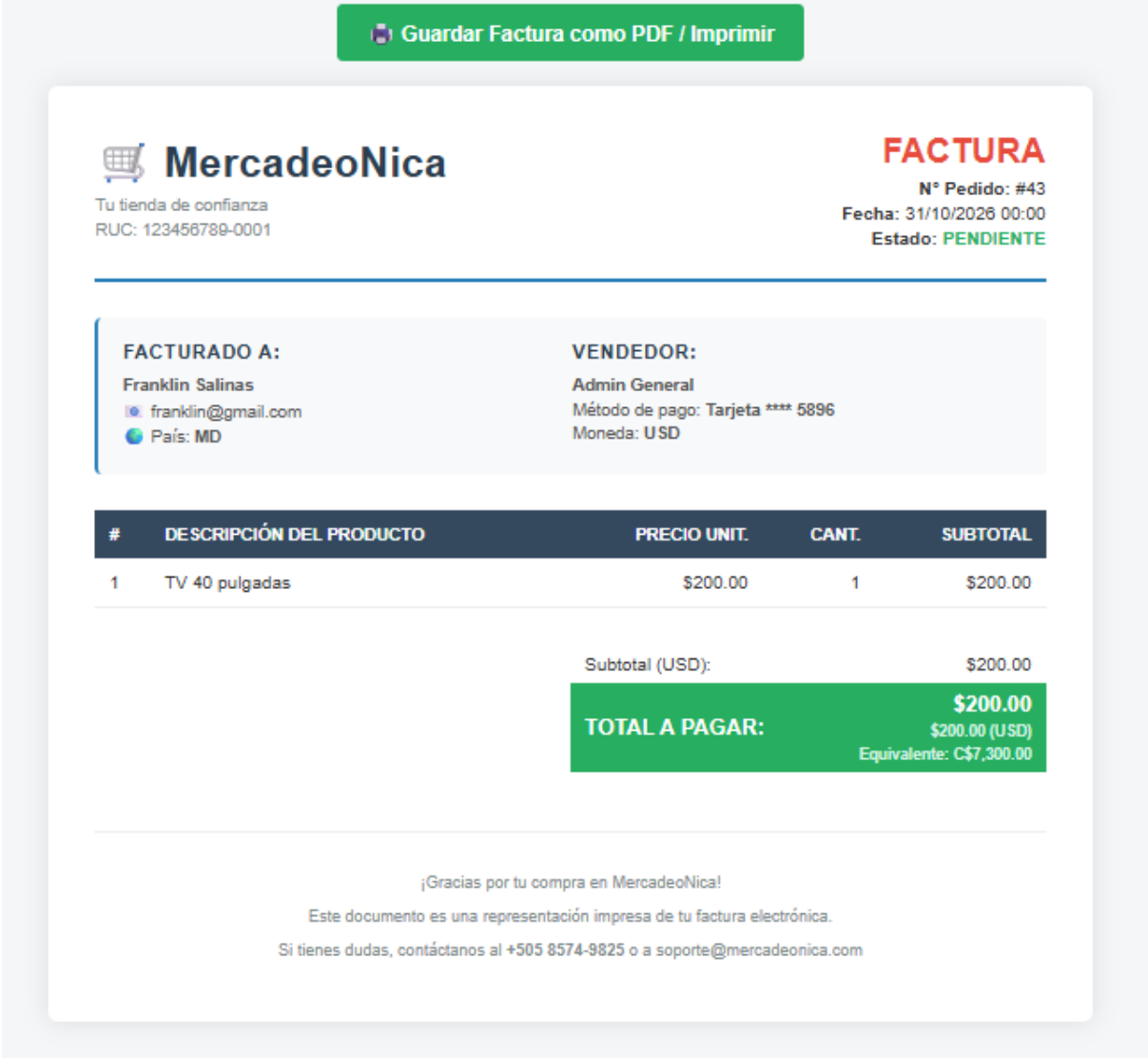
Carrito de compras



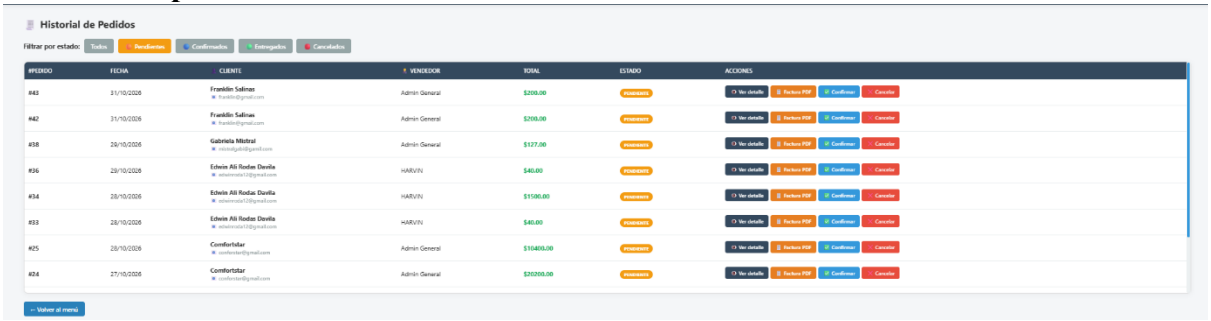
Historial de pedido que realiza el administrador al cliente en general



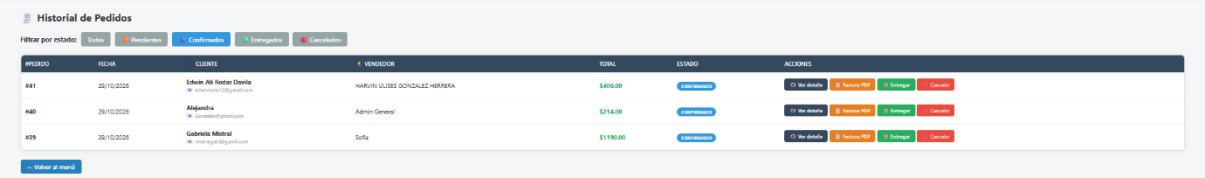
Factura que se realiza a la hora de la compra parta el cliente



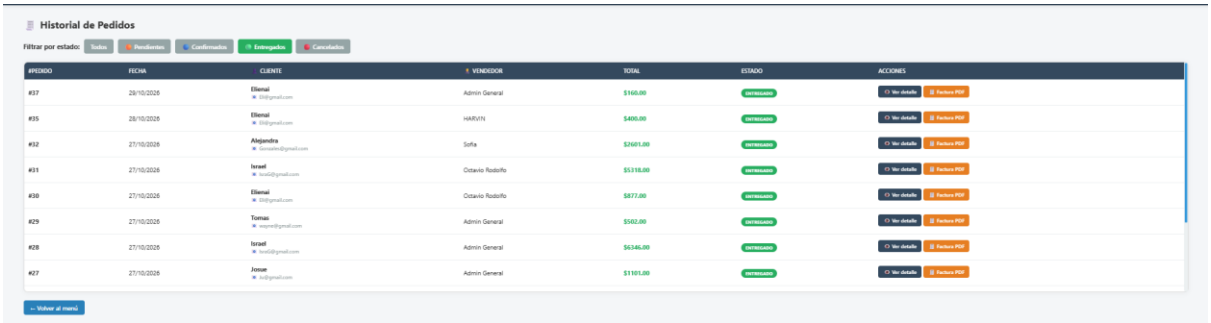
Historial de pedido “Pendientes”



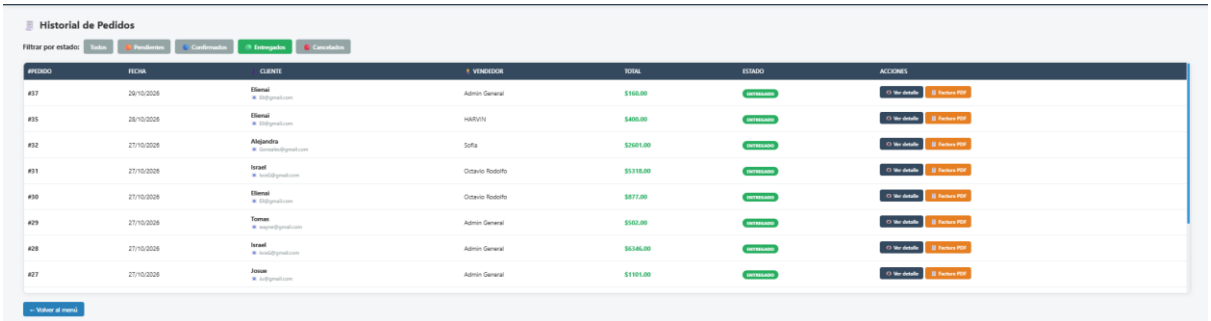
Historial de pedido “Confirmados”



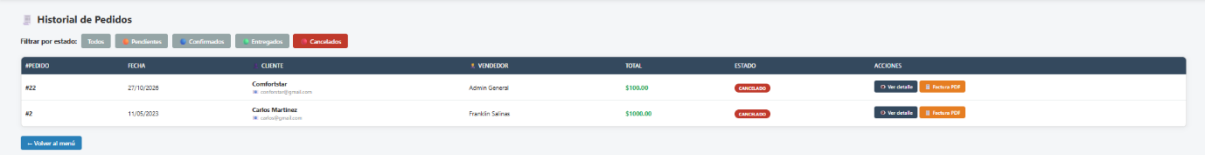
Historial de pedido “Confirmado”



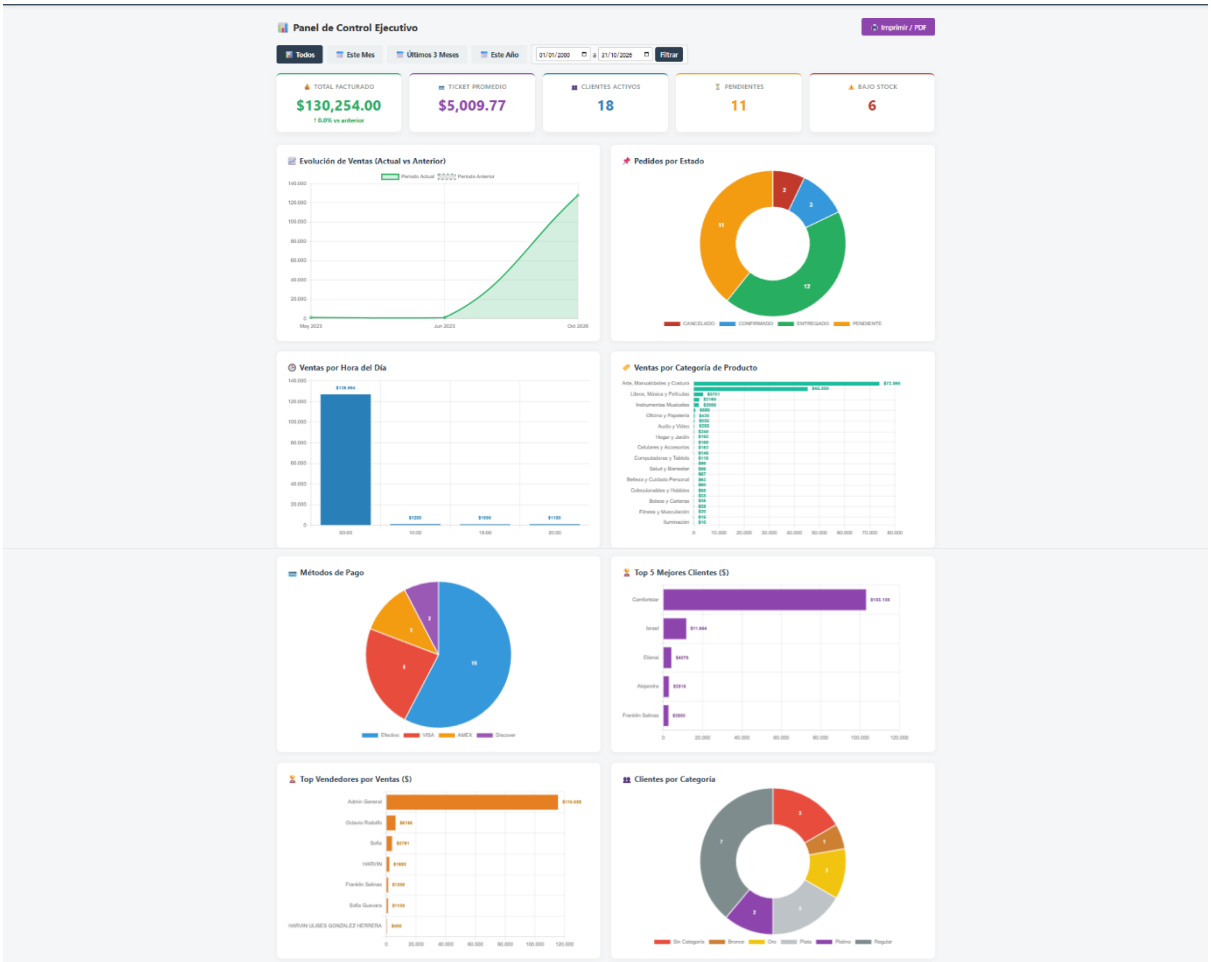
Historial de pedido “Entregados”



Historial de pedido “Cancelado”

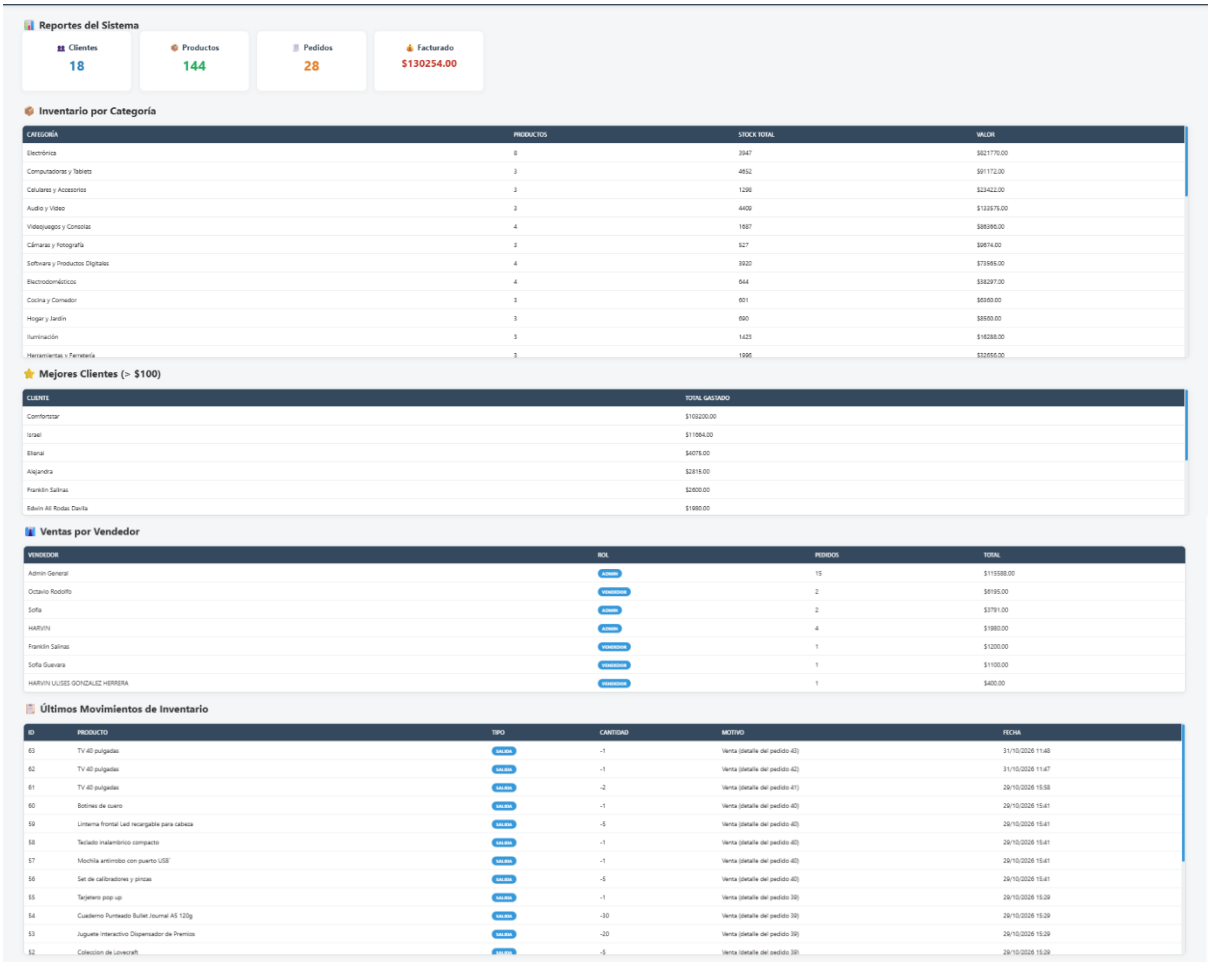


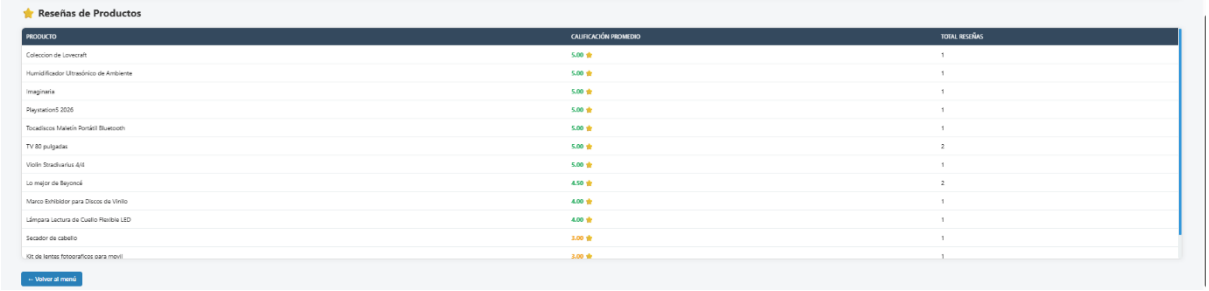
Visualización del Dashboard



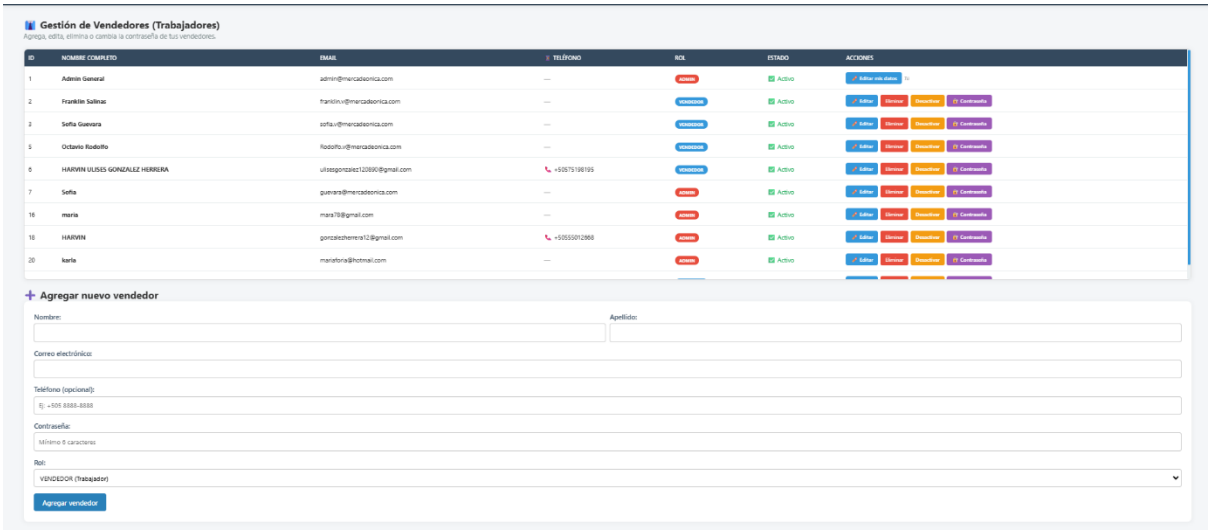


Reporte del sistema



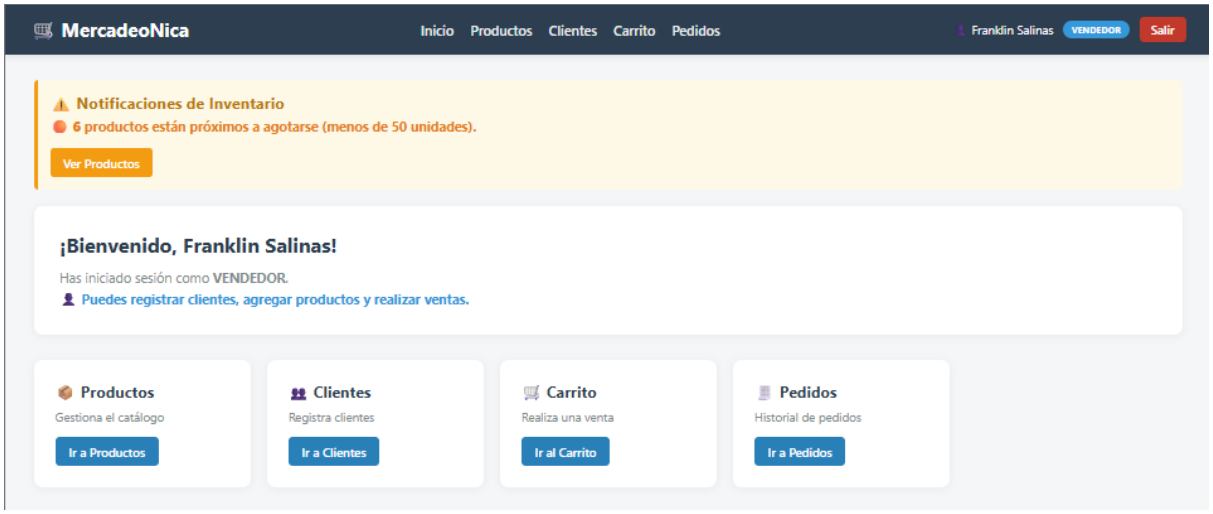


Visualización de vendedores

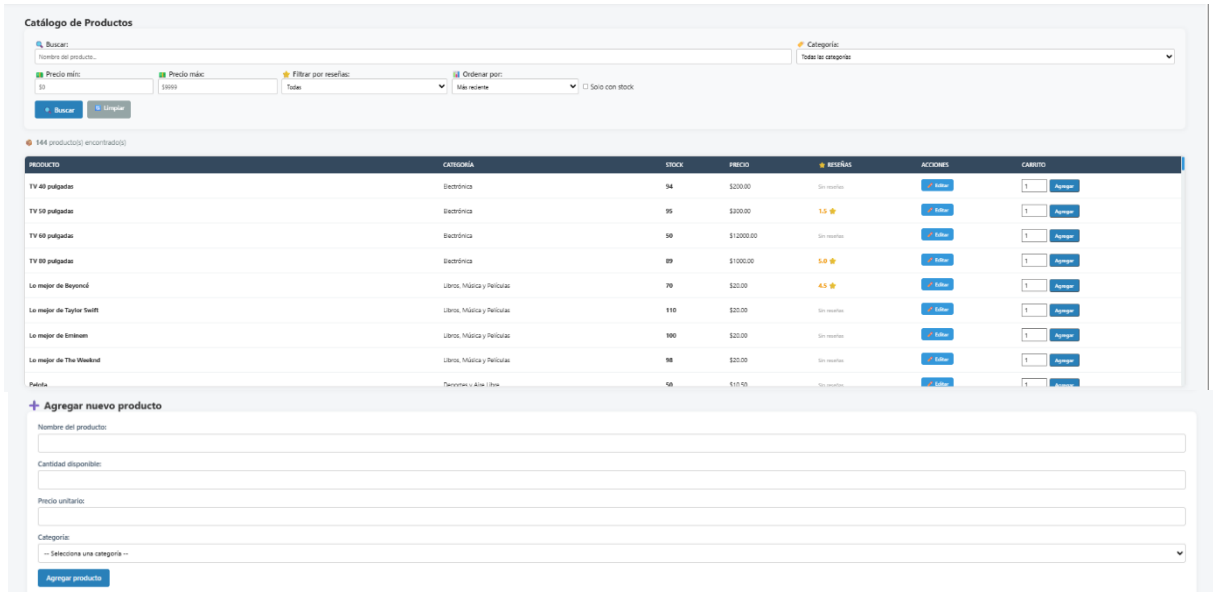


A nivel de Vendedor

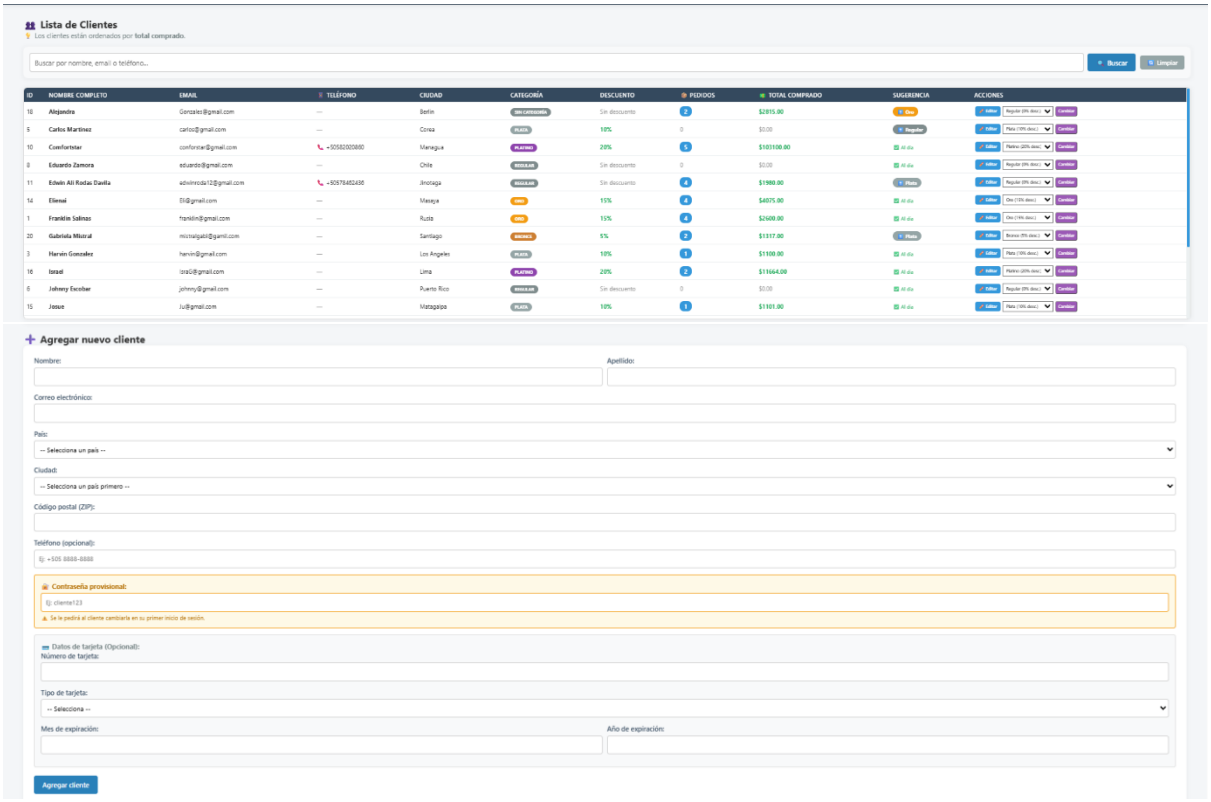
Visualización del panel de control



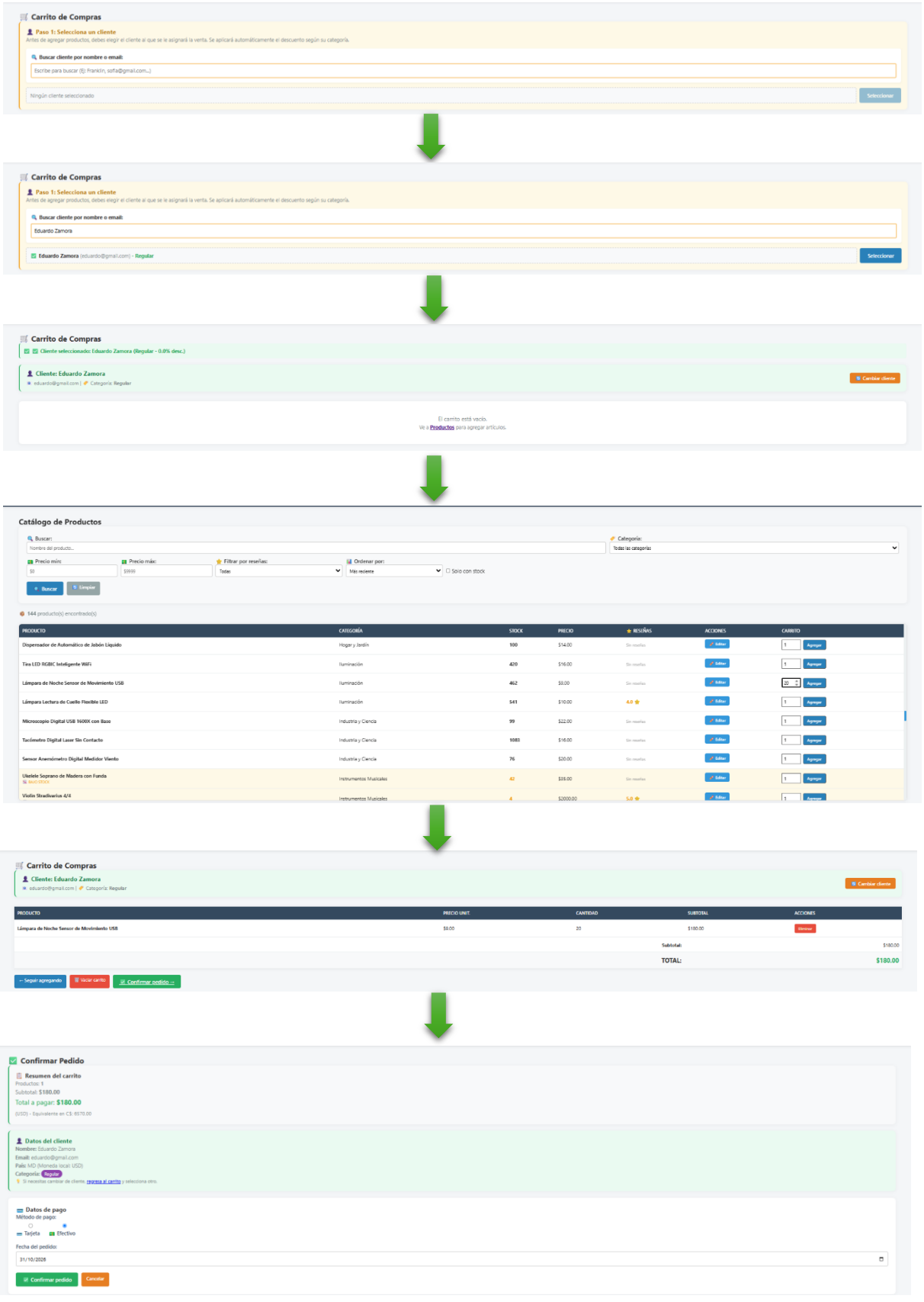
Visualización de los productos



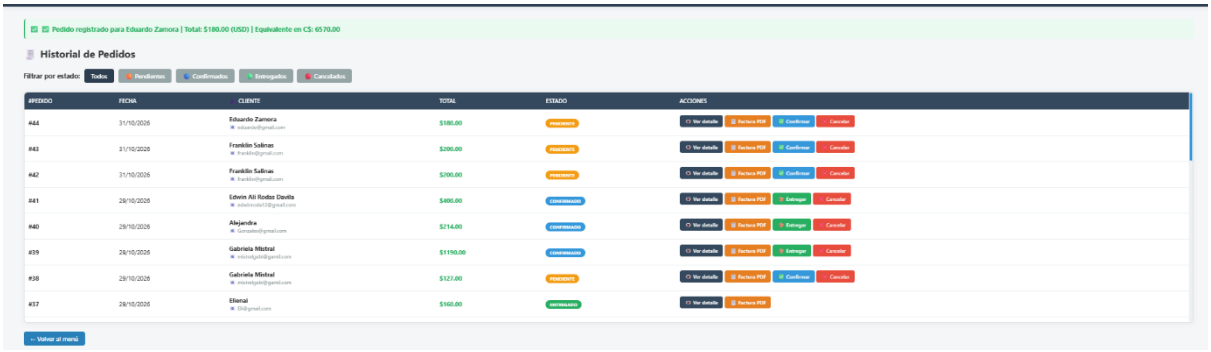
Visualización de los clientes



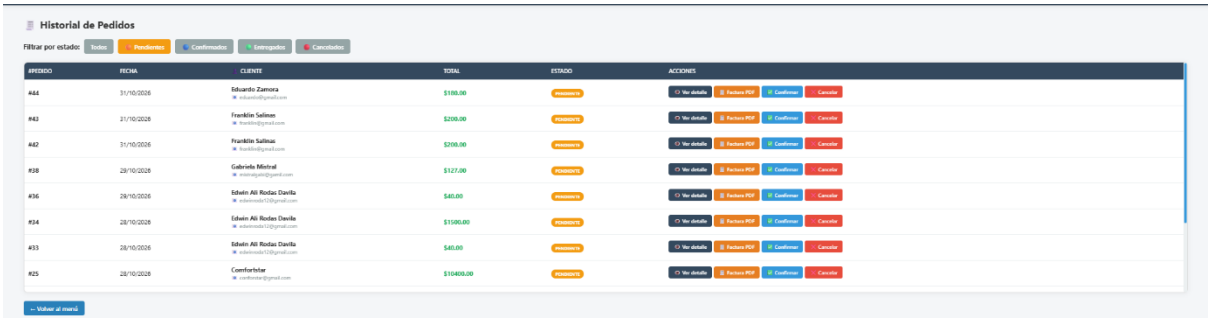
Visualización del carrito



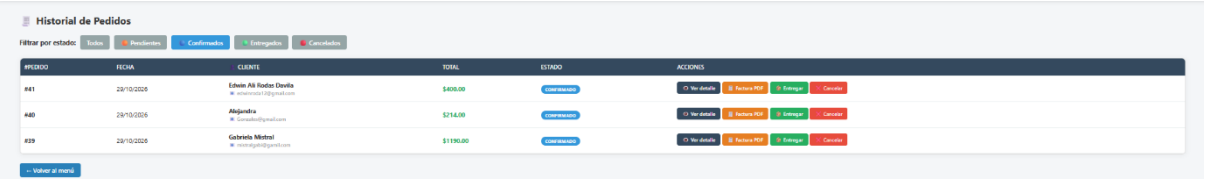
Visualización del historial de pedidos de los clientes



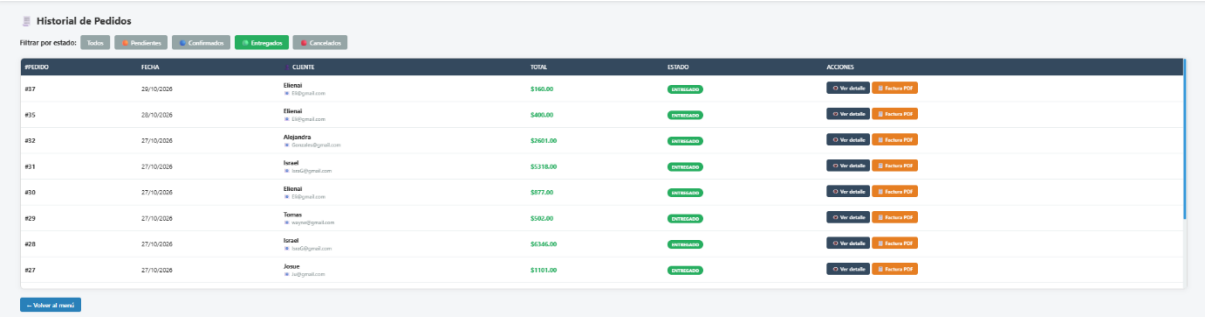
Historial de pedidos “Pendiente”



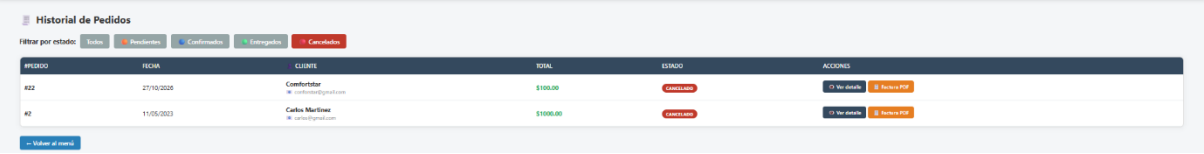
Historial de pedidos “Confirmación”



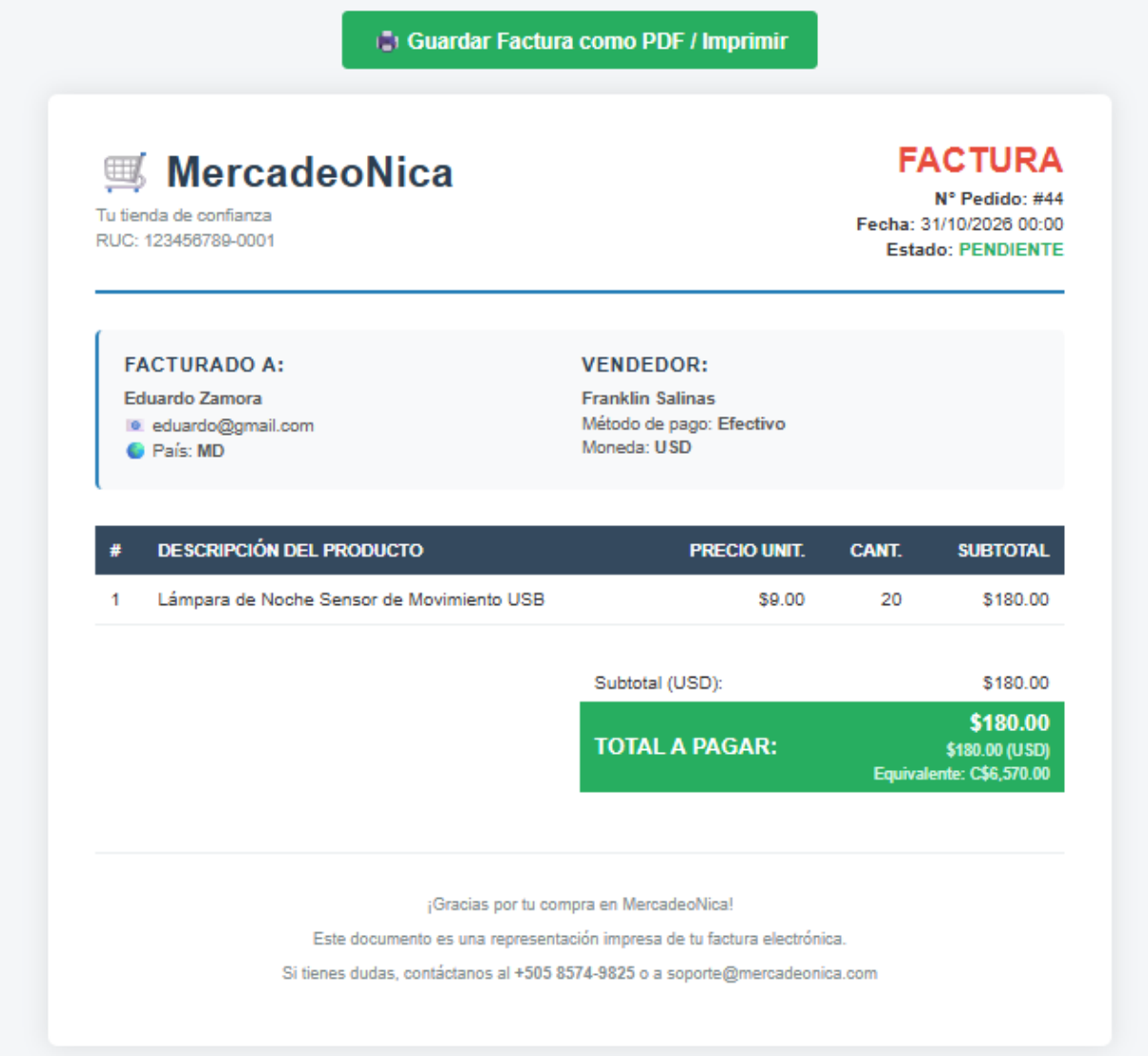
Historial de pedidos “Entregados”



Historial de pedidos “Cancelados”

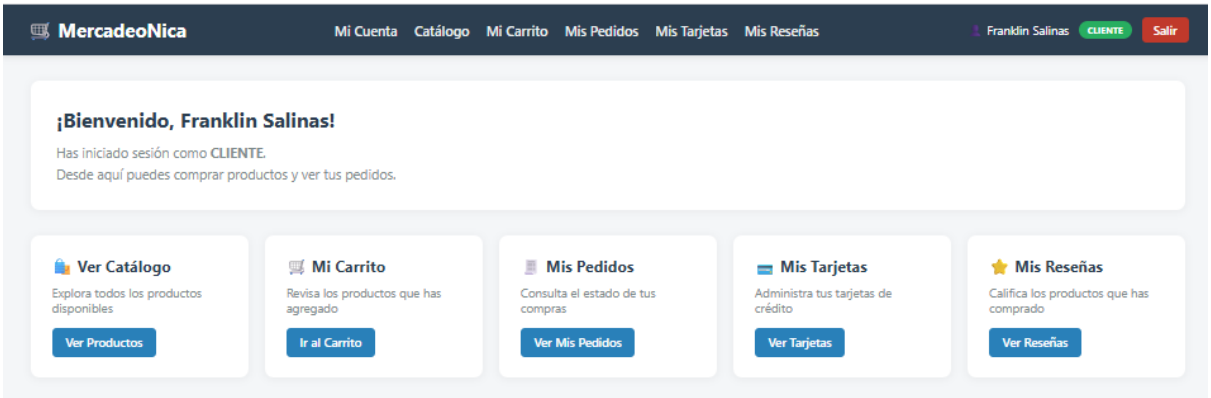


Visualización de la factura del cliente

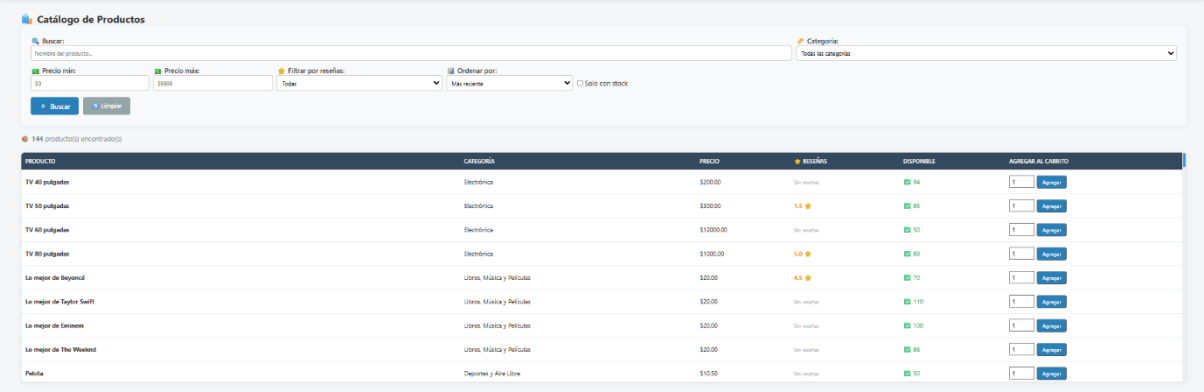


A nivel de Cliente

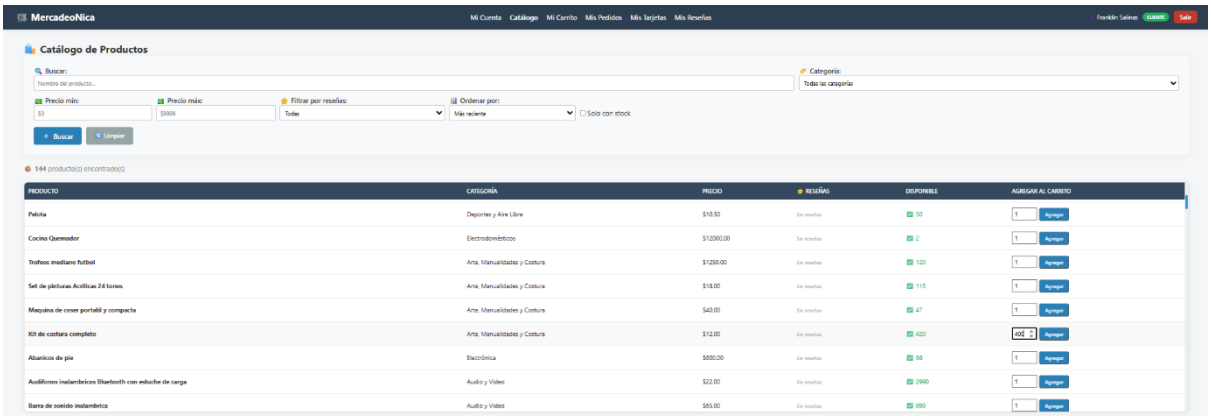
Visualización del panel de control



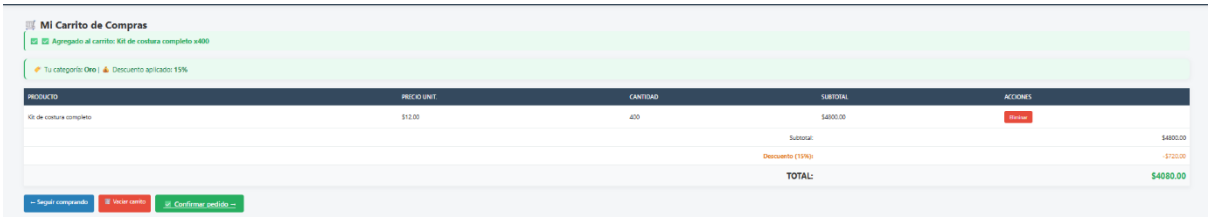
Visualización de los productos



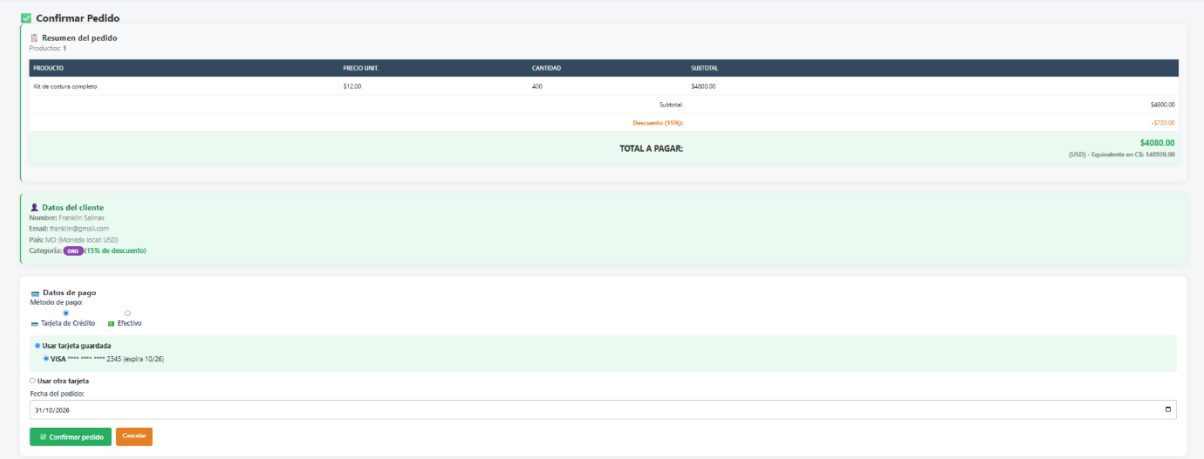
Visualización de compra



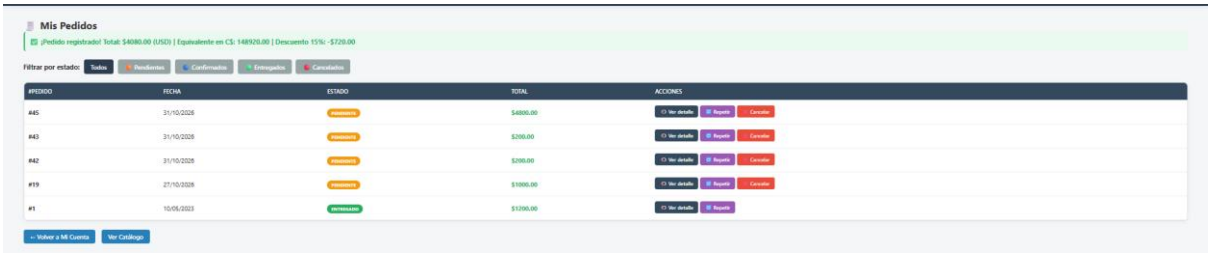
Visualización del carrito de compra



Visualización de la confirmación del pedido



Visualización del pedido del cliente que ha realizado.



Visualización de las tarjetas de crédito que el cliente ingrese.

